

Catatan Ujian Akhir Semester

IF2010 Pemrograman Berbasis Objek

Semester 4 Tahun 2025/2026



NARENDRA DHARMA WISTARA M.

13524044

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG

4 Juni 2026

Daftar Isi

1	Informasi Ujian	1
1.1	Capaian Pembelajaran Mata Kuliah	1
1.2	Keterpenuhan CPMK terhadap Materi yang Dibahas	1
2	Daftar Materi yang Diujikan	1
3	Outline Keseluruhan Materi	2
3.1	Prinsip OOP SOLID	2
3.2	Pengenalan Bahasa Java	2
3.3	Padanan Java - C++	2
3.4	Java String	2
3.5	Java Inheritance	2
3.6	Java Interface	2
3.7	Java Generics	2
3.8	Java Generics Wildcard	3
3.9	Java Collection	3
3.10	Java Stream API	3
3.11	Java Exception	3
3.12	Java Assertion	3
3.13	Java Multithreading	3
3.14	Java Reflection	3
3.15	Design Pattern	3
4	SOLID OOP Principle	4
4.1	Outline Fundamental Konsep Materi	4
4.2	Pentingnya Materi dan Relevansinya	5
4.2.1	Dependensi dan Hubungan dengan Materi Lain	6
4.3	Penjelasan Konsep Fundamental	6
4.3.1	<i>Bad Design</i> : Rigidity, Fragility, dan Immobility	6
4.3.2	Single Responsibility Principle (SRP)	7
4.3.3	Open/Closed Principle (OCP)	8
4.3.4	Liskov Substitution Principle (LSP)	9
4.3.5	Interface Segregation Principle (ISP)	11
4.3.6	Dependency Inversion Principle (DIP)	12
4.3.7	Package Principles, Coupling, Stability, dan Abstractness	13
4.4	Komponen Kunci Penting	15
4.5	Algoritma dan Rumus Penting	15
4.5.1	Metrik Instabilitas Paket	15
4.5.2	Rasio Abstractness	16
4.5.3	Prosedur Identifikasi Pelanggaran SOLID	16
4.6	Checklist Pemahaman	17
4.6.1	Submateri yang Telah Dibahas	17
4.6.2	Prioritas Belajar	17
5	Pengenalan Bahasa Java	18
5.1	Outline Fundamental Konsep Materi	18
5.2	Pentingnya Materi dan Relevansinya	19
5.2.1	Dependensi dan Hubungan dengan Materi Lain	20
5.3	Penjelasan Konsep Fundamental	20
5.3.1	Karakteristik Dasar Java	20
5.3.2	Struktur Minimal Program Java	20
5.3.3	Class, Object, Field, Method, dan Constructor	21

5.3.4	Type Primitif dan Tipe Referensi	22
5.3.5	Variabel, Scope, Assignment, dan Casting	23
5.3.6	Access Modifier dan Enkapsulasi	24
5.3.7	static , Instance Member, dan final	25
5.3.8	Control Flow: Percabangan dan Perulangan	25
5.3.9	Package, Import, dan Organisasi Kode	26
5.3.10	Kompilasi dan Eksekusi Program Java	27
5.3.11	Hubungan Java dengan OOP	27
5.4	Komponen Kunci Penting	29
5.5	Algoritma dan Rumus Penting	29
5.5.1	Alur Eksekusi Program Java	29
5.5.2	Formula Tanda Tangan Method	30
5.6	Checklist Pemahaman	30
5.6.1	Submateri yang Telah Dibahas	30
5.6.2	Prioritas Belajar	31
6	Padanan Java - C++	32
6.1	Outline Fundamental Konsep Materi	32
6.2	Pentingnya Materi dan Relevansinya	32
6.2.1	Dependensi dan Hubungan dengan Materi Lain	33
6.3	Penjelasan Konsep Fundamental	33
6.3.1	Model Kompilasi dan Eksekusi	33
6.3.2	Object, Pointer, Reference, dan Nilai	33
6.3.3	Memory Management dan Resource Management	34
6.3.4	Inheritance dan Interface	34
6.3.5	Header-Include vs Package-Import	35
6.3.6	Generics Java dan Template C++	35
6.4	Komponen Kunci Penting	36
6.5	Algoritma dan Rumus Penting	36
6.5.1	Prosedur Memetakan Konsep C++ ke Java	36
6.6	Checklist Pemahaman	36
6.6.1	Submateri yang Telah Dibahas	36
6.6.2	Prioritas Belajar	37
7	Java String	38
7.1	Outline Fundamental Konsep Materi	38
7.2	Pentingnya Materi dan Relevansinya	38
7.2.1	Dependensi dan Hubungan dengan Materi Lain	39
7.3	Penjelasan Konsep Fundamental	39
7.3.1	Immutability String	39
7.3.2	String Pool dan Perbandingan	39
7.3.3	Operasi String yang Sering Dipakai	40
7.3.4	StringBuilder dan StringBuffer	40
7.3.5	Parsing, Formatting, dan Regex	41
7.3.6	Unicode dan char	41
7.4	Komponen Kunci Penting	41
7.5	Algoritma dan Rumus Penting	42
7.5.1	Prosedur Memilih Operasi String	42
7.6	Checklist Pemahaman	42
7.6.1	Submateri yang Telah Dibahas	42
7.6.2	Prioritas Belajar	42

8	Java Inheritance	43
8.1	Outline Fundamental Konsep Materi	43
8.2	Pentingnya Materi dan Relevansinya	43
8.2.1	Dependensi dan Hubungan dengan Materi Lain	44
8.3	Penjelasan Konsep Fundamental	44
8.3.1	Relasi <i>is-a</i> dan Pewarisan Class	44
8.3.2	Constructor Chaining dan super	45
8.3.3	Overriding, Overloading, dan Dynamic Dispatch	45
8.3.4	Upcasting, Downcasting, dan instanceof	45
8.3.5	Abstract Class dan Final	46
8.3.6	LSP dan Risiko Inheritance yang Salah	46
8.4	Komponen Kunci Penting	47
8.5	Algoritma dan Rumus Penting	47
8.5.1	Prosedur Validasi Relasi Inheritance	47
8.6	Checklist Pemahaman	47
8.6.1	Submateri yang Telah Dibahas	47
8.6.2	Prioritas Belajar	48
9	Java Interface	49
9.1	Outline Fundamental Konsep Materi	49
9.2	Pentingnya Materi dan Relevansinya	49
9.2.1	Dependensi dan Hubungan dengan Materi Lain	50
9.3	Penjelasan Konsep Fundamental	50
9.3.1	Interface sebagai Kontrak	50
9.3.2	Default, Static, dan Private Method	51
9.3.3	Multiple Interface dan Konflik Default Method	51
9.3.4	Functional Interface dan Lambda	52
9.3.5	Interface, ISP, dan DIP	52
9.4	Komponen Kunci Penting	53
9.5	Algoritma dan Rumus Penting	53
9.5.1	Prosedur Mendesain Interface	53
9.6	Checklist Pemahaman	53
9.6.1	Submateri yang Telah Dibahas	53
9.6.2	Prioritas Belajar	54
10	Java Generics	55
10.1	Outline Fundamental Konsep Materi	55
10.2	Pentingnya Materi dan Relevansinya	55
10.2.1	Dependensi dan Hubungan dengan Materi Lain	56
10.3	Penjelasan Konsep Fundamental	56
10.3.1	Generic Class	56
10.3.2	Generic Method	56
10.3.3	Bounded Type Parameter	57
10.3.4	Type Erasure	57
10.3.5	Raw Type dan Heap Pollution	57
10.4	Komponen Kunci Penting	58
10.5	Algoritma dan Rumus Penting	58
10.5.1	Prosedur Menentukan Parameter Generics	58
10.6	Checklist Pemahaman	58
10.6.1	Submateri yang Telah Dibahas	58
10.6.2	Prioritas Belajar	59

11 Java Generics Wildcard	60
11.1 Outline Fundamental Konsep Materi	60
11.2 Pentingnya Materi dan Relevansinya	60
11.2.1 Dependensi dan Hubungan dengan Materi Lain	61
11.3 Penjelasan Konsep Fundamental	61
11.3.1 Invariance pada Generic Java	61
11.3.2 Unbounded Wildcard	61
11.3.3 Upper Bounded Wildcard: ? extends T	61
11.3.4 Lower Bounded Wildcard: ? super T	62
11.3.5 PECS: Producer Extends, Consumer Super	62
11.4 Komponen Kunci Penting	63
11.5 Algoritma dan Rumus Penting	63
11.5.1 Prosedur Memilih Wildcard	63
11.6 Checklist Pemahaman	63
11.6.1 Submateri yang Telah Dibahas	63
11.6.2 Prioritas Belajar	63
12 Java Collection	65
12.1 Outline Fundamental Konsep Materi	65
12.2 Pentingnya Materi dan Relevansinya	66
12.2.1 Dependensi dan Hubungan dengan Materi Lain	66
12.3 Penjelasan Konsep Fundamental	66
12.3.1 List	66
12.3.2 Set	66
12.3.3 Map	67
12.3.4 Equals dan hashCode	67
12.3.5 Comparable dan Comparator	67
12.4 Komponen Kunci Penting	68
12.5 Algoritma dan Rumus Penting	68
12.5.1 Prosedur Memilih Collection	68
12.5.2 Kompleksitas Umum	68
12.6 Checklist Pemahaman	69
12.6.1 Submateri yang Telah Dibahas	69
12.6.2 Prioritas Belajar	69
13 Java Stream API	70
13.1 Outline Fundamental Konsep Materi	70
13.2 Pentingnya Materi dan Relevansinya	71
13.2.1 Dependensi dan Hubungan dengan Materi Lain	71
13.3 Penjelasan Konsep Fundamental	71
13.3.1 Pipeline Stream	71
13.3.2 Lazy Evaluation	71
13.3.3 Map vs FlatMap	72
13.3.4 Reduce	72
13.3.5 Collect dan Grouping	72
13.3.6 Side Effect dan Parallel Stream	73
13.4 Komponen Kunci Penting	73
13.5 Algoritma dan Rumus Penting	73
13.5.1 Template Pipeline Stream	73
13.6 Checklist Pemahaman	74
13.6.1 Submateri yang Telah Dibahas	74
13.6.2 Prioritas Belajar	74

14 Java Exception	75
14.1 Outline Fundamental Konsep Materi	75
14.2 Pentingnya Materi dan Relevansinya	76
14.2.1 Dependensi dan Hubungan dengan Materi Lain	76
14.3 Penjelasan Konsep Fundamental	76
14.3.1 Hierarki Exception	76
14.3.2 Checked dan Unchecked Exception	76
14.3.3 <code>try</code> , <code>catch</code> , dan <code>finally</code>	77
14.3.4 <code>try-with-resources</code>	77
14.3.5 <code>throw</code> dan <code>throws</code>	78
14.3.6 Custom Exception	78
14.4 Komponen Kunci Penting	79
14.5 Algoritma dan Rumus Penting	79
14.5.1 Prosedur Memilih Jenis Exception	79
14.6 Checklist Pemahaman	79
14.6.1 Submateri yang Telah Dibahas	79
14.6.2 Prioritas Belajar	80
15 Java Assertion	81
15.1 Outline Fundamental Konsep Materi	81
15.2 Pentingnya Materi dan Relevansinya	81
15.2.1 Dependensi dan Hubungan dengan Materi Lain	82
15.3 Penjelasan Konsep Fundamental	82
15.3.1 Sintaks Dasar Assertion	82
15.3.2 Mengaktifkan Assertion	82
15.3.3 Assertion vs Exception	82
15.4 Komponen Kunci Penting	83
15.5 Algoritma dan Rumus Penting	83
15.5.1 Prosedur Menentukan Assert atau Exception	83
15.6 Checklist Pemahaman	83
15.6.1 Submateri yang Telah Dibahas	83
15.6.2 Prioritas Belajar	84
16 Java Multithreading	85
16.1 Outline Fundamental Konsep Materi	85
16.2 Pentingnya Materi dan Relevansinya	86
16.2.1 Dependensi dan Hubungan dengan Materi Lain	86
16.3 Penjelasan Konsep Fundamental	86
16.3.1 Thread, Runnable, dan Executor	86
16.3.2 Race Condition	86
16.3.3 <code>synchronized</code> dan Atomic	87
16.3.4 <code>volatile</code> dan Visibility	87
16.3.5 Deadlock	88
16.4 Komponen Kunci Penting	88
16.5 Algoritma dan Rumus Penting	89
16.5.1 Prosedur Mendeteksi Risiko Race Condition	89
16.6 Checklist Pemahaman	89
16.6.1 Submateri yang Telah Dibahas	89
16.6.2 Prioritas Belajar	89

17 Java Reflection	90
17.1 Outline Fundamental Konsep Materi	90
17.2 Pentingnya Materi dan Relevansinya	90
17.2.1 Dependensi dan Hubungan dengan Materi Lain	91
17.3 Penjelasan Konsep Fundamental	91
17.3.1 Class Object	91
17.3.2 Inspeksi Method dan Field	91
17.3.3 Membuat Object dan Memanggil Method	92
17.3.4 Mengakses Private Member	92
17.3.5 Annotation Runtime	92
17.4 Komponen Kunci Penting	93
17.5 Algoritma dan Rumus Penting	93
17.5.1 Prosedur Menggunakan Reflection	93
17.6 Checklist Pemahaman	93
17.6.1 Submateri yang Telah Dibahas	93
17.6.2 Prioritas Belajar	94
18 Design Pattern	95
18.1 Outline Fundamental Konsep Materi	95
18.2 Pentingnya Materi dan Relevansinya	95
18.2.1 Dependensi dan Hubungan dengan Materi Lain	96
18.3 Penjelasan Konsep Fundamental	96
18.3.1 Strategy Pattern	96
18.3.2 Factory Method	97
18.3.3 Builder Pattern	98
18.3.4 Adapter Pattern	99
18.3.5 Decorator Pattern	100
18.3.6 Observer Pattern	100
18.3.7 Template Method	101
18.3.8 Singleton Pattern	102
18.4 Komponen Kunci Penting	102
18.5 Algoritma dan Rumus Penting	103
18.5.1 Prosedur Memilih Design Pattern	103
18.6 Checklist Pemahaman	103
18.6.1 Submateri yang Telah Dibahas	103
18.6.2 Prioritas Belajar	103

1 Informasi Ujian

Jenis Ujian	UAS
Nama Mata Kuliah	IF2010 Pemrograman Berbasis Objek
Tanggal Ujian	Kamis, 4 Juni 2026
Waktu Tersisa	18 jam

Tabel 1: Informasi ujian

1.1 Capaian Pembelajaran Mata Kuliah

Tidak disebutkan dalam perkuliahan

1.2 Keterpenuhan CPMK terhadap Materi yang Dibahas

CPMK	Materi yang Dibahas	Keterpenuhan
Tidak disebutkan dalam perkuliahan	Seluruh materi UAS yang telah disusun, yaitu Prinsip OOP SOLID sampai Design Pattern	Tidak dapat dipetakan secara formal karena CPMK tidak disebutkan dalam perkuliahan. Materi tetap disusun berdasarkan daftar materi UAS dan sumber utama apabila tersedia, lalu dilengkapi pengetahuan umum yang ditandai.

Tabel 2: Keterpenuhan CPMK terhadap materi yang telah disusun

Komentar 1.1: Keterbatasan Sumber CPMK

CPMK disalin sesuai isian yang tersedia, yaitu “Tidak disebutkan dalam perkuliahan”. Karena tidak ada CPMK formal, hubungan materi dengan CPMK tidak dibuat sebagai klaim capaian resmi, melainkan hanya sebagai peta belajar berdasarkan daftar materi UAS.

2 Daftar Materi yang Diujikan

1. Prinsip OOP SOLID
2. Pengenalan Bahasa Java
3. Padanan Java - C++
4. Java String
5. Java Inheritance
6. Java Interface
7. Java Generics
8. Java Generics Wildcard
9. Java Collection
10. Java Stream API

11. Java Exception
12. Java Assertion
13. Java Multithreading
14. Java Reflection
15. Design Pattern

3 Outline Keseluruhan Materi

Bagian ini menyajikan peta belajar ringkas untuk seluruh materi yang diujikan. Penjelasan mendalam tersedia pada section masing-masing materi yang sudah disusun.

3.1 Prinsip OOP SOLID

- Memahami masalah desain buruk: *rigidity*, *fragility*, dan *immobility*.
- Menguasai lima prinsip SOLID: SRP, OCP, LSP, ISP, dan DIP.
- Mampu membaca cuplikan kode Java, mengidentifikasi pelanggaran prinsip, dan mengusulkan refactoring berbasis abstraksi.
- Memahami hubungan SOLID dengan package principles, coupling, stability, dan abstraction.

3.2 Pengenalan Bahasa Java

- Sintaks dasar Java, struktur program, tipe data, class, object, method, dan akses modifier.
- Perbedaan model kompilasi Java dengan bahasa lain melalui bytecode dan JVM.

3.3 Padanan Java - C++

- Perbandingan class, inheritance, pointer/reference, memory management, generic/template, dan exception.
- Batasan Java seperti single inheritance untuk class dan penggunaan interface untuk multiple typing.

3.4 Java String

- Sifat immutable, string pool, operasi perbandingan, dan penggunaan `StringBuilder`.

3.5 Java Inheritance

- Relasi *is-a*, overriding, constructor chaining, polymorphism, dan risiko desain hierarki yang salah.

3.6 Java Interface

- Kontrak perilaku, implementasi banyak interface, default method, static method, dan hubungan dengan DIP/ISP.

3.7 Java Generics

- Type parameter, type safety, generic class, generic method, bounded type parameter, dan type erasure.

3.8 Java Generics Wildcard

- Wildcard `?`, upper bounded wildcard, lower bounded wildcard, dan prinsip PECS.

3.9 Java Collection

- Struktur `List`, `Set`, `Map`, iterator, ordering, hashing, dan pemilihan collection.

3.10 Java Stream API

- Pipeline stream, intermediate operation, terminal operation, lazy evaluation, dan operasi transformasi data.

3.11 Java Exception

- Checked exception, unchecked exception, `try-catch-finally`, `throw`, `throws`, dan desain error handling.

3.12 Java Assertion

- Penggunaan `assert` untuk memeriksa asumsi internal program selama debugging.

3.13 Java Multithreading

- Thread, runnable task, race condition, synchronization, executor, dan koordinasi eksekusi paralel.

3.14 Java Reflection

- Inspeksi metadata class saat runtime, pemanggilan method dinamis, dan risiko terhadap enkapsulasi.

3.15 Design Pattern

- Pola desain umum sebagai solusi berulang untuk masalah desain OOP, serta hubungannya dengan SOLID.

4 SOLID OOP Principle

SOLID adalah kumpulan lima prinsip desain berorientasi objek yang membantu kode tetap mudah diubah, mudah diuji, mudah diperluas, dan tidak cepat menjadi rapuh ketika kebutuhan bertambah. Dalam konteks IF2010 Pemrograman Berbasis Objek, materi ini bukan sekadar hafalan akronim, melainkan kemampuan membaca desain class, memahami arah dependensi, lalu menjelaskan mengapa suatu desain akan sulit dirawat atau mudah diperbaiki.

Sumber NotebookLM menekankan bahwa SOLID diperlukan untuk menghindari *bad design*, terutama tiga gejala utama: *rigidity*, *fragility*, dan *immobility*. Catatan ini melengkapi pembahasan tersebut dengan sintaks Java dan contoh implementasi dunia nyata agar prinsip dapat diterapkan saat membaca atau menulis kode.

Komentar 4.1: Keterbatasan Sumber

NotebookLM dapat diakses dan memberikan rangkuman berbasis dokumen. Namun, sumber menyatakan bahwa tidak ada CPMK formal yang disebutkan, serta pembahasan DIP pada sumber awal lebih terbatas daripada SRP, OCP, LSP, dan ISP. Bagian sintaks Java, refactoring praktis, formula instabilitas, dan contoh industri dilengkapi menggunakan pengetahuan umum untuk memperjelas penerapan.

4.1 Outline Fundamental Konsep Materi

1. *Bad design* sebagai masalah yang dicegah SOLID **[SUDAH DIKUASAI]**
 - (a) *Rigidity*: sistem sulit diubah karena perubahan kecil menjalar ke banyak bagian.
 - (b) *Fragility*: perubahan di satu tempat menyebabkan kerusakan tak terduga di tempat lain.
 - (c) *Immobility*: komponen sulit digunakan ulang karena terlalu bergantung pada konteks aplikasi asal.
2. Single Responsibility Principle (SRP) **[SUDAH DIKUASAI]**
 - (a) Satu class memiliki satu alasan utama untuk berubah.
 - (b) Pemisahan domain logic, persistence, formatting, logging, dan komunikasi eksternal.
 - (c) Indikasi pelanggaran: class terlalu besar, method terlalu banyak, nama class terlalu umum, dan perubahan dari banyak aktor bisnis.
3. Open/Closed Principle (OCP) **[SUDAH DIKUASAI]**
 - (a) Terbuka untuk ekstensi, tertutup untuk modifikasi.
 - (b) Penggunaan polymorphism, interface, abstract class, composition, dan strategy.
 - (c) Menghindari percabangan tipe yang terus bertambah.
4. Liskov Substitution Principle (LSP) **[SUDAH DIKUASAI]**
 - (a) Subclass harus dapat menggantikan superclass tanpa merusak kebenaran program.
 - (b) Kontrak perilaku lebih penting daripada kemiripan konseptual di dunia nyata.
 - (c) Precondition tidak boleh diperketat, postcondition tidak boleh dilemahkan, invariant harus dipertahankan.
5. Interface Segregation Principle (ISP) **[SUDAH DIKUASAI]**
 - (a) Klien tidak boleh dipaksa bergantung pada method yang tidak digunakan.
 - (b) Interface kecil dan spesifik lebih baik daripada *fat interface*.

(c) Di Java, multiple inheritance perilaku dilakukan melalui banyak interface.

6. Dependency Inversion Principle (DIP) **[SUDAH DIKUASAI]**

- (a) Modul tingkat tinggi bergantung pada abstraksi, bukan implementasi konkret.
- (b) Detail implementasi bergantung pada kontrak abstrak.
- (c) Dependency injection membantu memasukkan implementasi konkret dari luar.

7. Package principles dan metrik desain

- (a) Prinsip kohesi paket: REP, CCP, dan CRP.
- (b) Prinsip coupling paket: ADP, SDP, dan SAP.
- (c) Coupling masuk, coupling keluar, instabilitas, dan abstractness.

8. Strategi menjawab soal desain

- (a) Identifikasi gejala desain buruk.
- (b) Tentukan prinsip SOLID yang dilanggar.
- (c) Jelaskan akibat praktisnya.
- (d) Usulkan refactoring Java yang menjaga kontrak dan mengurangi coupling.

4.2 Pentingnya Materi dan Relevansinya

Inti Relevansi.

SOLID penting karena ujian OOP sering menguji kemampuan menilai desain, bukan hanya kemampuan menulis sintaks. Prinsip ini memberi bahasa teknis untuk menjelaskan mengapa suatu class, interface, atau hierarki inheritance mudah rusak atau mudah diperluas.

Dalam proyek nyata, perubahan kebutuhan adalah hal yang normal. Aplikasi yang awalnya hanya memiliki satu jenis pembayaran dapat bertambah menjadi pembayaran kartu, transfer bank, dompet digital, dan cicilan. Sistem yang awalnya hanya menggambar lingkaran dan persegi dapat bertambah menjadi segitiga, poligon, dan gambar kompleks. Tanpa prinsip desain yang baik, setiap penambahan fitur membuat programmer harus membuka ulang kode lama, menambah percabangan baru, dan mengambil risiko merusak perilaku yang sebelumnya sudah benar.

SOLID membantu mengendalikan risiko tersebut dengan menempatkan perubahan pada lokasi yang tepat. SRP membatasi alasan perubahan pada satu unit. OCP membuat fitur baru dapat ditambahkan lewat class baru. LSP memastikan inheritance tidak menghasilkan substitusi yang menipu. ISP mencegah klien bergantung pada operasi yang tidak relevan. DIP membalik arah dependensi sehingga policy utama tidak dikunci oleh detail implementasi.

Komentar 4.2: Relevansi Industri

Dalam pengembangan backend Java, SOLID sering muncul pada service layer, repository, payment gateway, notification system, parser, validator, dan integrasi API. Prinsip ini tidak berarti semua kode harus penuh interface sejak awal. Penerapan yang realistis adalah menggunakan abstraksi ketika ada variasi perilaku, kebutuhan testing, pergantian implementasi, atau boundary eksternal yang sering berubah.

4.2.1 Dependensi dan Hubungan dengan Materi Lain

SOLID berhubungan langsung dengan Java Inheritance, Java Interface, Java Generics, Java Collection, Java Stream API, Java Exception, dan Design Pattern. Inheritance digunakan dalam OCP dan LSP, tetapi inheritance yang salah dapat merusak substitutability. Interface adalah alat utama untuk ISP dan DIP. Generics membantu membuat abstraksi tetap type-safe. Collection dan Stream API sering menjadi lokasi penerapan SRP dan DIP pada pengolahan data. Exception berkaitan dengan kontrak method karena subclass tidak boleh menghasilkan perilaku error yang mengejutkan klien. Design pattern seperti Strategy, Factory, Adapter, Observer, dan Template Method adalah bentuk penerapan praktis dari prinsip SOLID.

Komentar 4.3: Hubungan dengan Materi Lanjut

SOLID bukan pengganti design pattern. SOLID adalah prinsip evaluasi desain, sedangkan design pattern adalah pola solusi. Sebuah pattern dapat membantu memenuhi SOLID, tetapi pattern yang dipakai berlebihan juga dapat membuat desain terlalu kompleks.

4.3 Penjelasan Konsep Fundamental

4.3.1 *Bad Design*: Rigidity, Fragility, dan Immobility

Inti Konsep.

Bad design adalah desain perangkat lunak yang sulit diubah, mudah rusak, dan sulit digunakan ulang. SOLID muncul untuk menurunkan risiko tiga gejala utama: *rigidity*, *fragility*, dan *immobility*.

Rigidity berarti sistem kaku: satu perubahan kecil membutuhkan perubahan lanjutan di banyak class lain. Contohnya, ketika sistem pembayaran menambahkan metode baru, programmer harus mengubah controller, service, validator, invoice generator, dan report generator karena semua bagian mengenal detail tipe pembayaran. Masalahnya bukan hanya jumlah perubahan, tetapi juga kenyataan bahwa perubahan tersebut tersebar di banyak tempat sehingga risiko lupa mengubah satu lokasi menjadi tinggi.

Fragility berarti sistem rapuh: perubahan di satu bagian menyebabkan error pada bagian lain yang secara mental tidak terlihat berhubungan. Misalnya, mengubah format penyimpanan data customer ternyata merusak fitur email karena class email membaca langsung field internal customer. Fragility biasanya muncul karena coupling tinggi dan pelanggaran enkapsulasi.

Immobility berarti sistem sulit digunakan ulang. Sebuah class mungkin tampak berguna, tetapi ternyata bergantung pada database tertentu, format konfigurasi tertentu, logger tertentu, atau framework tertentu. Akibatnya, class tersebut tidak dapat dipindahkan ke aplikasi lain tanpa membawa banyak dependensi yang tidak relevan.

Komentar 4.4: Cara Membaca Gejala Desain

Saat melihat kode ujian, cari tanda seperti percabangan `if/switch` berdasarkan tipe objek, class bernama terlalu umum seperti `Manager` atau `Processor`, interface yang memaksa method kosong, subclass yang melempar `UnsupportedOperationException`, dan constructor yang membuat dependency konkret dengan `new`. Tanda tersebut sering mengarah ke pelanggaran SOLID.

4.3.2 Single Responsibility Principle (SRP)

Inti Konsep.

SRP menyatakan bahwa sebuah class sebaiknya hanya memiliki satu alasan untuk berubah. “Satu tanggung jawab” berarti satu sumbu perubahan, bukan sekadar satu method.

SRP sering disalahpahami sebagai aturan bahwa setiap class harus sangat kecil. Makna yang lebih tepat adalah sebuah class harus mewakili satu konsep atau satu aktor perubahan. Jika perubahan aturan bisnis, perubahan format tampilan, dan perubahan cara penyimpanan data semuanya memaksa class yang sama berubah, class tersebut kemungkinan melanggar SRP. Class seperti itu menjadi pusat terlalu banyak keputusan sehingga sulit diuji, sulit dipahami, dan mudah rusak ketika salah satu aspek berubah.

Contoh pelanggaran SRP dalam Java:

Listing 1: Pelanggaran SRP: domain, persistence, dan formatting tercampur

```
1 public class Invoice {
2     private final List<LineItem> items;
3
4     public Invoice(List<LineItem> items) {
5         this.items = items;
6     }
7
8     public BigDecimal total() {
9         return items.stream()
10             .map(LineItem::amount)
11             .reduce(BigDecimal.ZERO, BigDecimal::add);
12     }
13
14     public void saveToDatabase(Connection connection) throws SQLException {
15         PreparedStatement statement =
16             connection.prepareStatement("INSERT INTO invoices(total) VALUES (?)");
17         statement.setBigDecimal(1, total());
18         statement.executeUpdate();
19     }
20
21     public String toHtml() {
22         return "<h1>Invoice</h1><p>Total: " + total() + "</p>";
23     }
24 }
```

Pada contoh tersebut, `Invoice` berubah ketika aturan total berubah, ketika schema database berubah, dan ketika tampilan HTML berubah. Refactoring yang lebih baik adalah memisahkan tanggung jawab:

Listing 2: Penerapan SRP: tanggung jawab dipisahkan

```
1 public class Invoice {
2     private final List<LineItem> items;
3
4     public Invoice(List<LineItem> items) {
5         this.items = List.copyOf(items);
6     }
7
8     public BigDecimal total() {
9         return items.stream()
10             .map(LineItem::amount)
11             .reduce(BigDecimal.ZERO, BigDecimal::add);
12     }
13 }
14
15 public class InvoiceRepository {
```

```

16     public void save(Invoice invoice) {
17         // Simpan invoice ke storage.
18     }
19 }
20
21 public class InvoiceHtmlRenderer {
22     public String render(Invoice invoice) {
23         return "<h1>Invoice</h1><p>Total: " + invoice.total() + "</p>";
24     }
25 }

```

Pemisahan tersebut membuat perubahan tampilan tidak mengganggu perhitungan total, dan perubahan penyimpanan tidak mengubah objek domain. Untuk ujian, argumen pentingnya adalah: SRP mengurangi alasan perubahan pada satu class, sehingga menurunkan rigidity dan fragility.

4.3.3 Open/Closed Principle (OCP)

Inti Konsep.

OCP menyatakan bahwa entitas perangkat lunak harus terbuka untuk ekstensi, tetapi tertutup untuk modifikasi. Fitur baru sebaiknya ditambahkan dengan class atau implementasi baru, bukan dengan terus mengubah kode lama yang sudah stabil.

Sumber NotebookLM menekankan contoh `GraphicEditor`. Desain buruk terjadi ketika editor menggambar bentuk dengan memeriksa tipe bentuk melalui percabangan. Setiap kali bentuk baru ditambahkan, class editor harus dimodifikasi. Ini membuat class tersebut menjadi pusat perubahan dan melanggar OCP.

Listing 3: Pelanggaran OCP: percabangan tipe terus bertambah

```

1 public class GraphicEditor {
2     public void draw(Object shape) {
3         if (shape instanceof Rectangle rectangle) {
4             drawRectangle(rectangle);
5         } else if (shape instanceof Circle circle) {
6             drawCircle(circle);
7         }
8     }
9
10    private void drawRectangle(Rectangle rectangle) { }
11    private void drawCircle(Circle circle) { }
12 }

```

Desain yang lebih baik adalah memindahkan variasi perilaku ke abstraksi:

Listing 4: Penerapan OCP: variasi perilaku lewat polymorphism

```

1 public interface Shape {
2     void draw(Canvas canvas);
3 }
4
5 public class Rectangle implements Shape {
6     @Override
7     public void draw(Canvas canvas) {
8         canvas.drawRectangle(this);
9     }
10 }
11
12 public class Circle implements Shape {
13     @Override
14     public void draw(Canvas canvas) {
15         canvas.drawCircle(this);
16     }
17 }

```

```

17 }
18
19 public class GraphicsEditor {
20     public void drawAll(List<Shape> shapes, Canvas canvas) {
21         for (Shape shape : shapes) {
22             shape.draw(canvas);
23         }
24     }
25 }

```

Jika nanti ada **Triangle**, programmer cukup membuat class baru yang mengimplementasikan **Shape**. **GraphicsEditor** tidak perlu berubah selama kontrak **Shape** tetap sama. Inilah makna “tertutup untuk modifikasi”: bukan berarti tidak boleh pernah mengubah kode lama, tetapi perubahan normal untuk variasi baru tidak diarahkan ke class yang sudah stabil.

Komentar 4.5: Batas Wajar OCP

OCP tidak berarti semua kemungkinan variasi harus diabstraksikan sejak awal. Abstraksi terlalu dini membuat kode sulit dibaca. Gunakan OCP ketika variasi perilaku memang sering bertambah, ketika percabangan tipe mulai berulang, atau ketika perubahan di kode lama berisiko tinggi.

4.3.4 Liskov Substitution Principle (LSP)

Inti Konsep.

LSP menyatakan bahwa objek subclass harus dapat menggantikan objek superclass tanpa membuat program menjadi salah. Inheritance harus menjaga kontrak perilaku, bukan hanya hubungan nama atau kemiripan dunia nyata.

LSP adalah prinsip yang paling sering menjebak karena hubungan di dunia nyata belum tentu cocok sebagai inheritance dalam kode. Sumber NotebookLM menekankan contoh **Square** dan **Rectangle**. Secara matematika, persegi adalah persegi panjang. Namun, dalam desain class mutable, **Rectangle** biasanya mengizinkan **setWidth** dan **setHeight** secara independen. **Square** tidak dapat mempertahankan sifat persegi jika kedua setter tersebut benar-benar independen.

Listing 5: Pelanggaran LSP: Square tidak cocok menggantikan Rectangle mutable

```

1 public class Rectangle {
2     protected int width;
3     protected int height;
4
5     public void setWidth(int width) {
6         this.width = width;
7     }
8
9     public void setHeight(int height) {
10        this.height = height;
11    }
12
13    public int area() {
14        return width * height;
15    }
16 }
17
18 public class Square extends Rectangle {
19     @Override
20     public void setWidth(int width) {
21         this.width = width;
22         this.height = width;
23     }

```



```

24
25     @Override
26     public void setHeight(int height) {
27         this.width = height;
28         this.height = height;
29     }
30 }
31
32 public int resizeAndCompute(Rectangle rectangle) {
33     rectangle.setWidth(5);
34     rectangle.setHeight(4);
35     return rectangle.area(); // Klien wajar mengharapkan 20.
36 }

```

Jika `resizeAndCompute(new Square())` menghasilkan 16, program tidak error secara sintaks, tetapi kontrak perilaku klien rusak. Ini pelanggaran LSP. Solusi yang lebih baik adalah tidak memaksa `Square` menjadi subclass `Rectangle` mutable. Gunakan abstraksi yang benar-benar dimiliki keduanya, misalnya `Shape`.

Listing 6: Penerapan LSP: abstraksi perilaku yang aman

```

1 public interface Shape {
2     int area();
3 }
4
5 public final class Rectangle implements Shape {
6     private final int width;
7     private final int height;
8
9     public Rectangle(int width, int height) {
10         this.width = width;
11         this.height = height;
12     }
13
14     @Override
15     public int area() {
16         return width * height;
17     }
18 }
19
20 public final class Square implements Shape {
21     private final int side;
22
23     public Square(int side) {
24         this.side = side;
25     }
26
27     @Override
28     public int area() {
29         return side * side;
30     }
31 }

```

Secara formal, tanda pelanggaran LSP dapat dibaca dari kontrak berikut:

- Precondition subclass tidak boleh lebih ketat daripada superclass. Jika superclass menerima semua nilai positif, subclass tidak boleh tiba-tiba hanya menerima bilangan genap.
- Postcondition subclass tidak boleh lebih lemah. Jika superclass menjanjikan objek tersimpan, subclass tidak boleh hanya mencetak ke layar.
- Invariant superclass harus tetap dijaga. Jika superclass menjamin collection selalu terurut, subclass tidak boleh mengacak urutan.

- Subclass tidak boleh mengejutkan klien dengan exception baru untuk operasi yang secara kontrak valid.

4.3.5 Interface Segregation Principle (ISP)

Inti Konsep.

ISP menyatakan bahwa klien tidak boleh dipaksa bergantung pada method yang tidak digunakan. Interface sebaiknya kecil, spesifik, dan mengikuti kebutuhan klien.

Pelanggaran ISP biasanya muncul pada *fat interface*, yaitu interface yang menggabungkan terlalu banyak kemampuan. Class yang hanya membutuhkan sebagian kemampuan akhirnya dipaksa mengimplementasikan method kosong atau melempar `UnsupportedOperationException`. Ini bukan hanya buruk secara estetika, tetapi juga meningkatkan coupling: klien yang hanya membutuhkan satu operasi ikut terpengaruh ketika operasi lain pada interface berubah.

Listing 7: Pelanggaran ISP: interface terlalu gemuk

```

1 public interface SmartDevice {
2     void turnOn();
3     void turnOff();
4     void connectToWifi();
5     void recordVideo();
6 }
7
8 public class BasicLamp implements SmartDevice {
9     @Override
10    public void turnOn() { }
11
12    @Override
13    public void turnOff() { }
14
15    @Override
16    public void connectToWifi() {
17        throw new UnsupportedOperationException("Lampu biasa tidak punya Wi-Fi");
18    }
19
20    @Override
21    public void recordVideo() {
22        throw new UnsupportedOperationException("Lampu biasa tidak punya kamera");
23    }
24 }

```

Refactoring ISP memecah kontrak menjadi interface yang lebih kecil:

Listing 8: Penerapan ISP: interface spesifik per kemampuan

```

1 public interface Switchable {
2     void turnOn();
3     void turnOff();
4 }
5
6 public interface Networked {
7     void connectToWifi();
8 }
9
10 public interface VideoRecorder {
11     void recordVideo();
12 }
13
14 public class BasicLamp implements Switchable {
15     @Override

```

```

16     public void turnOn() { }
17
18     @Override
19     public void turnOff() { }
20 }
21
22 public class SecurityCamera implements Switchable, Networked, VideoRecorder {
23     @Override
24     public void turnOn() { }
25
26     @Override
27     public void turnOff() { }
28
29     @Override
30     public void connectToWifi() { }
31
32     @Override
33     public void recordVideo() { }
34 }

```

Sumber NotebookLM juga menekankan bahwa di Java, karena class hanya dapat **extends** satu superclass, bentuk multiple inheritance yang relevan untuk ISP dilakukan dengan **implements** banyak interface. Artinya, Java tetap dapat menyusun objek dengan banyak kemampuan tanpa mewarisi implementasi dari banyak class.

4.3.6 Dependency Inversion Principle (DIP)

Inti Konsep.

DIP menyatakan bahwa modul tingkat tinggi tidak boleh bergantung langsung pada modul tingkat rendah; keduanya sebaiknya bergantung pada abstraksi. Detail implementasi harus mengikuti kontrak abstrak, bukan sebaliknya.

Modul tingkat tinggi adalah bagian yang merepresentasikan kebijakan atau aturan penting aplikasi, misalnya proses checkout, registrasi pengguna, atau validasi transaksi. Modul tingkat rendah adalah detail teknis seperti database, email sender, file system, atau API eksternal. Jika modul tingkat tinggi membuat sendiri objek detail konkret, maka perubahan detail teknis akan memaksa perubahan pada logika utama.

Listing 9: Pelanggaran DIP: service bergantung pada detail konkret

```

1 public class OrderService {
2     private final MySQLOrderRepository repository = new MySQLOrderRepository();
3     private final SmtEmailSender emailSender = new SmtEmailSender();
4
5     public void placeOrder(Order order) {
6         repository.save(order);
7         emailSender.send(order.customerEmail(), "Order created");
8     }
9 }

```

Desain tersebut sulit diuji karena test akan ikut bergantung pada database MySQL dan SMTP. Desain juga sulit diubah karena mengganti email menjadi message queue mengharuskan perubahan pada `OrderService`. Dengan DIP, service bergantung pada interface:

Listing 10: Penerapan DIP dengan constructor injection

```

1 public interface OrderRepository {
2     void save(Order order);
3 }
4
5 public interface Notifier {
6     void notify(String recipient, String message);

```

```

7 }
8
9 public class OrderService {
10     private final OrderRepository repository;
11     private final Notifier notifier;
12
13     public OrderService(OrderRepository repository, Notifier notifier) {
14         this.repository = repository;
15         this.notifier = notifier;
16     }
17
18     public void placeOrder(Order order) {
19         repository.save(order);
20         notifier.notify(order.customerEmail(), "Order created");
21     }
22 }
23
24 public class MySqlOrderRepository implements OrderRepository {
25     @Override
26     public void save(Order order) {
27         // Simpan ke MySQL.
28     }
29 }
30
31 public class EmailNotifier implements Notifier {
32     @Override
33     public void notify(String recipient, String message) {
34         // Kirim email.
35     }
36 }

```

Pola di atas disebut *dependency injection*: dependency konkret diberikan dari luar, biasanya melalui constructor. Dalam aplikasi Java modern, framework seperti Spring sering mengotomatisasi injection, tetapi konsep dasarnya tetap sama: class penting tidak membuat sendiri detail yang mudah berubah.

Komentar 4.6: DIP Bukan Sekadar Interface

Menambahkan interface tidak otomatis memenuhi DIP. Interface harus dimiliki oleh arah abstraksi yang tepat dan mencerminkan kebutuhan modul tingkat tinggi. Jika interface hanya menyalin semua method class konkret tanpa alasan desain, coupling konseptual tetap tinggi.

4.3.7 Package Principles, Coupling, Stability, dan Abstractness

Inti Konsep.

Selain lima prinsip SOLID pada level class, sumber juga menyebut prinsip pemaketan untuk mengatur kohesi dan coupling antarpaket. Tujuannya sama: perubahan tidak menjarliar dan dependensi tidak membentuk siklus.

Ketika sistem membesar, masalah desain tidak hanya terjadi antarclass, tetapi juga antarpaket. Package principles membantu menjawab dua pertanyaan: class apa yang sebaiknya ditempatkan dalam paket yang sama, dan paket mana yang boleh bergantung pada paket lain.

Prinsip	Nama	Makna
REP	Release Reuse Equivalency Principle	Unit yang digunakan ulang sebaiknya juga menjadi unit rilis. Jika pengguna me-reuse paket, mereka perlu versi rilis yang jelas.
CCP	Common Closure Principle	Class yang cenderung berubah karena alasan yang sama sebaiknya ditempatkan bersama. Ini adalah versi paket dari SRP.
CRP	Common Reuse Principle	Class yang sering digunakan bersama sebaiknya ditempatkan bersama; class yang tidak digunakan bersama sebaiknya tidak memaksa klien mengambil dependensi tidak perlu. Ini berkaitan dengan ISP pada level paket.
ADP	Acyclic Dependencies Principle	Graf dependensi antarpaket tidak boleh memiliki siklus. Siklus membuat perubahan dan build saling mengunci.
SDP	Stable Dependencies Principle	Paket yang kurang stabil sebaiknya bergantung pada paket yang lebih stabil, bukan sebaliknya.
SAP	Stable Abstractions Principle	Paket yang stabil sebaiknya cukup abstrak agar dapat diperluas tanpa mudah diubah.

Tabel 3: Ringkasan package principles yang relevan dengan SOLID

4.4 Komponen Kunci Penting

Komponen Kunci	Makna atau Peran
SOLID	Akronim lima prinsip desain OOP: SRP, OCP, LSP, ISP, dan DIP.
<i>Rigidity</i>	Gejala desain buruk ketika perubahan kecil menuntut banyak perubahan lanjutan.
<i>Fragility</i>	Gejala desain buruk ketika perubahan di satu bagian merusak bagian lain yang tidak terduga.
<i>Immobility</i>	Gejala desain buruk ketika komponen sulit digunakan ulang karena dependensi terlalu melekat.
SRP	Prinsip satu alasan perubahan untuk satu class atau modul.
OCP	Prinsip menambah perilaku melalui ekstensi tanpa memodifikasi kode stabil.
LSP	Prinsip bahwa subclass harus aman menggantikan superclass sesuai kontrak perilaku.
ISP	Prinsip bahwa interface harus spesifik agar klien tidak bergantung pada method yang tidak digunakan.
DIP	Prinsip bahwa policy tingkat tinggi bergantung pada abstraksi, bukan detail konkret.
<i>Fat interface</i>	Interface yang terlalu besar dan memaksa implementor menyediakan method tidak relevan.
<i>Dependency injection</i>	Teknik memberikan dependency dari luar class untuk mendukung DIP dan testing.
<i>Afferent coupling</i>	Jumlah dependensi masuk: banyaknya komponen luar yang bergantung pada paket tertentu.
<i>Efferent coupling</i>	Jumlah dependensi keluar: banyaknya komponen luar yang dibutuhkan oleh paket tertentu.

Tabel 4: Komponen kunci dalam materi SOLID OOP Principle

4.5 Algoritma dan Rumus Penting

4.5.1 Metrik Instabilitas Paket

Makna Rumus.

Metrik instabilitas digunakan untuk membaca posisi paket dalam graf dependensi. Nilai mendekati 0 berarti paket stabil karena banyak yang bergantung padanya dan ia sedikit bergantung keluar; nilai mendekati 1 berarti paket instabil karena ia lebih banyak bergantung keluar.

$$I = \frac{C_e}{C_a + C_e} \quad (1)$$

dengan:

- I : instabilitas paket, bernilai dari 0 sampai 1.
- C_a : *afferent coupling*, yaitu jumlah class atau paket luar yang bergantung pada paket ini.
- C_e : *efferent coupling*, yaitu jumlah class atau paket luar yang menjadi dependency paket ini.

Jika C_a tinggi dan C_e rendah, paket tersebut sulit diubah karena banyak pihak bergantung padanya; paket ini stabil. Jika C_e tinggi dan C_a rendah, paket lebih mudah berubah karena sedikit pihak bergantung padanya, tetapi ia sensitif terhadap perubahan dependency.

4.5.2 Rasio Abstractness

Makna Rumus.

Rasio abstractness mengukur seberapa abstrak isi sebuah paket. Rumus ini relevan dengan SAP: paket yang sangat stabil sebaiknya memiliki abstraksi yang cukup agar tidak harus sering dimodifikasi.

$$A = \frac{N_a}{N_c} \quad (2)$$

dengan:

- A : rasio abstractness, bernilai dari 0 sampai 1.
- N_a : jumlah abstract class dan interface dalam paket.
- N_c : jumlah total class, abstract class, dan interface dalam paket.

Rumus ini bukan alat untuk menentukan nilai ujian secara mekanis, tetapi membantu menjelaskan mengapa paket yang stabil sering berisi interface atau abstract class. Paket yang banyak digunakan sebaiknya tidak terlalu bergantung pada detail konkret yang sering berubah.

4.5.3 Prosedur Identifikasi Pelanggaran SOLID

Tujuan Algoritma.

Prosedur ini membantu membaca cuplikan kode atau class diagram secara sistematis: cari gejala desain buruk, petakan ke prinsip SOLID, lalu usulkan refactoring yang tepat.

Listing 11: Pseudocode identifikasi pelanggaran SOLID

```

1 Input: Cuplikan kode Java atau class diagram
2 Output: Prinsip yang dilanggar dan usulan refactoring
3
4 1. Untuk setiap class:
5     a. Periksa apakah class memiliki lebih dari satu alasan perubahan.
6         Jika ya, tandai kemungkinan pelanggaran SRP.
7     b. Periksa apakah fitur baru menuntut modifikasi class lama
8         melalui if/switch berdasarkan tipe.
9         Jika ya, tandai kemungkinan pelanggaran OCP.
10    c. Jika class mewarisi class lain, periksa apakah subclass
11        dapat menggantikan superclass tanpa mengubah ekspektasi klien.
12        Jika tidak, tandai pelanggaran LSP.
13 2. Untuk setiap interface:
14    a. Periksa apakah implementor dipaksa membuat method kosong
15        atau melempar UnsupportedOperationException.
16        Jika ya, tandai pelanggaran ISP.
17 3. Untuk setiap dependency:
18    a. Periksa apakah modul high-level membuat atau bergantung langsung
19        pada class low-level konkret.
20        Jika ya, tandai pelanggaran DIP.
21 4. Usulkan perbaikan:
22    a. SRP: pisahkan tanggung jawab.
23    b. OCP: gunakan interface/polymorphism/strategy.
24    c. LSP: ubah hierarki atau kontrak abstraksi.
25    d. ISP: pecah interface menjadi kontrak kecil.
26    e. DIP: balik dependensi ke abstraksi dan gunakan injection.
```

4.6 Checklist Pemahaman

4.6.1 Submateri yang Telah Dibahas

- ☐ Saya dapat menjelaskan mengapa *rigidity*, *fragility*, dan *immobility* menunjukkan desain buruk.
- ☐ Saya dapat membedakan SRP, OCP, LSP, ISP, dan DIP tanpa hanya menghafal kepanjangannya.
- ☐ Saya dapat mengenali class Java yang melanggar SRP karena mencampur domain logic, persistence, dan formatting.
- ☐ Saya dapat menjelaskan mengapa percabangan tipe pada `GraphicEditor` melanggar OCP.
- ☐ Saya dapat menjelaskan mengapa `Square extends Rectangle` dapat melanggar LSP pada desain mutable.
- ☐ Saya dapat memecah *fat interface* menjadi beberapa interface kecil sesuai ISP.
- ☐ Saya dapat menerapkan DIP menggunakan interface dan constructor injection di Java.
- ☐ Saya dapat menjelaskan peran REP, CCP, CRP, ADP, SDP, dan SAP pada level paket.
- ☐ Saya dapat menghitung dan menafsirkan instabilitas paket dengan $I = \frac{C_e}{C_a + C_e}$.
- ☐ Saya dapat menjelaskan hubungan materi SOLID dengan CPMK, dengan catatan bahwa CPMK formal tidak disebutkan dalam perkuliahan.

4.6.2 Prioritas Belajar

Prioritas	Materi yang Perlu dikuasai
Wajib dikuasai	Definisi dan motivasi SOLID, tiga gejala <i>bad design</i> , SRP/OCP/LSP/ISP/DIP, contoh <code>GraphicEditor</code> , contoh <code>Square-Rectangle</code> , <i>fat interface</i> , dan dependency inversion dengan interface Java.
Cukup Paham Konsep	Package principles REP, CCP, CRP, ADP, SDP, SAP, metrik instabilitas, abstractness, dan hubungan SOLID dengan design pattern.
Sudah dikuasai	Materi 1 sampai 10 pernah diujikan dalam kuis, termasuk prinsip OOP SOLID, tetapi tetap perlu pendalaman melalui analisis kode dan refactoring.

5 Pengenalan Bahasa Java

Pengenalan Bahasa Java membahas cara Java memodelkan program sebagai kumpulan class, object, method, dan package, serta cara program Java dikompilasi menjadi bytecode yang dijalankan oleh Java Virtual Machine (JVM). Materi ini menjadi fondasi untuk seluruh topik setelahnya, karena inheritance, interface, generic, collection, stream, exception, multithreading, reflection, dan design pattern semuanya bergantung pada pemahaman struktur dasar Java.

Untuk ujian OOP, penguasaan Java dasar tidak cukup berhenti pada hafalan sintaks. Yang lebih penting adalah memahami mengapa Java memiliki struktur tertentu: mengapa semua program berada di dalam class, mengapa `main` harus `static`, mengapa tipe primitif berbeda dari object, mengapa access modifier memengaruhi enkapsulasi, dan bagaimana package/import membantu modularitas.

Komentar 5.1: Keterbatasan Sumber

Query NotebookLM untuk materi Pengenalan Bahasa Java tidak mengembalikan pembahasan Java dasar yang relevan; jawaban yang muncul kembali membahas SOLID. Karena itu, section ini disusun menggunakan pengetahuan umum Java yang stabil dan lazim dipakai pada pengajaran OOP. Bagian ini tidak mengklaim berasal langsung dari dokumen NotebookLM.

5.1 Outline Fundamental Konsep Materi

1. Identitas dan karakteristik Java **[SUDAH DIKUASAI]**
 - (a) Java sebagai bahasa berorientasi objek, statically typed, class-based, dan portable.
 - (b) Prinsip *write once, run anywhere* melalui bytecode dan JVM.
 - (c) Peran JDK, JRE, JVM, compiler `javac`, dan launcher `java`.
2. Struktur minimal program Java **[SUDAH DIKUASAI]**
 - (a) File `.java`, class, method, statement, block, dan semicolon.
 - (b) Method entry point `public static void main(String[] args)`.
 - (c) Aturan nama public class dan nama file.
3. Tipe data dan variabel **[SUDAH DIKUASAI]**
 - (a) Tipe primitif: `byte`, `short`, `int`, `long`, `float`, `double`, `char`, dan `boolean`.
 - (b) Tipe referensi: class, array, interface, enum, record, dan `String`.
 - (c) Literal, assignment, casting, overflow, dan nilai default.
4. Class, object, field, method, dan constructor **[SUDAH DIKUASAI]**
 - (a) Class sebagai blueprint dan object sebagai instance.
 - (b) Field menyimpan state; method mendefinisikan behavior.
 - (c) Constructor membentuk invariant awal object.
 - (d) `this`, `static`, `final`, dan overload method/constructor.
5. Access modifier dan enkapsulasi **[SUDAH DIKUASAI]**
 - (a) `private`, `package-private`, `protected`, dan `public`.
 - (b) Getter/setter yang validasi state, bukan sekadar membuka semua field.
 - (c) Hubungan access modifier dengan SRP, coupling, dan maintainability.
6. Control flow dan ekspresi dasar **[SUDAH DIKUASAI]**

- (a) Percabangan: `if`, `else`, `switch`.
 - (b) Perulangan: `for`, enhanced `for`, `while`, dan `do-while`.
 - (c) Operator, short-circuit evaluation, `break`, `continue`, dan `return`.
7. Package, import, dan organisasi kode **[SUDAH DIKUASAI]**
- (a) Package sebagai namespace dan unit organisasi.
 - (b) Import untuk menggunakan tipe dari package lain.
 - (c) Classpath/module path sebagai lokasi pencarian class.
8. Proses kompilasi dan eksekusi
- (a) `javac` mengubah source code menjadi bytecode `.class`.
 - (b) JVM memuat class, memverifikasi bytecode, lalu mengeksekusi program.
 - (c) Error kompilasi berbeda dari error runtime.
9. Hubungan Java dasar dengan materi OOP lanjut
- (a) Inheritance dan interface membutuhkan pemahaman class, method, dan access modifier.
 - (b) Generics dan collection membutuhkan pemahaman tipe referensi.
 - (c) Exception, assertion, multithreading, dan reflection membutuhkan pemahaman runtime Java.

5.2 Pentingnya Materi dan Relevansinya

Inti Relevansi.

Java dasar adalah lapisan sintaks dan runtime yang membuat konsep OOP dapat diterapkan secara konkret. Tanpa memahami struktur class, method, object, tipe, access modifier, dan proses eksekusi JVM, materi OOP lanjut akan terasa seperti aturan terpisah yang sulit dihubungkan.

Dalam ujian, materi pengenalan Java sering menjadi sumber kesalahan kecil tetapi fatal. Misalnya, mahasiswa dapat memahami konsep class secara umum, tetapi salah menulis `main`; memahami object, tetapi mencampur field instance dengan field `static`; memahami tipe data, tetapi tidak sadar bahwa `String` adalah object immutable; atau memahami access modifier, tetapi tidak tahu bahwa tanpa modifier berarti package-private, bukan `public`. Kesalahan seperti ini menunjukkan bahwa pemahaman OOP belum terikat kuat pada bahasa Java sebagai alat implementasi.

Java juga penting karena ia memisahkan source code, bytecode, dan runtime. Program Java tidak langsung menjadi executable native seperti beberapa bahasa lain. Source code dikompilasi menjadi bytecode yang dijalankan oleh JVM. Model ini menjelaskan portabilitas Java dan juga menjelaskan mengapa konsep seperti class loading, garbage collection, exception handling, reflection, dan multithreading menjadi bagian penting dari ekosistem Java.

Komentar 5.2: Relevansi Industri

Dalam pengembangan aplikasi nyata, Java banyak digunakan untuk backend, enterprise system, Android lama, service berbasis Spring, data processing, dan sistem yang membutuhkan ekosistem library matang. Pemahaman Java dasar memengaruhi kualitas desain: penempatan package, visibilitas class, immutability, pemilihan tipe, dan penggunaan dependency semuanya menentukan apakah kode mudah diuji dan dirawat.

5.2.1 Dependensi dan Hubungan dengan Materi Lain

Materi ini menjadi prasyarat langsung bagi semua materi setelahnya. Java String membutuhkan pemahaman tipe referensi dan object. Java Inheritance membutuhkan pemahaman class, constructor, access modifier, overriding, dan polymorphism. Java Interface membutuhkan pemahaman kontrak method dan reference type. Generics membutuhkan pemahaman tipe statis. Collection dan Stream API membutuhkan pemahaman object, interface, lambda, dan generic. Exception dan assertion membutuhkan pemahaman control flow serta runtime error. Multithreading membutuhkan pemahaman object bersama, method, dan memory visibility. Reflection membutuhkan pemahaman bahwa class dan method memiliki metadata pada runtime.

5.3 Penjelasan Konsep Fundamental

5.3.1 Karakteristik Dasar Java

Inti Konsep.

Java adalah bahasa pemrograman berorientasi objek yang *statically typed*, *class-based*, dan dirancang agar program dapat dikompilasi menjadi bytecode lalu dijalankan di JVM.

Java disebut *statically typed* karena tipe variabel, parameter, return value, dan field diperiksa saat kompilasi. Jika sebuah method mengharapkan `int`, pemanggil tidak dapat sembarangan memberikan `String`. Pemeriksaan tipe ini membuat banyak kesalahan ditemukan sebelum program berjalan. Java juga *class-based*, artinya struktur utama program adalah class. Object dibuat sebagai instance dari class, dan perilaku object dinyatakan melalui method.

Java sering disebut portable karena hasil kompilasinya bukan instruksi mesin khusus satu sistem operasi, melainkan bytecode. Bytecode ini dijalankan oleh JVM yang tersedia untuk berbagai platform. Dengan demikian, source yang sama dapat dikompilasi dan dijalankan di lingkungan berbeda selama tersedia JVM yang sesuai.

Komponen	Peran
JDK	Java Development Kit; berisi tools untuk mengembangkan program Java, termasuk <code>javac</code> , <code>java</code> , library standar, dan dokumentasi API.
JRE	Java Runtime Environment; lingkungan untuk menjalankan program Java. Pada praktik modern, JRE sering dipaketkan bersama JDK atau runtime distribusi tertentu.
JVM	Java Virtual Machine; mesin virtual yang memuat, memverifikasi, dan mengeksekusi bytecode Java.
<code>javac</code>	Compiler Java yang mengubah file <code>.java</code> menjadi file bytecode <code>.class</code> .
<code>java</code>	Launcher untuk menjalankan class yang memiliki entry point <code>main</code> .
Bytecode	Representasi hasil kompilasi yang portabel dan dijalankan oleh JVM.

Tabel 5: Komponen dasar ekosistem Java

5.3.2 Struktur Minimal Program Java

Inti Konsep.

Program Java minimal terdiri atas class dan method `main`. Method `main` menjadi entry point, yaitu titik awal eksekusi program ketika dijalankan dengan perintah `java`.

Contoh program Java paling sederhana:

Listing 12: Program Java minimal

```
1 public class HelloWorld {
2     public static void main(String[] args) {
3         System.out.println("Halo, Java!");
4     }
5 }
```

Setiap bagian dari program tersebut memiliki makna:

- `public class HelloWorld`: mendefinisikan class bernama `HelloWorld`. Jika class ini `public`, nama file harus `HelloWorld.java`.
- `public`: method dapat diakses dari luar class, sehingga JVM dapat memanggilnya sebagai entry point.
- `static`: method milik class, bukan object tertentu. JVM dapat menjalankan `main` tanpa membuat object `HelloWorld` terlebih dahulu.
- `void`: method tidak mengembalikan nilai.
- `main`: nama khusus yang dicari JVM sebagai entry point.
- `String[] args`: array argumen command-line yang diberikan saat program dijalankan.
- `System.out.println`: pemanggilan method untuk mencetak teks ke standard output.

Aturan pentingnya adalah: satu file `.java` boleh memiliki beberapa top-level class, tetapi hanya boleh ada satu top-level class `public`; jika ada class `public`, nama file harus sama persis dengan nama class tersebut. Java bersifat case-sensitive, sehingga `HelloWorld`, `helloworld`, dan `HELLOWORLD` adalah nama berbeda.

5.3.3 Class, Object, Field, Method, dan Constructor

Inti Konsep.

Class adalah cetak biru yang mendefinisikan state dan behavior; object adalah instance konkret dari class. Field menyimpan state, method menjalankan behavior, dan constructor membentuk object dalam keadaan awal yang valid.

Dalam OOP, class tidak seharusnya dipahami hanya sebagai tempat menaruh fungsi. Class memodelkan konsep. Jika kita membuat class `BankAccount`, maka field menyimpan state seperti saldo dan pemilik, sedangkan method merepresentasikan operasi yang sah seperti deposit dan withdraw. Constructor memastikan object dibuat dalam keadaan valid, misalnya saldo awal tidak negatif.

Listing 13: Class dengan field, constructor, dan method

```
1 public class BankAccount {
2     private final String owner;
3     private long balance;
4
5     public BankAccount(String owner, long initialBalance) {
6         if (owner == null || owner.isBlank()) {
7             throw new IllegalArgumentException("Owner wajib diisi");
8         }
9         if (initialBalance < 0) {
10             throw new IllegalArgumentException("Saldo awal tidak boleh negatif");
11         }
12         this.owner = owner;
13         this.balance = initialBalance;
14     }
15 }
```

```

14     }
15
16     public void deposit(long amount) {
17         if (amount <= 0) {
18             throw new IllegalArgumentException("Deposit harus positif");
19         }
20         balance += amount;
21     }
22
23     public void withdraw(long amount) {
24         if (amount <= 0 || amount > balance) {
25             throw new IllegalArgumentException("Penarikan tidak valid");
26         }
27         balance -= amount;
28     }
29
30     public long getBalance() {
31         return balance;
32     }
33 }

```

Kata kunci **this** merujuk pada object saat ini. Pada constructor di atas, **this.owner** berarti field **owner** milik object yang sedang dibuat. Kata kunci **final** pada field **owner** berarti field tersebut harus diisi sekali dan tidak dapat diganti setelah constructor selesai. Namun **final** pada reference object bukan berarti object yang dirujuk selalu immutable; ia hanya berarti variabel referensinya tidak dapat diarahkan ke object lain.

Listing 14: Membuat dan menggunakan object

```

1 public class Main {
2     public static void main(String[] args) {
3         BankAccount account = new BankAccount("Narendra", 100_000);
4         account.deposit(50_000);
5         account.withdraw(25_000);
6
7         System.out.println(account.getBalance()); // 125000
8     }
9 }

```

Operator **new** membuat object baru di heap dan mengembalikan reference ke object tersebut. Variabel **account** menyimpan reference, bukan menyalin seluruh object. Ini penting untuk memahami aliasing, mutability, parameter passing, collection, dan object sharing pada multithreading.

5.3.4 Tipe Primitif dan Tipe Referensi

Inti Konsep.

Java memiliki dua kelompok besar tipe: tipe primitif yang menyimpan nilai sederhana secara langsung, dan tipe referensi yang menyimpan reference menuju object atau array.

Tipe primitif adalah tipe bawaan yang bukan object. Java memiliki delapan tipe primitif. Tipe referensi mencakup class, interface, array, enum, record, dan **String**. Perbedaan ini penting karena operasi assignment, perbandingan, nilai default, dan kemungkinan **null** berbeda.

Tipe	Contoh	Catatan
byte	byte b = 10;	Integer 8-bit bertanda. Jarang dipakai kecuali untuk data biner.
short	short s = 1000;	Integer 16-bit bertanda.
int	int n = 42;	Integer 32-bit; pilihan default untuk bilangan bulat.
long	long x = 10L;	Integer 64-bit; gunakan suffix L untuk literal long.
float	float f = 1.5f;	Floating point 32-bit; gunakan suffix f.
double	double d = 1.5;	Floating point 64-bit; pilihan default untuk bilangan desimal.
char	char c = 'A';	Karakter 16-bit berbasis unit UTF-16.
boolean	boolean ok = true;	Hanya bernilai true atau false ; tidak dapat diganti dengan 0/1.

Tabel 6: Tipe primitif Java

Contoh perbedaan tipe primitif dan referensi:

Listing 15: Tipe primitif vs tipe referensi

```

1 int a = 10;
2 int b = a;
3 b = 20;
4 System.out.println(a); // 10, karena nilai int disalin.
5
6 BankAccount x = new BankAccount("A", 100);
7 BankAccount y = x;
8 y.deposit(50);
9 System.out.println(x.getBalance()); // 150, karena x dan y merujuk object sama.

```

Tipe referensi dapat bernilai **null**, yaitu tidak merujuk object apa pun. Memanggil method pada reference **null** menyebabkan **NullPointerException**. Tipe primitif tidak dapat bernilai **null**. Untuk kebutuhan generic atau collection, Java menyediakan wrapper class seperti **Integer**, **Long**, **Double**, dan **Boolean**.

Komentar 5.3: Autoboxing dan Unboxing

Java dapat mengubah primitif ke wrapper secara otomatis melalui autoboxing, misalnya **Integer x = 10;**. Sebaliknya, unboxing mengubah wrapper ke primitif, misalnya **int y = x;**. Hati-hati: unboxing reference **null** akan menyebabkan **NullPointerException**.

5.3.5 Variabel, Scope, Assignment, dan Casting

Inti Konsep.

Variabel adalah nama untuk menyimpan nilai atau reference. Scope menentukan wilayah program tempat variabel dapat diakses, sedangkan casting mengubah tipe nilai ketika konversi diperlukan dan valid.

Java membedakan local variable, parameter, field instance, dan field static. Local variable berada di dalam method atau block dan harus diinisialisasi sebelum dipakai. Field memiliki nilai default jika tidak diinisialisasi eksplisit. Misalnya, field **int** default-nya 0, **boolean** default-nya **false**, dan reference default-nya **null**. Namun, bergantung pada default field tanpa alasan jelas biasanya membuat kode sulit dibaca.

Listing 16: Scope dan shadowing variabel

```

1 public class Counter {
2     private int value;
3

```

```

4     public Counter(int value) {
5         this.value = value; // parameter value menutupi field value.
6     }
7
8     public void add(int delta) {
9         int oldValue = value; // local variable hanya hidup di method ini.
10        value = oldValue + delta;
11    }
12 }

```

Konversi tipe numerik dapat terjadi secara widening atau narrowing. Widening, seperti `int` ke `long`, aman karena rentang tipe tujuan lebih besar. Narrowing, seperti `long` ke `int`, membutuhkan cast eksplisit dan dapat kehilangan data.

Listing 17: Widening dan narrowing conversion

```

1 int small = 100;
2 long wide = small; // widening, otomatis.
3
4 long big = 3_000_000_000L;
5 int narrowed = (int) big; // narrowing, data dapat berubah karena overflow.

```

5.3.6 Access Modifier dan Enkapsulasi

Inti Konsep.

Access modifier mengatur siapa yang boleh mengakses class, field, constructor, atau method. Dalam OOP, access modifier adalah alat utama untuk enkapsulasi, yaitu menyembunyikan detail internal dan menjaga invariant object.

Java memiliki empat tingkat akses utama:

Modifier	Akses	Penggunaan Umum
<code>private</code>	Hanya di dalam class yang sama.	Field internal, helper method, detail implementasi.
<code>package-private</code>	Class dalam package yang sama. Ditulis tanpa modifier.	Kolaborasi internal antarclass dalam package.
<code>protected</code>	Package yang sama dan subclass.	Method/field yang memang dirancang untuk inheritance.
<code>public</code>	Dapat diakses dari mana saja.	API utama yang sengaja diekspos.

Tabel 7: Access modifier Java

Enkapsulasi bukan sekadar membuat field `private` lalu menghasilkan getter dan setter untuk semua field. Jika semua field dapat diubah bebas melalui setter, invariant object tetap bocor. Enkapsulasi yang baik berarti object menyediakan operasi yang bermakna dan menjaga validitas state di dalam operasi tersebut.

Listing 18: Getter/setter buruk vs method domain yang menjaga invariant

```

1 // Kurang baik: saldo dapat diset sembarangan dari luar.
2 public void setBalance(long balance) {
3     this.balance = balance;
4 }
5
6 // Lebih baik: operasi bermakna dan tervalidasi.
7 public void withdraw(long amount) {
8     if (amount <= 0 || amount > balance) {

```

```

9         throw new IllegalArgumentException("Penarikan tidak valid");
10    }
11    balance -= amount;
12 }

```

5.3.7 static, Instance Member, dan final

Inti Konsep.

Member instance melekat pada object tertentu, sedangkan member **static** melekat pada class. Kata kunci **final** mencegah reassignment, inheritance, atau overriding tergantung tempat penggunaannya.

Field instance dimiliki tiap object. Jika ada dua object **BankAccount**, masing-masing memiliki **balance** sendiri. Field static dimiliki class, sehingga nilainya dibagi oleh semua object dari class tersebut. Static cocok untuk konstanta atau operasi yang tidak membutuhkan state object. Static tidak cocok untuk menyimpan state global yang mudah berubah karena dapat meningkatkan coupling dan menyulitkan testing.

Listing 19: Instance member, static member, dan final

```

1 public class MathUtil {
2     public static final double PI = 3.141592653589793;
3
4     private MathUtil() {
5         // Mencegah instansiasi utility class.
6     }
7
8     public static int square(int x) {
9         return x * x;
10    }
11 }

```

Makna **final** bergantung pada lokasinya:

- **final** pada variabel: variabel tidak dapat diassign ulang.
- **final** pada field: field harus diisi sebelum constructor selesai dan tidak dapat diganti setelahnya.
- **final** pada method: method tidak dapat dioverride subclass.
- **final** pada class: class tidak dapat diwarisi.

5.3.8 Control Flow: Percabangan dan Perulangan

Inti Konsep.

Control flow menentukan urutan eksekusi statement. Java menyediakan percabangan untuk memilih jalur eksekusi dan perulangan untuk mengulang proses selama kondisi tertentu terpenuhi.

Percabangan menggunakan **if/else** cocok untuk kondisi boolean umum. **switch** cocok ketika pilihan bergantung pada satu nilai diskret seperti enum, integer, atau string.

Listing 20: Percabangan if dan switch

```

1 if (score >= 80) {
2     grade = "A";
3 } else if (score >= 70) {
4     grade = "B";

```



```

5 } else {
6     grade = "C";
7 }
8
9 switch (day) {
10     case MONDAY -> System.out.println("Awal minggu");
11     case FRIDAY -> System.out.println("Menjelang akhir minggu");
12     default -> System.out.println("Hari biasa");
13 }

```

Perulangan `for` cocok ketika jumlah iterasi jelas. `while` cocok ketika pengulangan bergantung kondisi yang belum tentu diketahui jumlahnya. Enhanced `for` cocok untuk membaca elemen array atau collection tanpa memerlukan indeks.

Listing 21: Perulangan dasar Java

```

1 for (int i = 0; i < 10; i++) {
2     System.out.println(i);
3 }
4
5 while (!queue.isEmpty()) {
6     Task task = queue.remove();
7     task.execute();
8 }
9
10 for (String name : names) {
11     System.out.println(name);
12 }

```

Operator logika `&&` dan `||` menggunakan short-circuit evaluation. Pada `a && b`, jika `a` sudah `false`, `b` tidak dievaluasi. Pada `a || b`, jika `a` sudah `true`, `b` tidak dievaluasi. Ini sering dipakai untuk mencegah `NullPointerException`.

Listing 22: Short-circuit untuk menjaga keamanan akses

```

1 if (user != null && user.isActive()) {
2     System.out.println("User aktif");
3 }

```

5.3.9 Package, Import, dan Organisasi Kode

Inti Konsep.

Package adalah namespace untuk mengelompokkan class yang berhubungan. Import membuat kode dapat menggunakan nama class dari package lain tanpa menulis fully qualified name setiap kali.

Package biasanya ditulis di baris pertama file Java. Struktur folder umumnya mengikuti nama package. Misalnya, class dengan package `id.ac.itb.oop.bank` biasanya berada pada folder `id/ac/itb/oop/bank`. Package membantu mencegah tabrakan nama dan memisahkan area tanggung jawab.

Listing 23: Package dan import

```

1 package id.ac.itb.oop.bank;
2
3 import java.math.BigDecimal;
4 import java.time.LocalDate;
5 import java.util.List;
6
7 public class Invoice {
8     private final List<BigDecimal> amounts;
9     private final LocalDate issuedAt;

```

```

10
11     public Invoice(List<BigDecimal> amounts, LocalDate issuedAt) {
12         this.amounts = List.copyOf(amounts);
13         this.issuedAt = issuedAt;
14     }
15 }

```

Import tidak memasukkan kode secara tekstual seperti `#include` di C/C++. Import hanya memberi tahu compiler bahwa nama pendek seperti `List` mengacu pada tipe `java.util.List`. Package `java.lang` otomatis diimpor, sehingga class seperti `String`, `System`, dan `Math` dapat langsung digunakan.

5.3.10 Kompilasi dan Eksekusi Program Java

Inti Konsep.

Source code Java dikompilasi oleh `javac` menjadi bytecode `.class`. Bytecode kemudian dijalankan oleh JVM menggunakan perintah `java`.

Prosedur sederhana untuk program tanpa package:

Listing 24: Kompilasi dan eksekusi program tanpa package

```

1 Input: File HelloWorld.java
2 Output: Program berjalan di JVM
3
4 1. Tulis source code di HelloWorld.java.
5 2. Jalankan: javac HelloWorld.java
6 3. Compiler menghasilkan HelloWorld.class.
7 4. Jalankan: java HelloWorld
8 5. JVM mencari method public static void main(String[] args).
9 6. JVM mengeksekusi bytecode dari method main.

```

Untuk program dengan package, lokasi output class dan nama yang dijalankan harus memperhatikan package:

Listing 25: Kompilasi dan eksekusi program dengan package

```

1 Misal file: src/id/ac/itb/oop/Main.java
2 Isi package: package id.ac.itb.oop;
3
4 1. Kompilasi:
5     javac -d out src/id/ac/itb/oop/Main.java
6
7 2. Jalankan:
8     java -cp out id.ac.itb.oop.Main

```

Error kompilasi terjadi ketika source code tidak memenuhi aturan bahasa, misalnya tipe tidak cocok, semicolon hilang, atau method tidak ditemukan. Error runtime terjadi setelah program berjalan, misalnya `NullPointerException`, pembagian dengan nol pada integer, atau akses indeks array di luar batas.

5.3.11 Hubungan Java dengan OOP

Inti Konsep.

Java menerapkan OOP melalui class, object, encapsulation, inheritance, polymorphism, dan abstraction. Namun, desain OOP yang baik tetap membutuhkan pemahaman prinsip, bukan hanya penggunaan sintaks class.

Class dan object adalah dasar OOP di Java. Encapsulation diterapkan melalui private field dan method publik yang menjaga invariant. Inheritance diterapkan melalui `extends`; interface

diterapkan melalui **implements**; polymorphism muncul ketika reference bertipe superclass atau interface dapat menunjuk object subclass atau implementasi konkret. Abstraction muncul ketika detail implementasi disembunyikan di balik method, abstract class, atau interface.

Listing 26: Polymorphism sederhana melalui interface

```
1 public interface NotificationSender {
2     void send(String recipient, String message);
3 }
4
5 public class EmailSender implements NotificationSender {
6     @Override
7     public void send(String recipient, String message) {
8         System.out.println("Email to " + recipient + ": " + message);
9     }
10 }
11
12 public class SmsSender implements NotificationSender {
13     @Override
14     public void send(String recipient, String message) {
15         System.out.println("SMS to " + recipient + ": " + message);
16     }
17 }
18
19 public class NotificationService {
20     private final NotificationSender sender;
21
22     public NotificationService(NotificationSender sender) {
23         this.sender = sender;
24     }
25
26     public void notifyUser(String recipient) {
27         sender.send(recipient, "Akun berhasil dibuat");
28     }
29 }
```

Contoh tersebut menunjukkan hubungan materi Java dasar dengan SOLID. Service notifikasi tidak peduli apakah pengiriman dilakukan lewat email atau SMS; ia hanya bergantung pada interface. Ini adalah dasar polymorphism sekaligus pintu masuk menuju DIP dan design pattern.

5.4 Komponen Kunci Penting

Komponen Kunci	Makna atau Peran
Java	Bahasa pemrograman OOP, statically typed, class-based, dan berjalan di JVM.
JDK	Paket pengembangan Java yang berisi compiler, launcher, dan library standar.
JVM	Mesin virtual yang menjalankan bytecode Java.
Bytecode	Hasil kompilasi Java dalam file <code>.class</code> .
Class	Blueprint yang mendefinisikan field, constructor, dan method.
Object	Instance konkret dari class.
Field	Data atau state yang dimiliki object atau class.
Method	Operasi atau behavior yang dapat dipanggil.
Constructor	Blok khusus untuk menginisialisasi object baru.
<code>main</code>	Entry point program Java. Bentuk umumnya adalah <code>public static void main</code> dengan parameter <code>String[] args</code> .
Primitive type	Tipe nilai sederhana seperti <code>int</code> , <code>double</code> , dan <code>boolean</code> .
Reference type	Tipe yang menyimpan reference ke object, array, class, interface, enum, atau record.
Access modifier	Pengatur visibilitas seperti <code>private</code> , <code>package-private</code> , <code>protected</code> , dan <code>public</code> .
Package	Namespace dan unit organisasi kode Java.
Import	Deklarasi untuk menggunakan tipe dari package lain dengan nama pendek.

Tabel 8: Komponen kunci dalam materi Pengenalan Bahasa Java

5.5 Algoritma dan Rumus Penting

5.5.1 Alur Eksekusi Program Java

Tujuan Algoritma.

Alur ini menjelaskan perjalanan program dari source code sampai berjalan di JVM. Pemahaman ini membantu membedakan error kompilasi, error runtime, dan masalah classpath.

Listing 27: Pseudocode alur kompilasi dan eksekusi Java

```
1 Input: Source code Java
2 Output: Program berjalan atau error terdeteksi
3
4 1. Programmer menulis file .java.
5 2. Compiler javac membaca source code.
6 3. Compiler memeriksa sintaks, tipe, access modifier, dan referensi simbol.
7 4. Jika ada kesalahan bahasa:
8     hentikan proses dan tampilkan compile-time error.
9 5. Jika valid:
10     hasilkan satu atau lebih file .class berisi bytecode.
11 6. Perintah java menjalankan class utama.
12 7. JVM mencari method:
13     public static void main(String[] args)
14 8. JVM memuat class yang diperlukan melalui class loader.
15 9. JVM memverifikasi bytecode.
16 10. JVM mengeksekusi instruksi bytecode.
17 11. Jika terjadi kesalahan saat berjalan:
18     tangani exception jika ada handler, jika tidak program berhenti.
```

5.5.2 Formula Tanda Tangan Method

Makna Rumus.

Dalam Java, method signature dipakai untuk membedakan method overload. Signature terdiri dari nama method dan daftar tipe parameter, bukan return type.

$$\text{signature(method)} = \text{nama method} + \text{urutan tipe parameter} \quad (3)$$

dengan:

- nama method: identifier method, misalnya `print`.
- urutan tipe parameter: daftar tipe parameter secara berurutan, misalnya `(String, int)`.

Contohnya, `print(String)` dan `print(int)` adalah overload valid karena tipe parameternya berbeda. Namun, dua method yang hanya berbeda return type tidak valid karena return type tidak menjadi bagian dari signature untuk overload.

Listing 28: Overload valid dan tidak valid

```
1 public class Printer {
2     public void print(String text) { }
3     public void print(int number) { }
4
5     // Tidak valid jika hanya berbeda return type:
6     // public int print(String text) { return text.length(); }
7 }
```

5.6 Checklist Pemahaman

5.6.1 Submateri yang Telah Dibahas

- ☐ Saya dapat menjelaskan perbedaan JDK, JRE, JVM, `javac`, `java`, dan bytecode.
- ☐ Saya dapat menulis struktur program Java minimal dengan `public static void main(String[] args)`.
- ☐ Saya dapat menjelaskan mengapa class `public` harus memiliki nama yang sama dengan file.
- ☐ Saya dapat membedakan class, object, field, method, dan constructor.
- ☐ Saya dapat membedakan tipe primitif dan tipe referensi, termasuk risiko `null`.
- ☐ Saya dapat menjelaskan perbedaan `private`, `package-private`, `protected`, dan `public`.
- ☐ Saya dapat menggunakan `static` dan `final` secara tepat.
- ☐ Saya dapat membaca dan menulis control flow dasar: `if`, `switch`, `for`, `while`, dan enhanced `for`.
- ☐ Saya dapat menjelaskan fungsi package dan import.
- ☐ Saya dapat menjelaskan alur kompilasi dan eksekusi program Java.
- ☐ Saya dapat menghubungkan Java dasar dengan konsep OOP seperti encapsulation, abstraction, inheritance, dan polymorphism.

5.6.2 Prioritas Belajar

Prioritas	Materi yang Perlu dikuasai
Wajib dikuasai	Struktur program Java, <code>main</code> , class-object-method-field-constructor, tipe primitif vs referensi, access modifier, <code>static</code> , <code>final</code> , package/import, dan alur <code>javac</code> ke JVM.
Cukup Paham Konsep	Detail widening/narrowing conversion, classpath, default value field, autoboxing/unboxing, dan method signature.
Sudah dikuasai	Materi ini termasuk kelompok 1 sampai 10 yang sudah pernah diujikan dalam kuis, tetapi tetap perlu diperkuat karena menjadi prasyarat semua materi Java lanjutan.

6 Padanan Java - C++

Materi Padanan Java - C++ membantu memetakan konsep yang mungkin sudah dikenal dari C++ ke cara Java mengekspresikan konsep yang sama. Fokus utamanya bukan menghafal perbedaan sintaks, melainkan memahami perbedaan model bahasa: Java berjalan di JVM, memakai garbage collection, semua object diakses melalui reference, tidak memiliki pointer arithmetic, tidak memiliki multiple inheritance untuk class, dan menggunakan interface serta generics untuk banyak kebutuhan abstraksi.

Komentar 6.1: Keterbatasan Sumber

Section ini disusun terutama menggunakan pengetahuan umum Java dan C++ karena sumber NotebookLM yang berhasil diakses lebih dominan membahas SOLID. Penjelasan tetap diarahkan pada kebutuhan IF2010 PBO dan ditandai sebagai pelengkap umum.

6.1 Outline Fundamental Konsep Materi

1. Model eksekusi **[SUDAH DIKUASAI]**
 - (a) C++ dikompilasi menjadi native binary; Java dikompilasi menjadi bytecode.
 - (b) Java dijalankan oleh JVM dengan class loading, verification, JIT, dan garbage collection.
2. Object dan reference **[SUDAH DIKUASAI]**
 - (a) C++ membedakan value object, pointer, dan reference.
 - (b) Java menggunakan reference untuk object; tipe primitif tetap menyimpan nilai.
3. Memory management **[SUDAH DIKUASAI]**
 - (a) C++ memakai RAII, destructor, `new/delete`, smart pointer.
 - (b) Java memakai garbage collector dan tidak memiliki destructor deterministik.
4. Class, inheritance, dan polymorphism **[SUDAH DIKUASAI]**
 - (a) Java: `extends` satu class dan `implements` banyak interface.
 - (b) C++: multiple inheritance class dimungkinkan.
 - (c) Java method instance virtual secara default kecuali `final`, `static`, atau `private`.
5. Header, package, dan import
 - (a) C++ memakai header dan `#include`; Java memakai package dan import.
 - (b) Import bukan copy-paste source, hanya resolusi nama tipe.
6. Generics dan templates
 - (a) C++ template bekerja saat kompilasi dan menghasilkan instansiasi tipe.
 - (b) Java generics memakai type erasure untuk menjaga kompatibilitas bytecode.
7. Exception dan resource management
 - (a) Java memiliki checked dan unchecked exception.
 - (b) Java memakai `try-with-resources`; C++ mengandalkan destructor RAII.

6.2 Pentingnya Materi dan Relevansinya

Inti Relevansi.

Perbandingan Java dan C++ penting karena banyak konsep OOP terlihat serupa di permukaan, tetapi memiliki konsekuensi desain dan runtime yang berbeda. Salah membawa kebiasaan C++ ke Java dapat menyebabkan desain Java yang tidak idiomatis.

Contoh kesalahan umum adalah mencari destructor Java untuk menutup file, padahal Java menyediakan `try-with-resources`. Kesalahan lain adalah mengira `import` bekerja seperti `#include`, padahal `import` tidak menyisipkan source code. Perbedaan paling penting untuk OOP adalah inheritance: C++ memungkinkan sebuah class mewarisi beberapa class, sedangkan Java hanya mengizinkan satu superclass dan banyak interface. Karena itu, di Java abstraksi perilaku lebih sering dimodelkan dengan interface dan composition.

6.2.1 Dependensi dan Hubungan dengan Materi Lain

Materi ini menghubungkan Pengenalan Java dengan Inheritance, Interface, Generics, Exception, dan Design Pattern. Perbedaan Java-C++ menjelaskan mengapa Java Interface sangat penting untuk ISP dan DIP, mengapa Generics Java memiliki wildcard, dan mengapa resource management Java muncul pada materi Exception.

6.3 Penjelasan Konsep Fundamental

6.3.1 Model Kompilasi dan Eksekusi

Inti Konsep.

C++ umumnya dikompilasi menjadi executable native, sedangkan Java dikompilasi menjadi bytecode yang dijalankan JVM. Perbedaan ini memengaruhi portabilitas, runtime checking, class loading, dan optimisasi.

Pada C++, compiler menghasilkan binary untuk arsitektur dan sistem operasi tertentu. Pada Java, `javac` menghasilkan file `.class`. File ini berisi bytecode yang dipahami JVM. Saat program berjalan, JVM memuat class, memverifikasi bytecode, mengelola memory, dan dapat melakukan Just-In-Time compilation untuk mengubah bagian bytecode yang sering dijalankan menjadi native code.

Listing 29: Perbandingan alur build sederhana

```
1 C++:  
2 1. g++ main.cpp -o app  
3 2. ./app  
4  
5 Java:  
6 1. javac -d out Main.java  
7 2. java -cp out Main
```

6.3.2 Object, Pointer, Reference, dan Nilai

Inti Konsep.

Java tidak memiliki pointer arithmetic dan tidak mengekspos alamat memory. Object Java diakses melalui reference, sedangkan tipe primitif menyimpan nilai langsung.

Dalam C++, object dapat dibuat di stack, heap, sebagai pointer, atau sebagai reference. Di Java, object selalu dibuat dengan `new` atau factory dan diakses melalui reference. Variabel object tidak menyimpan object secara langsung, melainkan reference menuju object. Karena itu, assignment antarvariabel object menyalin reference, bukan menyalin isi object.

Listing 30: Assignment object Java menyalin reference

```

1 class Box {
2     int value;
3 }
4
5 Box a = new Box();
6 a.value = 10;
7
8 Box b = a;
9 b.value = 20;
10
11 System.out.println(a.value); // 20

```

Padanan kasar di C++ adalah pointer atau reference, tetapi Java tidak mengizinkan operasi seperti menaikkan alamat, mengambil alamat dengan `&`, atau melakukan `delete`. Ini membuat Java lebih aman dari kelas bug tertentu, tetapi programmer tetap harus memahami aliasing karena beberapa reference dapat menunjuk object yang sama.

6.3.3 Memory Management dan Resource Management

Inti Konsep.

C++ mengandalkan lifetime object dan destructor deterministik, sedangkan Java mengandalkan garbage collector untuk memory dan `try-with-resources` untuk resource eksternal.

Garbage collector Java membersihkan object yang tidak lagi dapat dijangkau. Namun, garbage collector hanya mengelola memory heap, bukan resource eksternal secara deterministik. File, socket, database connection, dan lock harus ditutup secara eksplisit. Cara idiomatisnya adalah menggunakan `try-with-resources` pada object yang mengimplementasikan `AutoCloseable`.

Listing 31: Resource management Java dengan try-with-resources

```

1 try (BufferedReader reader = Files.newBufferedReader(Path.of("data.txt"))) {
2     String line = reader.readLine();
3     System.out.println(line);
4 } catch (IOException e) {
5     System.err.println("Gagal membaca file: " + e.getMessage());
6 }

```

6.3.4 Inheritance dan Interface

Inti Konsep.

Java hanya mendukung single inheritance untuk class, tetapi mendukung multiple typing melalui banyak interface. Ini membuat interface menjadi alat utama untuk abstraksi perilaku.

Di Java, sebuah class hanya dapat `extends` satu superclass. Namun, class dapat `implements` banyak interface. Ini menghindari sebagian masalah diamond inheritance pada state dan implementasi class, sambil tetap memungkinkan object memiliki banyak peran.

Listing 32: Single class inheritance dan multiple interface implementation

```

1 interface Printable {
2     void print();
3 }
4
5 interface Exportable {
6     byte[] export();
7 }

```

```

8
9 abstract class Document {
10     private final String title;
11
12     protected Document(String title) {
13         this.title = title;
14     }
15
16     public String title() {
17         return title;
18     }
19 }
20
21 class PdfDocument extends Document implements Printable, Exportable {
22     PdfDocument(String title) {
23         super(title);
24     }
25
26     @Override
27     public void print() { }
28
29     @Override
30     public byte[] export() {
31         return new byte[0];
32     }
33 }

```

6.3.5 Header-Include vs Package-Import

Inti Konsep.

#include pada C++ menyisipkan deklarasi dari header secara tekstual sebelum kompilasi, sedangkan import pada Java hanya membantu resolusi nama tipe.

Java tidak memerlukan header terpisah karena informasi tipe tersedia dari source atau bytecode class. Jika memakai `import java.util.List;`, compiler hanya mengetahui bahwa nama `List` mengacu pada `java.util.List`. Import wildcard seperti `import java.util.*;` hanya mengimpor tipe dalam package tersebut, bukan subpackage.

6.3.6 Generics Java dan Template C++

Inti Konsep.

C++ template adalah mekanisme compile-time yang dapat menghasilkan kode berbeda untuk setiap tipe, sedangkan Java generics berfokus pada type safety dan umumnya dihapus menjadi raw type melalui type erasure.

Akibat type erasure, informasi tipe generic seperti `List<String>` tidak sepenuhnya tersedia pada runtime. Ini menjelaskan mengapa Java tidak mengizinkan `new T()` secara langsung dan mengapa array generic bermasalah.

Listing 33: Generic Java memberi type safety saat kompilasi

```

1 List<String> names = new ArrayList<>();
2 names.add("Narendra");
3
4 String first = names.get(0); // Tidak perlu cast manual.
5 // names.add(123);           // Compile-time error.

```

6.4 Komponen Kunci Penting

Komponen Kunci	Makna atau Peran
Bytecode	Format hasil kompilasi Java yang dijalankan JVM.
Native binary	Hasil kompilasi C++ yang spesifik platform.
Reference Java	Pegangan menuju object tanpa pointer arithmetic.
Pointer C++	Variabel yang menyimpan alamat memory dan dapat dimanipulasi.
Garbage collector	Pengelola memory otomatis untuk object Java yang tidak lagi reachable.
RAII	Idiom C++ yang mengikat resource pada lifetime object dan destructor.
Interface	Padanan Java untuk kontrak perilaku dan multiple typing.
Type erasure	Mekanisme implementasi generics Java yang menghapus tipe generic pada runtime.

Tabel 9: Komponen kunci dalam padanan Java dan C++

6.5 Algoritma dan Rumus Penting

6.5.1 Prosedur Memetakan Konsep C++ ke Java

Tujuan Algoritma.

Prosedur ini membantu menerjemahkan desain C++ ke Java tanpa membawa idiom yang tidak cocok.

Listing 34: Pseudocode pemetaan konsep C++ ke Java

```
1 Input: Konsep atau kode C++ yang ingin ditulis dalam Java
2 Output: Desain Java yang idiomatis
3
4 1. Jika konsep memakai pointer:
5     gunakan reference object Java atau Optional jika nilai boleh tidak ada.
6 2. Jika konsep memakai delete/destructor:
7     gunakan garbage collection untuk memory,
8     gunakan try-with-resources untuk resource eksternal.
9 3. Jika konsep memakai multiple inheritance class:
10    pisahkan state ke satu superclass atau composition,
11    pisahkan perilaku ke beberapa interface.
12 4. Jika konsep memakai template container:
13    gunakan Java generics dan pahami type erasure.
14 5. Jika konsep memakai header/include:
15    gunakan package dan import.
16 6. Jika konsep memakai operator overloading:
17    gunakan method bernama eksplisit karena Java tidak mendukung operator
    overloading umum.
```

6.6 Checklist Pemahaman

6.6.1 Submateri yang Telah Dibahas

- ☐ Saya dapat membedakan alur kompilasi C++ dan Java.
- ☐ Saya dapat menjelaskan object reference Java dan perbedaannya dari pointer C++.
- ☐ Saya dapat menjelaskan mengapa Java tidak memakai destructor untuk menutup file.
- ☐ Saya dapat menerapkan try-with-resources.

- ☐ Saya dapat membedakan `extends` dan `implements`.
- ☐ Saya dapat menjelaskan perbedaan import Java dan `include` C++.
- ☐ Saya dapat menjelaskan garis besar perbedaan generics Java dan template C++.

6.6.2 Prioritas Belajar

Prioritas	Materi yang Perlu dikuasai
Wajib dikuasai	JVM-bytecode vs native binary, reference vs pointer, garbage collection, <code>try-with-resources</code> , single inheritance Java, interface, package/import.
Cukup Pahami Konsep	Type erasure, RAII, classpath, JIT, dan perbedaan template dengan generics.
Sudah dikuasai	Materi ini termasuk kelompok 1 sampai 10 yang sudah pernah diuji, tetapi perlu diperdalam untuk menghindari transfer konsep C++ yang keliru ke Java.

7 Java String

Java String membahas cara Java merepresentasikan teks melalui class **String** dan class pendukung seperti **StringBuilder**, **StringBuffer**, **Character**, dan API regular expression. Materi ini penting karena **String** terlihat sederhana, tetapi memiliki banyak detail yang sering menjadi sumber bug: immutability, string pool, perbedaan `==` dan `equals`, biaya concatenation, encoding Unicode, dan operasi parsing.

Komentar 7.1: Keterbatasan Sumber

Section ini disusun menggunakan pengetahuan umum Java karena query NotebookLM sebelumnya tidak mengembalikan materi Java dasar yang relevan. Pembahasan diarahkan pada konsep yang lazim muncul dalam evaluasi OOP dan pemrograman Java.

7.1 Outline Fundamental Konsep Materi

1. Karakteristik **String** [SUDAH DIKUASAI]
 - (a) **String** adalah object immutable.
 - (b) Literal string disimpan di string pool.
 - (c) **String** adalah reference type, bukan primitive type.
2. Perbandingan string [SUDAH DIKUASAI]
 - (a) `==` membandingkan reference.
 - (b) `equals` membandingkan isi.
 - (c) `equalsIgnoreCase`, `compareTo`, dan `compareToIgnoreCase`.
3. Operasi dasar [SUDAH DIKUASAI]
 - (a) `length`, `charAt`, `substring`, `indexOf`, `contains`.
 - (b) `trim`, `strip`, `toLowerCase`, `toUpperCase`.
 - (c) `split`, `join`, `replace`, `matches`.
4. Mutability dan performa
 - (a) Concatenation berulang dapat mahal.
 - (b) **StringBuilder** untuk konstruksi string mutable non-thread-safe.
 - (c) **StringBuffer** untuk varian synchronized.
5. Parsing dan formatting
 - (a) Konversi string ke number dan sebaliknya.
 - (b) `String.format` dan formatted string.
 - (c) Risiko `NumberFormatException`.
6. Unicode dan character
 - (a) `char` adalah unit UTF-16, bukan selalu satu karakter manusia.
 - (b) Code point dapat membutuhkan surrogate pair.

7.2 Pentingnya Materi dan Relevansinya

Inti Relevansi.

`String` penting karena hampir semua program memproses teks: input pengguna, file, log, query, pesan error, konfigurasi, dan data dari API. Memahami immutability dan comparison adalah syarat untuk menulis Java yang benar.

Kesalahan paling umum adalah memakai `==` untuk membandingkan isi string. Kadang hasilnya tampak benar karena string literal berada di pool, tetapi program menjadi salah saat string berasal dari input, file, atau hasil konstruksi runtime. Kesalahan lain adalah membuat string panjang dengan concatenation dalam loop, padahal setiap perubahan menghasilkan object baru karena `String` immutable.

7.2.1 Dependensi dan Hubungan dengan Materi Lain

`String` berkaitan dengan `Exception` melalui parsing yang dapat gagal, dengan `Collection` karena string sering disimpan dan diproses dalam list/map, dengan `Stream` API untuk transformasi teks, dan dengan `Reflection` karena nama class/method sering direpresentasikan sebagai string. Immutability `String` juga menjadi contoh penting untuk desain object yang aman.

7.3 Penjelasan Konsep Fundamental

7.3.1 Immutability String

Inti Konsep.

`String` bersifat immutable: setelah object string dibuat, isi teksnya tidak dapat diubah. Operasi seperti `replace` atau `toUpperCase` menghasilkan string baru.

Immutability membuat `String` aman dibagikan, cocok sebagai key `Map`, dan aman digunakan di string pool. Namun, immutability juga berarti operasi yang tampak mengubah string sebenarnya membuat object baru.

Listing 35: Operasi String menghasilkan object baru

```
1 String name = "java";
2 String upper = name.toUpperCase();
3
4 System.out.println(name); // java
5 System.out.println(upper); // JAVA
```

7.3.2 String Pool dan Perbandingan

Inti Konsep.

String literal disimpan di string pool sehingga literal yang sama dapat berbagi object. Namun, perbandingan isi string tetap harus memakai `equals`, bukan `==`.

`==` membandingkan apakah dua reference menunjuk object yang sama. `equals` membandingkan isi teks. String pool membuat beberapa kasus `==` tampak berhasil, tetapi itu bukan aturan perbandingan isi.

Listing 36: Perbedaan `==` dan `equals` pada String

```
1 String a = "ITB";
2 String b = "ITB";
3 String c = new String("ITB");
4
5 System.out.println(a == b); // true, literal dari pool.
6 System.out.println(a == c); // false, object berbeda.
7 System.out.println(a.equals(c)); // true, isi sama.
```

Untuk menghindari `NullPointerException`, jika nilai pembanding berupa konstanta, letakkan konstanta di kiri:

Listing 37: Null-safe equals dengan konstanta

```
1 String role = getRole();
2
3 if ("ADMIN".equals(role)) {
4     System.out.println("Akses admin");
5 }
```

7.3.3 Operasi String yang Sering Dipakai

Inti Konsep.

Operasi string dasar digunakan untuk memeriksa, mengambil, membersihkan, memecah, mengganti, dan menggabungkan teks. Setiap method harus dipahami dari input, output, dan apakah ia memakai regex.

Method	Makna
<code>length()</code>	Mengembalikan jumlah unit <code>char</code> UTF-16.
<code>charAt(i)</code>	Mengambil karakter pada indeks ke- <i>i</i> .
<code>substring(a,b)</code>	Mengambil bagian string dari indeks <i>a</i> inklusif sampai <i>b</i> eksklusif.
<code>contains(s)</code>	Mengecek apakah substring ada.
<code>indexOf(s)</code>	Mengembalikan indeks pertama substring atau -1 jika tidak ada.
<code>strip()</code>	Menghapus whitespace Unicode di awal dan akhir.
<code>split(regex)</code>	Memecah string berdasarkan regular expression.
<code>replace(a,b)</code>	Mengganti literal character sequence.

Tabel 10: Operasi penting pada String

Listing 38: Contoh operasi dasar String

```
1 String raw = " IF2010,PB0,Java ";
2 String cleaned = raw.strip();
3 String[] parts = cleaned.split(",");
4
5 System.out.println(parts[0]); // IF2010
6 System.out.println(cleaned.contains("Java")); // true
```

7.3.4 StringBuilder dan StringBuffer

Inti Konsep.

`StringBuilder` menyediakan buffer teks mutable untuk membangun string secara efisien. Gunakan ketika string dibentuk bertahap, terutama dalam loop.

Karena `String` immutable, concatenation berulang dapat menghasilkan banyak object sementara. Compiler dapat mengoptimalkan concatenation sederhana, tetapi untuk loop yang membangun teks bertahap, `StringBuilder` lebih jelas dan efisien.

Listing 39: Membangun string dengan `StringBuilder`

```
1 List<String> names = List.of("Ayu", "Budi", "Citra");
2
3 StringBuilder builder = new StringBuilder();
4 for (String name : names) {
```

```

5     builder.append("- ").append(name).append(System.lineSeparator());
6 }
7
8 String result = builder.toString();

```

`StringBuffer` mirip `StringBuilder`, tetapi method-nya `synchronized`. Dalam kode modern, `StringBuilder` lebih sering dipakai untuk konteks `single-thread` karena lebih ringan.

7.3.5 Parsing, Formatting, dan Regex

Inti Konsep.

Parsing mengubah string menjadi tipe lain; formatting mengubah data menjadi string. Keduanya harus menangani kemungkinan format input tidak valid.

Listing 40: Parsing dan formatting String

```

1 String input = "13524044";
2 int nim = Integer.parseInt(input);
3
4 String message = String.format("NIM: %08d", nim);
5 System.out.println(message);

```

Jika parsing menerima teks yang bukan integer valid, Java melempar exception format angka. Karena itu, parsing input eksternal harus divalidasi atau ditangani dengan exception.

Method `split` dan `matches` memakai regular expression. Ini penting karena karakter seperti titik `.` bermakna khusus pada regex. Untuk memecah berdasarkan titik literal, gunakan `"\\."`.

7.3.6 Unicode dan char

Inti Konsep.

`char` Java adalah unit UTF-16, bukan selalu satu karakter manusia. Beberapa karakter Unicode membutuhkan dua `char` yang disebut surrogate pair.

Untuk banyak kebutuhan ujian dasar, `length` dan `charAt` cukup. Namun, secara konseptual penting memahami bahwa `length()` mengembalikan jumlah unit UTF-16, bukan jumlah grapheme yang dilihat manusia.

7.4 Komponen Kunci Penting

Komponen Kunci	Makna atau Peran
<code>String</code>	Class immutable untuk teks.
String pool	Area penyimpanan literal string agar literal yang sama dapat berbagi object.
<code>equals</code>	Method untuk membandingkan isi string.
<code>==</code>	Operator untuk membandingkan reference, bukan isi object.
<code>StringBuilder</code>	Buffer mutable untuk membangun string secara efisien.
<code>StringBuffer</code>	Versi <code>synchronized</code> dari <code>StringBuilder</code> .
Regex	Pola teks yang dipakai oleh method seperti <code>split</code> dan <code>matches</code> .
UTF-16	Representasi internal <code>char</code> Java.

Tabel 11: Komponen kunci dalam materi Java String

7.5 Algoritma dan Rumus Penting

7.5.1 Prosedur Memilih Operasi String

Tujuan Algoritma.

Prosedur ini membantu menentukan kapan memakai `String`, `StringBuilder`, `equals`, parsing, atau regex.

Listing 41: Pseudocode pemilihan operasi String

```
1 Input: Kebutuhan pemrosesan teks
2 Output: API String yang sesuai
3
4 1. Jika membandingkan isi:
5     gunakan equals atau equalsIgnoreCase.
6 2. Jika membandingkan urutan leksikografis:
7     gunakan compareTo atau compareToIgnoreCase.
8 3. Jika membangun teks dalam loop:
9     gunakan StringBuilder.
10 4. Jika memecah teks berdasarkan pola:
11     gunakan split dengan regex yang benar.
12 5. Jika mengubah teks ke angka:
13     gunakan parseInt/parseLong/parseDouble dan tangani format invalid.
14 6. Jika input dapat null:
15     validasi null sebelum memanggil method.
```

7.6 Checklist Pemahaman

7.6.1 Submateri yang Telah Dibahas

- ☐ Saya dapat menjelaskan immutability `String`.
- ☐ Saya dapat membedakan `==` dan `equals`.
- ☐ Saya dapat menjelaskan string pool.
- ☐ Saya dapat memakai operasi dasar seperti `substring`, `indexOf`, `split`, dan `replace`.
- ☐ Saya dapat memakai `StringBuilder` untuk concatenation dalam loop.
- ☐ Saya dapat menjelaskan risiko parsing dan `NumberFormatException`.
- ☐ Saya memahami bahwa `char` bukan selalu satu karakter manusia.

7.6.2 Prioritas Belajar

Prioritas	Materi yang Perlu dikuasai
Wajib dikuasai	Immutability, string pool, <code>==</code> vs <code>equals</code> , operasi dasar, <code>StringBuilder</code> , parsing.
Cukup Pahami Konsep	Regex pada <code>split</code> , Unicode, <code>StringBuffer</code> , formatting.
Sudah dikuasai	Materi ini termasuk kelompok 1 sampai 10 yang pernah diuji, tetapi tetap rawan kesalahan detail sintaks.

8 Java Inheritance

Java Inheritance membahas mekanisme pewarisan class menggunakan **extends**, pemanggilan constructor superclass melalui **super**, overriding method, dynamic dispatch, dan polymorphism. Materi ini penting karena inheritance adalah salah satu pilar OOP, tetapi juga salah satu sumber desain buruk jika digunakan hanya karena dua class tampak mirip.

Komentar 8.1: Keterbatasan Sumber

Bagian ini disusun dengan pengetahuan umum Java dan dikaitkan dengan pembahasan SOLID yang tersedia dari NotebookLM, terutama Liskov Substitution Principle. Penjelasan sintaks dan contoh Java merupakan pelengkap umum.

8.1 Outline Fundamental Konsep Materi

1. Konsep inheritance **[SUDAH DIKUASAI]**
 - (a) Relasi *is-a* antara subclass dan superclass.
 - (b) Subclass mewarisi field/method yang dapat diakses.
 - (c) Java hanya mengizinkan satu superclass langsung.
2. Constructor dan **super** **[SUDAH DIKUASAI]**
 - (a) Constructor tidak diwarisi.
 - (b) Constructor subclass harus memanggil constructor superclass secara eksplisit atau implisit.
 - (c) **super(...)** harus menjadi statement pertama constructor.
3. Overriding dan dynamic dispatch **[SUDAH DIKUASAI]**
 - (a) Subclass dapat mengganti implementasi method instance.
 - (b) Method yang dijalankan ditentukan oleh tipe runtime object.
 - (c) **@Override** membantu validasi kompilasi.
4. Polymorphism
 - (a) Reference superclass dapat menunjuk object subclass.
 - (b) Upcasting aman; downcasting berisiko **ClassCastException**.
5. Access modifier dan inheritance
 - (a) **private** tidak dapat diakses langsung oleh subclass.
 - (b) **protected** memberi akses pada subclass dan package yang sama.
 - (c) Field hiding berbeda dari method overriding.
6. Abstract class dan final
 - (a) Abstract class tidak dapat diinstansiasi.
 - (b) Abstract method wajib diimplementasikan subclass konkret.
 - (c) **final** mencegah inheritance atau overriding.
7. Risiko desain
 - (a) Inheritance harus menjaga LSP.
 - (b) Prefer composition ketika relasi bukan *is-a* yang kuat.

8.2 Pentingnya Materi dan Relevansinya

Inti Relevansi.

Inheritance memungkinkan reuse dan polymorphism, tetapi harus dipakai untuk relasi substitusi yang benar. Ujian sering menguji apakah subclass benar-benar dapat menggantikan superclass.

Inheritance membuat program dapat menulis kode terhadap tipe umum. Misalnya, daftar `List<Shape>` dapat berisi `Circle`, `Rectangle`, dan `Triangle`. Namun, inheritance juga memperkuat coupling antara subclass dan superclass. Jika superclass berubah, subclass dapat ikut terdampak. Karena itu, inheritance sebaiknya dipakai ketika ada kontrak perilaku yang stabil dan subclass benar-benar merupakan bentuk khusus dari superclass.

8.2.1 Dependensi dan Hubungan dengan Materi Lain

Inheritance berhubungan langsung dengan SOLID, khususnya LSP dan OCP. Interface memperluas konsep polymorphism tanpa mewarisi state. Exception memakai inheritance untuk hierarki `Throwable`. Collection memakai interface dan abstract class. Design pattern seperti Template Method, Factory Method, dan Strategy sering melibatkan inheritance atau polymorphism.

8.3 Penjelasan Konsep Fundamental

8.3.1 Relasi *is-a* dan Pewarisan Class

Inti Konsep.

Inheritance menyatakan bahwa subclass adalah bentuk khusus dari superclass. Jika `Dog` extends `Animal`, maka setiap `Dog` harus aman diperlakukan sebagai `Animal`.

Listing 42: Inheritance dasar di Java

```
1 public class Animal {
2     private final String name;
3
4     public Animal(String name) {
5         this.name = name;
6     }
7
8     public String name() {
9         return name;
10    }
11
12    public void speak() {
13        System.out.println("Animal sound");
14    }
15 }
16
17 public class Cat extends Animal {
18     public Cat(String name) {
19         super(name);
20     }
21
22     @Override
23     public void speak() {
24         System.out.println("Meow");
25     }
26 }
```

Class `Cat` mewarisi method `name` dan mengganti implementasi `speak`. Field `name` tetap `private`; subclass tidak mengaksesnya langsung, tetapi melalui method superclass.

8.3.2 Constructor Chaining dan super

Inti Konsep.

Constructor subclass selalu memanggil constructor superclass sebelum menjalankan inisialisasi miliknya sendiri. Ini menjamin bagian superclass dari object terbentuk lebih dulu.

Jika constructor subclass tidak menulis `super(...)` secara eksplisit, compiler menyisipkan `super()` tanpa argumen. Jika superclass tidak memiliki no-argument constructor, subclass wajib memanggil constructor yang tersedia.

Listing 43: Constructor subclass wajib memanggil superclass

```
1 class Vehicle {
2     private final String plateNumber;
3
4     Vehicle(String plateNumber) {
5         this.plateNumber = plateNumber;
6     }
7 }
8
9 class Car extends Vehicle {
10     private final int seatCount;
11
12     Car(String plateNumber, int seatCount) {
13         super(plateNumber);
14         this.seatCount = seatCount;
15     }
16 }
```

8.3.3 Overriding, Overloading, dan Dynamic Dispatch

Inti Konsep.

Overriding mengganti implementasi method superclass pada subclass dengan signature kompatibel. Overloading membuat beberapa method bernama sama tetapi parameter berbeda.

Dynamic dispatch berarti method instance yang dijalankan ditentukan oleh tipe runtime object, bukan tipe deklarasi reference.

Listing 44: Dynamic dispatch pada method overriding

```
1 Animal animal = new Cat("Milo");
2 animal.speak(); // Meow, karena object runtime adalah Cat.
```

Gunakan `@Override` setiap kali berniat override. Jika signature salah, compiler akan memberi error sehingga bug lebih cepat ditemukan.

8.3.4 Upcasting, Downcasting, dan instanceof

Inti Konsep.

Upcasting dari subclass ke superclass aman karena setiap subclass adalah superclass. Downcasting membutuhkan pemeriksaan karena tidak setiap superclass adalah subclass tertentu.

Listing 45: Upcasting dan downcasting aman

```
1 Animal animal = new Cat("Milo"); // upcasting, aman.
```

```

2
3 if (animal instanceof Cat cat) {
4     cat.speak();
5 }

```

Downcasting tanpa pemeriksaan dapat menyebabkan `ClassCastException`. Jika desain terlalu sering membutuhkan downcasting, itu tanda bahwa abstraksi superclass kurang tepat atau polymorphism belum dimanfaatkan.

8.3.5 Abstract Class dan Final

Inti Konsep.

Abstract class mendefinisikan sebagian implementasi dan sebagian kontrak yang harus dilengkapi subclass. `final` membatasi inheritance atau overriding untuk menjaga kontrak.

Listing 46: Abstract class dan abstract method

```

1 public abstract class Shape {
2     public abstract double area();
3
4     public void printArea() {
5         System.out.println("Area = " + area());
6     }
7 }
8
9 public final class Circle extends Shape {
10     private final double radius;
11
12     public Circle(double radius) {
13         this.radius = radius;
14     }
15
16     @Override
17     public double area() {
18         return Math.PI * radius * radius;
19     }
20 }

```

Class `Circle` diberi `final` jika desain tidak mengizinkan subclass lanjutan. Ini berguna ketika class menjaga invariant yang tidak ingin diubah melalui overriding.

8.3.6 LSP dan Risiko Inheritance yang Salah

Inti Konsep.

Subclass harus mematuhi kontrak superclass. Jika subclass hanya cocok secara nama tetapi merusak ekspektasi method superclass, inheritance tersebut melanggar LSP.

Contoh klasik adalah `Square` extends `Rectangle` pada object mutable. Secara matematika terlihat benar, tetapi dalam kode, `Rectangle` yang mengizinkan `setWidth` dan `setHeight` independen tidak dapat digantikan oleh `Square` tanpa perilaku mengejutkan. Untuk ujian, argumen pentingnya adalah: inheritance harus diuji dari sudut pandang klien superclass.

8.4 Komponen Kunci Penting

Komponen Kunci	Makna atau Peran
<code>extends</code>	Kata kunci untuk mewarisi satu superclass.
Superclass	Class yang diwarisi.
Subclass	Class turunan yang memperluas atau mengkhususkan superclass.
<code>super</code>	Referensi ke constructor atau member superclass.
Overriding	Penggantian implementasi method superclass oleh subclass.
Dynamic dispatch	Pemilihan method berdasarkan tipe runtime object.
Abstract class	Class yang tidak dapat diinstansiasi dan dapat memiliki abstract method.
<code>final</code>	Pembatas inheritance, overriding, atau reassignment.
LSP	Prinsip bahwa subclass harus aman menggantikan superclass.

Tabel 12: Komponen kunci dalam materi Java Inheritance

8.5 Algoritma dan Rumus Penting

8.5.1 Prosedur Validasi Relasi Inheritance

Tujuan Algoritma.

Prosedur ini membantu memutuskan apakah inheritance layak dipakai atau sebaiknya diganti composition/interface.

Listing 47: Pseudocode validasi inheritance

```
1 Input: Kandidat subclass S dan superclass T
2 Output: Keputusan inheritance layak atau tidak
3
4 1. Tanyakan: apakah setiap S benar-benar T?
5 2. Tinjau semua method publik T.
6 3. Untuk setiap method:
7     periksa apakah S dapat menjalankan kontrak yang sama
8     tanpa memperketat precondition dan tanpa melemahkan postcondition.
9 4. Periksa invariant T:
10    S harus mempertahankan invariant tersebut.
11 5. Jika klien T perlu mengecek instanceof S:
12    curigai desain polymorphism belum tepat.
13 6. Jika S hanya memakai sebagian kecil perilaku T:
14    pertimbangkan composition.
15 7. Jika semua kontrak aman:
16    inheritance dapat digunakan.
```

8.6 Checklist Pemahaman

8.6.1 Submateri yang Telah Dibahas

- ☐ Saya dapat menjelaskan relasi *is-a*.
- ☐ Saya dapat menulis class dengan `extends` dan constructor `super`.
- ☐ Saya dapat membedakan overriding dan overloading.
- ☐ Saya dapat menjelaskan dynamic dispatch.
- ☐ Saya dapat memakai upcasting dan memahami risiko downcasting.
- ☐ Saya dapat membedakan abstract class dan concrete class.

- ☐ Saya dapat menjelaskan mengapa inheritance yang melanggar LSP berbahaya.

8.6.2 Prioritas Belajar

Prioritas	Materi yang Perlu dikuasai
Wajib dikuasai	<code>extends</code> , <code>super</code> , constructor chaining, overriding, dynamic dispatch, polymorphism, LSP.
Cukup Pahami Konsep	Downcasting, <code>instanceof</code> , field hiding, abstract class, <code>final</code> .
Sudah dikuasai	Materi ini pernah diuji dalam kuis, tetapi perlu diperdalam karena menjadi dasar interface, exception, dan design pattern.

9 Java Interface

Java Interface membahas cara Java mendefinisikan kontrak perilaku tanpa mengikat klien pada implementasi konkret. Interface adalah pusat dari abstraction, polymorphism, ISP, DIP, generics, collection framework, lambda expression, dan banyak design pattern. Karena Java hanya mengizinkan satu superclass, interface menjadi alat utama untuk membuat object memiliki banyak peran.

Komentar 9.1: Keterbatasan Sumber

NotebookLM pada materi SOLID menekankan bahwa Java menggunakan interface untuk menggantikan multiple inheritance class dalam konteks ISP dan DIP. Sintaks dan detail teknis interface di section ini dilengkapi menggunakan pengetahuan umum Java.

9.1 Outline Fundamental Konsep Materi

1. Konsep interface **[SUDAH DIKUASAI]**
 - (a) Interface sebagai kontrak method.
 - (b) Class mengimplementasikan interface dengan `implements`.
 - (c) Reference bertipe interface dapat menunjuk berbagai implementasi.
2. Method dalam interface **[SUDAH DIKUASAI]**
 - (a) Abstract method implicit `public abstract`.
 - (b) Default method dengan implementasi.
 - (c) Static method pada interface.
 - (d) Private helper method pada interface modern.
3. Field dalam interface
 - (a) Field interface implicit `public static final`.
 - (b) Interface bukan tempat state instance.
4. Multiple interface implementation
 - (a) Class dapat mengimplementasikan banyak interface.
 - (b) Konflik default method harus diselesaikan eksplisit.
5. Functional interface
 - (a) Interface dengan satu abstract method.
 - (b) Dapat digunakan dengan lambda.
 - (c) `@FunctionalInterface` untuk validasi.
6. Interface dalam desain
 - (a) ISP: interface kecil dan spesifik.
 - (b) DIP: modul high-level bergantung pada interface.
 - (c) Collection Framework dibangun di atas interface seperti `List`, `Set`, dan `Map`.

9.2 Pentingnya Materi dan Relevansinya

Inti Relevansi.

Interface memungkinkan kode bergantung pada apa yang bisa dilakukan object, bukan pada class konkret object tersebut. Ini membuat desain lebih fleksibel, mudah diuji, dan mudah diperluas.

Jika sebuah service bergantung langsung pada `EmailSender`, maka service sulit diganti untuk mengirim SMS, push notification, atau fake sender saat testing. Jika service bergantung pada interface `Notifier`, implementasi konkret dapat diganti tanpa mengubah service. Ini adalah inti polymorphism dan DIP dalam Java.

9.2.1 Dependensi dan Hubungan dengan Materi Lain

Interface berkaitan erat dengan SOLID, terutama ISP dan DIP. Collection memakai interface sebagai tipe utama. Generics sering memakai bound seperti `T extends Comparable<T>`. Stream API memakai functional interface seperti `Predicate`, `Function`, dan `Consumer`. Design Pattern seperti Strategy, Adapter, Observer, Factory, dan Repository sering dimodelkan dengan interface.

9.3 Penjelasan Konsep Fundamental

9.3.1 Interface sebagai Kontrak

Inti Konsep.

Interface mendefinisikan kontrak perilaku: method apa yang tersedia, bukan bagaimana method tersebut diimplementasikan.

Listing 48: Interface dan implementasi konkret

```
1 public interface PaymentGateway {
2     PaymentResult pay(Invoice invoice);
3 }
4
5 public class BankTransferGateway implements PaymentGateway {
6     @Override
7     public PaymentResult pay(Invoice invoice) {
8         return PaymentResult.success();
9     }
10 }
11
12 public class CheckoutService {
13     private final PaymentGateway gateway;
14
15     public CheckoutService(PaymentGateway gateway) {
16         this.gateway = gateway;
17     }
18
19     public PaymentResult checkout(Invoice invoice) {
20         return gateway.pay(invoice);
21     }
22 }
```

`CheckoutService` tidak perlu mengetahui apakah pembayaran dilakukan melalui transfer bank, kartu, atau dompet digital. Ia hanya membutuhkan kontrak `PaymentGateway`.

9.3.2 Default, Static, dan Private Method

Inti Konsep.

Interface modern dapat memiliki default method, static method, dan private helper method. Namun, state instance tetap milik class implementasi.

Default method memungkinkan interface menambahkan perilaku umum tanpa memaksa semua implementasi lama langsung berubah. Namun, default method harus dipakai hati-hati agar interface tidak menjadi terlalu gemuk.

Listing 49: Default dan static method pada interface

```
1 public interface Identifiable {
2     String id();
3
4     default boolean hasSameId(Identifiable other) {
5         return other != null && id().equals(other.id());
6     }
7
8     static boolean isValidId(String id) {
9         return id != null && !id.isBlank();
10    }
11 }
```

9.3.3 Multiple Interface dan Konflik Default Method

Inti Konsep.

Sebuah class dapat mengimplementasikan banyak interface. Jika dua interface memberi default method dengan signature sama, class harus memilih atau membuat implementasi sendiri.

Listing 50: Resolusi konflik default method

```
1 interface A {
2     default String name() {
3         return "A";
4     }
5 }
6
7 interface B {
8     default String name() {
9         return "B";
10    }
11 }
12
13 class C implements A, B {
14     @Override
15     public String name() {
16         return A.super.name() + B.super.name();
17     }
18 }
```

9.3.4 Functional Interface dan Lambda

Inti Konsep.

Functional interface adalah interface dengan tepat satu abstract method. Interface seperti ini dapat diimplementasikan menggunakan lambda expression.

Listing 51: Functional interface dan lambda

```
1 @FunctionalInterface
2 public interface Validator<T> {
3     boolean isValid(T value);
4 }
5
6 Validator<String> notBlank = text -> text != null && !text.isBlank();
7
8 System.out.println(notBlank.isValid("Java")); // true
```

Java standard library menyediakan functional interface umum seperti `Predicate<T>`, `Function<T,R>`, `Consumer<T>`, dan `Supplier<T>`. Ini menjadi dasar Stream API.

9.3.5 Interface, ISP, dan DIP

Inti Konsep.

Interface yang baik kecil, stabil, dan mencerminkan kebutuhan klien. Interface yang terlalu besar melanggar ISP; modul yang bergantung pada class konkret melanggar DIP.

Contoh interface terlalu besar:

Listing 52: Fat interface yang memaksa method tidak relevan

```
1 public interface Worker {
2     void work();
3     void eat();
4     void sleep();
5 }
```

Robot worker mungkin bisa `work`, tetapi tidak `eat` atau `sleep`. Desain lebih baik adalah memisahkan interface:

Listing 53: Interface kecil sesuai ISP

```
1 public interface Workable {
2     void work();
3 }
4
5 public interface Eatable {
6     void eat();
7 }
8
9 public class Robot implements Workable {
10     @Override
11     public void work() { }
12 }
```

9.4 Komponen Kunci Penting

Komponen Kunci	Makna atau Peran
interface	Deklarasi kontrak perilaku.
implements	Kata kunci class untuk memenuhi interface.
Default method	Method interface dengan implementasi default.
Static method	Method milik interface, dipanggil melalui nama interface.
Functional interface	Interface dengan satu abstract method, cocok untuk lambda.
@FunctionalInterface	Annotation untuk validasi functional interface.
ISP	Prinsip agar klien tidak bergantung pada method yang tidak digunakan.
DIP	Prinsip agar modul high-level bergantung pada abstraksi.

Tabel 13: Komponen kunci dalam materi Java Interface

9.5 Algoritma dan Rumus Penting

9.5.1 Prosedur Mendesain Interface

Tujuan Algoritma.

Prosedur ini membantu menentukan interface yang cukup abstrak tanpa menjadi terlalu besar.

Listing 54: Pseudocode desain interface

```
1 Input: Sekumpulan class dan kebutuhan klien
2 Output: Interface yang stabil dan spesifik
3
4 1. Identifikasi operasi yang benar-benar dibutuhkan klien.
5 2. Kelompokkan operasi berdasarkan alasan perubahan yang sama.
6 3. Jika satu calon interface memiliki method yang tidak dipakai sebagian klien:
7     pecah interface tersebut.
8 4. Hindari memasukkan state instance ke interface.
9 5. Gunakan default method hanya untuk perilaku umum yang stabil.
10 6. Pastikan implementasi dapat diganti tanpa mengubah klien.
11 7. Jika interface hanya punya satu abstract method dan akan dipakai sebagai
    lambda:
12     beri @FunctionalInterface.
```

9.6 Checklist Pemahaman

9.6.1 Submateri yang Telah Dibahas

- ☐ Saya dapat mendefinisikan interface dan implementasinya.
- ☐ Saya dapat menjelaskan polymorphism dengan reference bertipe interface.
- ☐ Saya dapat memakai default dan static method interface.
- ☐ Saya dapat menyelesaikan konflik default method.
- ☐ Saya dapat menjelaskan functional interface dan lambda.
- ☐ Saya dapat menghubungkan interface dengan ISP dan DIP.

9.6.2 Prioritas Belajar

Prioritas	Materi yang Perlu dikuasai
Wajib dikuasai	interface, implements, polymorphism, default method, functional interface, ISP, DIP.
Cukup Pahami Konsep	Static/private method interface, konflik default method, package <code>java.util.function</code> .
Sudah dikuasai	Materi ini pernah diuji dalam kuis, tetapi tetap krusial karena menjadi dasar collection, stream, dan design pattern.

10 Java Generics

Java Generics memungkinkan class, interface, dan method ditulis dengan parameter tipe sehingga kode tetap reusable sekaligus type-safe. Tanpa generics, collection seperti `List` hanya menyimpan `Object`, sehingga programmer harus melakukan cast manual dan risiko error baru muncul saat runtime. Dengan generics, compiler dapat memastikan bahwa `List<String>` hanya diperlakukan sebagai list berisi `String`.

Komentar 10.1: Keterbatasan Sumber

Section ini disusun menggunakan pengetahuan umum Java. Materi generics sangat berkaitan dengan Java Collection dan Wildcard, sehingga pembahasannya diarahkan untuk menjadi fondasi dua materi tersebut.

10.1 Outline Fundamental Konsep Materi

1. Motivasi generics **[SUDAH DIKUASAI]**
 - (a) Type safety saat kompilasi.
 - (b) Mengurangi cast manual.
 - (c) Membuat struktur data dan algoritma reusable.
2. Generic class dan generic interface **[SUDAH DIKUASAI]**
 - (a) Parameter tipe seperti `T`, `E`, `K`, dan `V`.
 - (b) Instansiasi seperti `Box<String>` atau `Map<String, Integer>`.
3. Generic method **[SUDAH DIKUASAI]**
 - (a) Parameter tipe milik method, bukan class.
 - (b) Type inference oleh compiler.
4. Bounded type parameter
 - (a) `T extends Number`.
 - (b) Multiple bounds seperti `T extends Number & Comparable<T>`.
5. Type erasure
 - (a) Informasi generic sebagian besar hilang di runtime.
 - (b) Tidak bisa `new T()`, `T.class`, atau array generic langsung.
6. Raw type dan heap pollution
 - (a) Raw type melemahkan type safety.
 - (b) Warning compiler harus dipahami, bukan diabaikan.

10.2 Pentingnya Materi dan Relevansinya

Inti Relevansi.

Generics adalah dasar type-safe programming di Java modern. Hampir semua API penting Java memakai generics: collection, stream, optional, comparator, future, dan functional interface.

Dalam ujian, generics biasanya diuji melalui pembacaan tipe: mengapa `List<String>` tidak dapat diisi `Integer`, mengapa `List<Integer>` bukan subtype dari `List<Number>`, kapan memakai bounded type parameter, dan apa dampak type erasure. Generics juga menjadi dasar untuk memahami wildcard pada materi berikutnya.

10.2.1 Dependensi dan Hubungan dengan Materi Lain

Generics bergantung pada pemahaman class, interface, inheritance, dan type system Java. Collection Framework hampir seluruhnya generic. Stream API memakai generic type dalam pipeline. Wildcard adalah perluasan generics untuk variance. Reflection memiliki keterbatasan karena type erasure.

10.3 Penjelasan Konsep Fundamental

10.3.1 Generic Class

Inti Konsep.

Generic class adalah class yang menerima parameter tipe. Parameter tipe membuat class dapat bekerja dengan banyak tipe tanpa kehilangan pemeriksaan tipe saat kompilasi.

Listing 55: Generic class sederhana

```
1 public class Box<T> {
2     private T value;
3
4     public Box(T value) {
5         this.value = value;
6     }
7
8     public T get() {
9         return value;
10    }
11
12    public void set(T value) {
13        this.value = value;
14    }
15 }
16
17 Box<String> nameBox = new Box<>("Java");
18 String name = nameBox.get();
```

Huruf T bukan tipe khusus; ia hanya konvensi untuk “type”. Konvensi umum lain adalah E untuk element, K untuk key, V untuk value, dan R untuk return/result.

10.3.2 Generic Method

Inti Konsep.

Generic method memiliki parameter tipe sendiri yang dideklarasikan sebelum return type. Parameter tipe method dapat dipakai meskipun class-nya bukan generic.

Listing 56: Generic method

```
1 public class ArrayUtil {
2     public static <T> T first(List<T> values) {
3         if (values.isEmpty()) {
4             throw new IllegalArgumentException("List kosong");
5         }
6         return values.get(0);
7     }
8 }
9
10 String firstName = ArrayUtil.first(List.of("Ayu", "Budi"));
11 Integer firstNumber = ArrayUtil.first(List.of(10, 20, 30));
```

Compiler biasanya dapat menyimpulkan tipe T dari argumen method, sehingga pemanggil tidak perlu menulis tipe secara eksplisit.

10.3.3 Bounded Type Parameter

Inti Konsep.

Bounded type parameter membatasi tipe yang boleh digunakan sehingga method dapat memakai operasi yang dijamin tersedia pada bound tersebut.

Jika parameter `T` tidak dibatasi, compiler hanya tahu bahwa `T` adalah subtype dari `Object`. Jika ingin memakai method khusus `Number`, buat bound `T extends Number`.

Listing 57: Bounded type parameter

```
1 public static <T extends Number> double sum(List<T> numbers) {  
2     double total = 0.0;  
3     for (T number : numbers) {  
4         total += number.doubleValue();  
5     }  
6     return total;  
7 }
```

Kata `extends` pada generics berarti batas atas, baik untuk class maupun interface. Multiple bounds dapat digunakan, tetapi jika ada bound class, ia harus ditulis paling depan.

Listing 58: Multiple bounds pada generic type

```
1 public static <T extends Number & Comparable<T>> T max(T a, T b) {  
2     return a.compareTo(b) >= 0 ? a : b;  
3 }
```

10.3.4 Type Erasure

Inti Konsep.

Java generics diimplementasikan dengan type erasure: compiler memakai informasi generic untuk type checking, lalu sebagian besar informasi tipe tersebut dihapus pada bytecode.

Akibat type erasure, `List<String>` dan `List<Integer>` memiliki raw runtime class yang sama, yaitu `List`. Ini menjaga kompatibilitas dengan kode Java lama sebelum generics, tetapi menimbulkan batasan.

- Tidak dapat membuat object `new T()` karena runtime tidak tahu class konkret `T`.
- Tidak dapat memakai `T.class`.
- Tidak dapat membuat array generic langsung seperti `new T[10]`.
- Tidak dapat overload method hanya berdasarkan parameter generic yang terhapus.

10.3.5 Raw Type dan Heap Pollution

Inti Konsep.

Raw type adalah penggunaan generic class tanpa argumen tipe, misalnya `List` alih-alih `List<String>`. Raw type melemahkan type safety dan dapat menyebabkan error runtime.

Listing 59: Raw type berbahaya

```
1 List<String> names = new ArrayList<>();  
2 List raw = names;  
3 raw.add(123); // Warning, tetapi masih bisa dikompilasi.  
4  
5 String first = names.get(0); // ClassCastException saat runtime.
```


Dalam kode modern, hindari raw type kecuali saat berinteraksi dengan API legacy. Warning unchecked harus dibaca sebagai tanda bahwa compiler tidak lagi dapat menjamin type safety.

10.4 Komponen Kunci Penting

Komponen Kunci	Makna atau Peran
Type parameter	Placeholder tipe seperti T, E, K, dan V.
Generic class	Class dengan parameter tipe.
Generic method	Method dengan parameter tipe sendiri.
Bounded type	Parameter tipe dengan batas atas seperti <code>T extends Number</code> .
Type erasure	Penghapusan sebagian informasi generic pada runtime.
Raw type	Generic type yang dipakai tanpa argumen tipe.
Heap pollution	Kondisi ketika variabel generic menyimpan object yang tidak sesuai tipe deklarasinya.

Tabel 14: Komponen kunci dalam materi Java Generics

10.5 Algoritma dan Rumus Penting

10.5.1 Prosedur Menentukan Parameter Generics

Tujuan Algoritma.

Prosedur ini membantu memilih antara generic class, generic method, bounded type, atau tipe konkret.

Listing 60: Pseudocode pemilihan generics

```
1 Input: Kode yang ingin dibuat reusable terhadap tipe
2 Output: Bentuk generics yang sesuai
3
4 1. Jika seluruh class menyimpan atau memproses tipe yang sama:
5     gunakan generic class/interface.
6 2. Jika hanya satu method yang membutuhkan parameter tipe:
7     gunakan generic method.
8 3. Jika method perlu memanggil operasi khusus dari tipe:
9     gunakan bounded type parameter.
10 4. Jika tipe hanya dibaca dari struktur data:
11     pertimbangkan wildcard extends pada materi wildcard.
12 5. Jika tipe hanya ditulis ke struktur data:
13     pertimbangkan wildcard super pada materi wildcard.
14 6. Hindari raw type.
```

10.6 Checklist Pemahaman

10.6.1 Submateri yang Telah Dibahas

- ☐ Saya dapat menjelaskan motivasi generics.
- ☐ Saya dapat menulis generic class dan generic method.
- ☐ Saya dapat memakai bounded type parameter.
- ☐ Saya dapat menjelaskan type erasure dan batasannya.
- ☐ Saya dapat menjelaskan bahaya raw type.

10.6.2 Prioritas Belajar

Prioritas	Materi yang Perlu dikuasai
Wajib dikuasai	Generic class, generic method, bounded type, type safety, raw type, type erasure.
Cukup Pahami Konsep	Multiple bounds, heap pollution, batasan <code>new T()</code> dan generic array.
Sudah dikuasai	Termasuk materi kuis 1 sampai 10, tetapi tetap perlu dipahami kuat untuk Collection dan Stream API.

11 Java Generics Wildcard

Generics wildcard membahas penggunaan tanda `?` untuk menyatakan tipe generic yang tidak diketahui secara tepat. Wildcard diperlukan karena generic Java bersifat invariant: `List<Integer>` bukan subtype dari `List<Number>` walaupun `Integer` adalah subtype dari `Number`. Untuk membuat API fleksibel tetapi tetap aman, Java menyediakan upper bounded wildcard `? extends T` dan lower bounded wildcard `? super T`.

Komentar 11.1: Keterbatasan Sumber

Section ini menggunakan pengetahuan umum Java. Wildcard adalah materi yang biasanya tidak dapat dipahami hanya dari sintaks; karena itu penekanan utama diberikan pada alasan type safety dan prinsip PECS.

11.1 Outline Fundamental Konsep Materi

1. Masalah invariance **[SUDAH DIKUASAI]**
 - (a) `List<Integer>` bukan subtype dari `List<Number>`.
 - (b) Invariance mencegah penulisan nilai yang tidak aman.
2. Unbounded wildcard **[SUDAH DIKUASAI]**
 - (a) `List<?>` berarti list dengan elemen tipe tidak diketahui.
 - (b) Aman untuk membaca sebagai `Object`, tidak aman untuk menambah selain `null`.
3. Upper bounded wildcard **[SUDAH DIKUASAI]**
 - (a) `? extends T`.
 - (b) Cocok untuk producer: data dibaca dari struktur.
4. Lower bounded wildcard
 - (a) `? super T`.
 - (b) Cocok untuk consumer: data ditulis ke struktur.
5. PECS
 - (a) Producer Extends.
 - (b) Consumer Super.
6. Wildcard capture
 - (a) Compiler dapat menangkap tipe wildcard dalam helper method.
 - (b) Berguna untuk operasi internal yang perlu tipe konsisten.

11.2 Pentingnya Materi dan Relevansinya

Inti Relevansi.

Wildcard membuat API generic cukup fleksibel untuk menerima banyak subtype, tetapi tetap menjaga type safety. Materi ini penting untuk memahami `Collection`, `Stream`, `Comparator`, dan banyak method library Java.

Tanpa wildcard, method yang menerima list number akan terlalu kaku. Dengan `List<? extends Number>`, method dapat menerima `List<Integer>`, `List<Double>`, dan `List<BigDecimal>`. Namun, fleksibilitas ini dibayar dengan batasan: jika tipe tepat elemen tidak diketahui, compiler membatasi operasi tulis.

11.2.1 Dependensi dan Hubungan dengan Materi Lain

Wildcard bergantung pada Generics, Inheritance, dan type system. Collection sering memakai wildcard pada method seperti `addAll`. Comparator memakai `Comparator<? super T>`. Stream API juga sering memanfaatkan variance dalam method generic.

11.3 Penjelasan Konsep Fundamental

11.3.1 Invariance pada Generic Java

Inti Konsep.

Generic Java invariant: meskipun `Integer` extends `Number`, `List<Integer>` bukan subtype dari `List<Number>`. Ini mencegah penulisan elemen yang salah tipe.

Jika `List<Integer>` boleh diperlakukan sebagai `List<Number>`, maka kode dapat menambahkan `Double` ke list integer, yang jelas merusak type safety.

Listing 61: Alasan `List<Integer>` bukan `List<Number>`

```
1 List<Integer> integers = new ArrayList<>();
2 // List<Number> numbers = integers; // Tidak boleh.
3
4 // Jika boleh, ini akan berbahaya:
5 // numbers.add(3.14);
6 // Integer x = integers.get(0); // Akan rusak.
```

11.3.2 Unbounded Wildcard

Inti Konsep.

`List<?>` berarti list berisi elemen dari tipe tertentu yang tidak diketahui. Elemen aman dibaca sebagai `Object`, tetapi tidak aman ditambah nilai konkret.

Listing 62: Unbounded wildcard

```
1 public static void printAll(List<?> values) {
2     for (Object value : values) {
3         System.out.println(value);
4     }
5 }
```

Karena tipe elemen tidak diketahui, `values.add("x")` tidak aman. List tersebut mungkin sebenarnya `List<Integer>`. Nilai yang boleh ditambahkan hanya `null`, tetapi ini jarang berguna.

11.3.3 Upper Bounded Wildcard: `? extends T`

Inti Konsep.

`? extends T` berarti tipe tidak diketahui yang merupakan subtype dari `T`. Struktur ini cocok sebagai producer karena elemen dapat dibaca sebagai `T`.

Listing 63: Upper bounded wildcard untuk membaca number

```
1 public static double sum(List<? extends Number> numbers) {
2     double total = 0.0;
3     for (Number number : numbers) {
4         total += number.doubleValue();
5     }
6     return total;
}
```

```

7 }
8
9 sum(List.of(1, 2, 3));
10 sum(List.of(1.5, 2.5));

```

Namun, kita tidak dapat menambahkan `Integer` ke `List<? extends Number>` karena list tersebut mungkin sebenarnya `List<Double>`. Compiler hanya menjamin bahwa elemen yang keluar minimal dapat diperlakukan sebagai `Number`.

11.3.4 Lower Bounded Wildcard: `? super T`

Inti Konsep.

`? super T` berarti tipe tidak diketahui yang merupakan supertype dari `T`. Struktur ini cocok sebagai consumer karena aman menerima nilai bertipe `T`.

Listing 64: Lower bounded wildcard untuk menulis data

```

1 public static void addDefaultIntegers(List<? super Integer> target) {
2     target.add(0);
3     target.add(1);
4     target.add(2);
5 }
6
7 List<Number> numbers = new ArrayList<>();
8 addDefaultIntegers(numbers);

```

Ketika membaca dari `List<? super Integer>`, compiler hanya aman menganggap hasilnya `Object`, karena list tersebut bisa `List<Integer>`, `List<Number>`, atau `List<Object>`.

11.3.5 PECS: Producer Extends, Consumer Super

Inti Konsep.

PECS adalah aturan praktis: jika sumber data menghasilkan nilai untuk dibaca, gunakan `extends`; jika target data menerima nilai untuk ditulis, gunakan `super`.

Listing 65: PECS pada copy method

```

1 public static <T> void copy(List<? extends T> source, List<? super T> target) {
2     for (T item : source) {
3         target.add(item);
4     }
5 }
6
7 List<Integer> ints = List.of(1, 2, 3);
8 List<Number> nums = new ArrayList<>();
9 copy(ints, nums);

```

`source` adalah producer karena menghasilkan `T`; `target` adalah consumer karena menerima `T`. Ini contoh paling penting untuk memahami wildcard.

11.4 Komponen Kunci Penting

Komponen Kunci	Makna atau Peran
Wildcard	Tanda ? untuk tipe generic yang tidak diketahui.
Invariant	Sifat bahwa <code>Generic<Sub></code> bukan subtype dari <code>Generic<Super></code> .
<code>? extends T</code>	Upper bounded wildcard; cocok untuk producer.
<code>? super T</code>	Lower bounded wildcard; cocok untuk consumer.
PECS	Producer Extends, Consumer Super.
Wildcard capture	Mekanisme compiler/helper method untuk menangkap tipe wildcard.

Tabel 15: Komponen kunci dalam materi Java Generics Wildcard

11.5 Algoritma dan Rumus Penting

11.5.1 Prosedur Memilih Wildcard

Tujuan Algoritma.

Prosedur ini membantu menentukan wildcard yang tepat berdasarkan operasi baca dan tulis.

Listing 66: Pseudocode memilih wildcard

```
1 Input: Parameter generic pada method
2 Output: Bentuk tipe parameter
3
4 1. Jika method hanya membaca elemen dari parameter:
5     gunakan ? extends T.
6 2. Jika method hanya menulis elemen T ke parameter:
7     gunakan ? super T.
8 3. Jika method membaca dan menulis elemen dengan tipe yang sama:
9     gunakan parameter tipe biasa T, bukan wildcard.
10 4. Jika method hanya butuh mencetak/mengecek ukuran:
11     gunakan ?.
12 5. Jika wildcard membuat kode internal sulit:
13     gunakan helper method dengan parameter tipe untuk capture.
```

11.6 Checklist Pemahaman

11.6.1 Submateri yang Telah Dibahas

- ☐ Saya dapat menjelaskan mengapa `List<Integer>` bukan `List<Number>`.
- ☐ Saya dapat memakai `List<?>`.
- ☐ Saya dapat memakai `? extends T` untuk producer.
- ☐ Saya dapat memakai `? super T` untuk consumer.
- ☐ Saya dapat menerapkan prinsip PECS.

11.6.2 Prioritas Belajar

Prioritas	Materi yang Perlu dikuasai
Wajib dikuasai	Invariance, ?, ? extends, ? super, PECS, contoh copy.
Cukup Pahami Konsep	Wildcard capture dan hubungan wildcard dengan Comparator/Collection API.
Sudah dikuasai	Pernah tercakup dalam materi kuis, tetapi perlu latihan karena sering membingungkan secara tipe.

12 Java Collection

Java Collection membahas framework struktur data standar Java untuk menyimpan, mencari, mengurutkan, mengelompokkan, dan memproses object. Collection Framework memakai interface seperti `Collection`, `List`, `Set`, `Queue`, dan `Map`, serta implementasi konkret seperti `ArrayList`, `LinkedList`, `HashSet`, `TreeSet`, `HashMap`, dan `TreeMap`.

Komentar 12.1: Keterbatasan Sumber

Section ini disusun berdasarkan pengetahuan umum Java. Fokus dipilih untuk kebutuhan ujian: pemilihan struktur data, kontrak equals-hashCode, ordering, generics, iterator, dan kompleksitas umum.

12.1 Outline Fundamental Konsep Materi

1. Hierarki Collection Framework **[SUDAH DIKUASAI]**
 - (a) `Iterable`, `Collection`, `List`, `Set`, `Queue`.
 - (b) `Map` bukan subtype `Collection`, tetapi bagian dari framework.
2. `List` **[SUDAH DIKUASAI]**
 - (a) `Ordered`, boleh duplikat, akses berdasarkan indeks.
 - (b) `ArrayList` vs `LinkedList`.
3. `Set` **[SUDAH DIKUASAI]**
 - (a) Tidak menyimpan duplikat.
 - (b) `HashSet`, `LinkedHashSet`, `TreeSet`.
4. `Map` **[SUDAH DIKUASAI]**
 - (a) Pasangan key-value.
 - (b) Key unik; value boleh duplikat.
 - (c) `HashMap`, `LinkedHashMap`, `TreeMap`.
5. Iterator dan enhanced for
 - (a) Iterasi aman dengan `Iterator`.
 - (b) `ConcurrentModificationException` saat modifikasi tidak tepat.
6. Equality, hashing, dan ordering
 - (a) `equals` dan `hashCode` untuk hash-based collection.
 - (b) `Comparable` dan `Comparator` untuk ordering.
7. Pemilihan collection
 - (a) Berdasarkan duplikasi, urutan, pencarian, indeks, dan sorting.

12.2 Pentingnya Materi dan Relevansinya

Inti Relevansi.

Collection adalah API inti untuk menyimpan banyak object. Kemampuan memilih collection yang tepat menentukan kebenaran, performa, dan kejelasan desain program Java.

Jika data membutuhkan urutan input dan akses indeks, **List** cocok. Jika data harus unik, **Set** cocok. Jika data dicari berdasarkan key, **Map** cocok. Kesalahan memilih collection sering menyebabkan kode tambahan yang tidak perlu, performa buruk, atau bug karena duplikasi dan urutan tidak dipahami.

12.2.1 Dependensi dan Hubungan dengan Materi Lain

Collection bergantung pada Generics dan Interface. String sering diproses dalam collection. Stream API adalah cara deklaratif memproses collection. Equality/hashing berkaitan dengan desain class dan enkapsulasi. Wildcard muncul pada method collection seperti `addAll(Collection<? extends E>)`.

12.3 Penjelasan Konsep Fundamental

12.3.1 List

Inti Konsep.

List adalah collection berurutan yang mengizinkan duplikat dan menyediakan akses berdasarkan indeks.

Listing 67: Penggunaan List

```
1 List<String> names = new ArrayList<>();
2 names.add("Ayu");
3 names.add("Budi");
4 names.add("Ayu");
5
6 System.out.println(names.get(0)); // Ayu
7 System.out.println(names.size()); // 3
```

ArrayList umumnya menjadi pilihan default untuk list karena akses indeks cepat dan performa iterasi baik. **LinkedList** jarang menjadi pilihan default; meskipun insertion di tengah secara teori dapat murah jika node sudah diketahui, pencarian posisi tetap mahal.

12.3.2 Set

Inti Konsep.

Set menyimpan elemen unik. Keunikan bergantung pada `equals/hashCode` untuk hash set atau ordering untuk tree set.

Listing 68: Penggunaan Set untuk menghapus duplikasi

```
1 Set<String> uniqueNames = new HashSet<>();
2 uniqueNames.add("Ayu");
3 uniqueNames.add("Budi");
4 uniqueNames.add("Ayu");
5
6 System.out.println(uniqueNames.size()); // 2
```

HashSet tidak menjamin urutan. **LinkedHashSet** mempertahankan urutan insertion. **TreeSet** menjaga elemen terurut berdasarkan **Comparable** atau **Comparator**.

12.3.3 Map

Inti Konsep.

Map menyimpan pasangan key-value. Key harus unik; value boleh sama. Map cocok ketika data perlu dicari berdasarkan identitas tertentu.

Listing 69: Penggunaan Map

```
1 Map<String, Integer> scores = new HashMap<>();
2 scores.put("Ayu", 90);
3 scores.put("Budi", 85);
4 scores.put("Ayu", 95); // Mengganti value untuk key Ayu.
5
6 System.out.println(scores.get("Ayu")); // 95
```

Iterasi map biasanya dilakukan melalui `entrySet` agar key dan value tersedia bersamaan.

Listing 70: Iterasi Map dengan `entrySet`

```
1 for (Map.Entry<String, Integer> entry : scores.entrySet()) {
2     System.out.println(entry.getKey() + " = " + entry.getValue());
3 }
```

12.3.4 Equals dan hashCode

Inti Konsep.

Object yang digunakan sebagai key `HashMap` atau elemen `HashSet` harus memiliki kontrak `equals` dan `hashCode` yang konsisten.

Kontrak pentingnya: jika `a.equals(b)` bernilai `true`, maka `a.hashCode()` dan `b.hashCode()` harus sama. Jika kontrak ini dilanggar, lookup pada `HashMap` atau `HashSet` dapat gagal.

Listing 71: Record otomatis memiliki `equals` dan `hashCode` berbasis komponen

```
1 public record StudentId(String value) { }
2
3 Map<StudentId, String> students = new HashMap<>();
4 students.put(new StudentId("13524044"), "Narendra");
5
6 System.out.println(students.get(new StudentId("13524044"))); // Narendra
```

12.3.5 Comparable dan Comparator

Inti Konsep.

`Comparable` mendefinisikan natural ordering pada class, sedangkan `Comparator` mendefinisikan ordering eksternal yang dapat berubah sesuai kebutuhan.

Listing 72: Sorting dengan `Comparator`

```
1 List<Student> students = new ArrayList<>(List.of(
2     new Student("Ayu", 90),
3     new Student("Budi", 85)
4 ));
5
6 students.sort(Comparator
7     .comparingInt(Student::score)
8     .reversed()
9     .thenComparing(Student::name));
```

12.4 Komponen Kunci Penting

Komponen Kunci	Makna atau Peran
List	Collection berurutan, boleh duplikat, akses indeks.
Set	Collection unik tanpa duplikasi.
Map	Struktur key-value dengan key unik.
ArrayList	Implementasi list berbasis array dinamis.
HashSet	Set berbasis hashing.
HashMap	Map berbasis hashing.
TreeSet/TreeMap	Collection/map terurut.
Iterator	Object untuk mengiterasi collection.
Comparable	Natural ordering.
Comparator	Ordering eksternal.

Tabel 16: Komponen kunci dalam materi Java Collection

12.5 Algoritma dan Rumus Penting

12.5.1 Prosedur Memilih Collection

Tujuan Algoritma.

Prosedur ini membantu memilih struktur data Java berdasarkan kebutuhan data.

Listing 73: Pseudocode pemilihan collection

```
1 Input: Kebutuhan penyimpanan data
2 Output: Jenis collection yang sesuai
3
4 1. Jika data dipetakan dari key ke value:
5     gunakan Map.
6 2. Jika data harus unik:
7     gunakan Set.
8 3. Jika data perlu urutan dan akses indeks:
9     gunakan List.
10 4. Jika perlu urutan insertion pada set/map:
11     gunakan LinkedHashSet atau LinkedHashMap.
12 5. Jika perlu urutan sorted:
13     gunakan TreeSet atau TreeMap, atau sort List dengan Comparator.
14 6. Jika operasi utama adalah FIFO:
15     gunakan Queue atau Deque.
16 7. Jika operasi banyak dilakukan secara concurrent:
17     pertimbangkan collection dari java.util.concurrent.
```

12.5.2 Kompleksitas Umum

Makna Rumus.

Kompleksitas membantu membandingkan biaya operasi collection. Nilai berikut adalah aturan umum, bukan jaminan absolut untuk semua kondisi.

Struktur	Operasi Umum	Kompleksitas Rata-Rata
ArrayList	get(i)	$O(1)$
ArrayList	tambah di akhir	amortized $O(1)$
HashSet/HashMap	add/get/contains	rata-rata $O(1)$
TreeSet/TreeMap	add/get/contains	$O(\log n)$

Tabel 17: Kompleksitas umum collection Java

12.6 Checklist Pemahaman

12.6.1 Submateri yang Telah Dibahas

- ☐ Saya dapat membedakan List, Set, dan Map.
- ☐ Saya dapat memilih ArrayList, HashSet, HashMap, atau struktur lain sesuai kebutuhan.
- ☐ Saya dapat menjelaskan kontrak equals dan hashCode.
- ☐ Saya dapat memakai Comparable dan Comparator.
- ☐ Saya dapat membaca kompleksitas umum collection.

12.6.2 Prioritas Belajar

Prioritas	Materi yang Perlu dikuasai
Wajib Dikuasai	List, Set, Map, implementasi utama, equals-hashCode, Comparator, pemilihan collection.
Cukup Paham Konsep	Iterator, fail-fast behavior, kompleksitas umum, concurrent collection.
Sudah Dikuasai	Termasuk materi kuis, tetapi penting sebagai basis Stream API dan soal implementasi Java.

13 Java Stream API

Java Stream API adalah API untuk memproses rangkaian data secara deklaratif melalui pipeline operasi. Stream bukan struktur data; stream adalah aliran elemen dari sumber seperti collection, array, file, atau generator. Stream membantu menulis transformasi data seperti filter, map, sort, group, dan reduce dengan gaya yang lebih ringkas dan komposisional.

Komentar 13.1: Keterbatasan Sumber

Section ini disusun berdasarkan pengetahuan umum Java. Fokus pembahasan adalah konsep pipeline, lazy evaluation, intermediate-terminal operation, functional interface, dan contoh dunia nyata yang sering muncul.

13.1 Outline Fundamental Konsep Materi

1. Konsep stream **[SUDAH DIKUASAI]**
 - (a) Stream adalah pipeline pemrosesan data, bukan penyimpanan data.
 - (b) Sumber stream dapat berupa collection, array, file, atau generator.
2. Pipeline stream **[SUDAH DIKUASAI]**
 - (a) Source.
 - (b) Intermediate operations.
 - (c) Terminal operation.
3. Intermediate operation **[SUDAH DIKUASAI]**
 - (a) `filter`, `map`, `flatMap`, `sorted`, `distinct`, `limit`, `skip`.
 - (b) Bersifat lazy.
4. Terminal operation
 - (a) `collect`, `toList`, `forEach`, `reduce`, `count`, `anyMatch`.
 - (b) Menjalankan pipeline dan mengonsumsi stream.
5. Functional interface dan lambda
 - (a) `Predicate`, `Function`, `Consumer`, `Comparator`, `BinaryOperator`.
6. Reduce dan collect
 - (a) Reduction menjadi satu nilai.
 - (b) Collector untuk list, set, map, grouping, dan joining.
7. Parallel stream
 - (a) Potensi percepatan.
 - (b) Risiko side effect, ordering, dan overhead.

13.2 Pentingnya Materi dan Relevansinya

Inti Relevansi.

Stream API penting karena banyak tugas pemrosesan collection lebih mudah dibaca sebagai transformasi pipeline: pilih data, ubah bentuk data, kelompokkan, lalu hasilkan output.

Stream mendukung gaya deklaratif: programmer menyatakan apa yang ingin dilakukan, bukan detail loop-nya. Namun, stream bukan pengganti semua loop. Jika operasi memiliki banyak state mutasi, side effect kompleks, atau membutuhkan kontrol alur yang rumit, loop biasa bisa lebih jelas.

13.2.1 Dependensi dan Hubungan dengan Materi Lain

Stream bergantung pada Collection, Generics, Interface, Lambda, dan Functional Interface. Exception menjadi relevan karena lambda pada stream tidak mudah melempar checked exception. Multithreading muncul pada parallel stream. Design Pattern dapat dikaitkan dengan pipeline dan strategy.

13.3 Penjelasan Konsep Fundamental

13.3.1 Pipeline Stream

Inti Konsep.

Pipeline stream terdiri atas source, nol atau lebih intermediate operation, dan satu terminal operation. Intermediate operation lazy; terminal operation memicu eksekusi.

Listing 74: Pipeline Stream dasar

```
1 List<String> names = List.of("Ayu", "Budi", "Citra", "Andi");
2
3 List<String> result = names.stream()
4     .filter(name -> name.startsWith("A"))
5     .map(String::toUpperCase)
6     .sorted()
7     .toList();
8
9 System.out.println(result); // [ANDI, AYU]
```

Pipeline tersebut membaca data dari list, memilih nama yang diawali A, mengubah ke huruf besar, mengurutkan, lalu menghasilkan list baru.

13.3.2 Lazy Evaluation

Inti Konsep.

Intermediate operation pada stream tidak langsung berjalan. Operasi baru dieksekusi ketika terminal operation dipanggil.

Listing 75: Lazy evaluation pada stream

```
1 Stream<String> stream = names.stream()
2     .filter(name -> {
3         System.out.println("filter " + name);
4         return name.startsWith("A");
5     });
6
7 // Belum ada output karena belum ada terminal operation.
8 long count = stream.count(); // Baru pipeline berjalan.
```

Lazy evaluation memungkinkan optimisasi seperti short-circuit. Misalnya, `findFirst` tidak perlu memproses semua elemen jika hasil sudah ditemukan.

13.3.3 Map vs FlatMap

Inti Konsep.

`map` mengubah setiap elemen menjadi satu elemen lain. `flatMap` mengubah setiap elemen menjadi stream lalu meratakannya menjadi satu stream.

Listing 76: map dan flatMap

```
1 List<String> sentences = List.of("Java Stream", "OOP UAS");
2
3 List<String> words = sentences.stream()
4     .flatMap(sentence -> Arrays.stream(sentence.split(" ")))
5     .map(String::toLowerCase)
6     .toList();
7
8 System.out.println(words); // [java, stream, oop, uas]
```

13.3.4 Reduce

Inti Konsep.

`reduce` menggabungkan banyak elemen menjadi satu nilai menggunakan accumulator. Operasi ini cocok untuk sum, product, min, max, atau penggabungan custom.

Listing 77: Reduce untuk menjumlahkan nilai

```
1 int total = List.of(1, 2, 3, 4).stream()
2     .reduce(0, (a, b) -> a + b);
3
4 System.out.println(total); // 10
```

Accumulator pada `reduce` sebaiknya associative, terutama jika stream paralel. Jangan mengubah external mutable state dari dalam lambda `reduce`.

13.3.5 Collect dan Grouping

Inti Konsep.

`collect` mengumpulkan hasil stream ke struktur data atau bentuk agregat. Collector umum adalah `toList`, `toSet`, `toMap`, `groupingBy`, dan `joining`.

Listing 78: Grouping dengan Collectors

```
1 record Student(String name, String className, int score) { }
2
3 Map<String, List<Student>> byClass = students.stream()
4     .collect(Collectors.groupingBy(Student::className));
5
6 Map<String, Double> averageByClass = students.stream()
7     .collect(Collectors.groupingBy(
8         Student::className,
9         Collectors.averagingInt(Student::score)
10    ));
```

13.3.6 Side Effect dan Parallel Stream

Inti Konsep.

Stream paling aman ketika operasi lambdanya stateless dan tidak memiliki side effect. Parallel stream hanya aman dan berguna jika operasi dapat diparalelkan tanpa konflik state.

Contoh buruk:

Listing 79: Side effect buruk pada stream

```
1 List<String> result = new ArrayList<>();
2 names.parallelStream()
3   .filter(name -> name.startsWith("A"))
4   .forEach(result::add); // Tidak aman tanpa sinkronisasi.
```

Gunakan collector:

Listing 80: Menghindari side effect dengan collector

```
1 List<String> result = names.parallelStream()
2   .filter(name -> name.startsWith("A"))
3   .toList();
```

13.4 Komponen Kunci Penting

Komponen Kunci	Makna atau Peran
Stream	Pipeline pemrosesan elemen.
Source	Asal elemen stream seperti collection atau array.
Intermediate operation	Operasi lazy yang menghasilkan stream baru.
Terminal operation	Operasi yang mengeksekusi dan mengonsumsi stream.
filter	Memilih elemen berdasarkan predicate.
map	Mengubah elemen.
flatMap	Mengubah elemen menjadi stream lalu meratakan.
reduce	Menggabungkan elemen menjadi satu nilai.
collect	Mengumpulkan hasil stream.

Tabel 18: Komponen kunci dalam materi Java Stream API

13.5 Algoritma dan Rumus Penting

13.5.1 Template Pipeline Stream

Tujuan Algoritma.

Template ini membantu menyusun stream dari kebutuhan pemrosesan data.

Listing 81: Pseudocode desain pipeline stream

```
1 Input: Collection data dan tujuan transformasi
2 Output: Hasil pemrosesan
3
4 1. Tentukan source stream.
5 2. Jika perlu menyaring data:
6   gunakan filter.
7 3. Jika perlu mengubah bentuk elemen:
8   gunakan map.
9 4. Jika setiap elemen menghasilkan banyak elemen:
10  gunakan flatMap.
```



```

11 5. Jika perlu mengurutkan atau menghapus duplikat:
12     gunakan sorted atau distinct.
13 6. Pilih terminal operation:
14     toList untuk list baru,
15     collect untuk grouping/map,
16     reduce untuk satu nilai,
17     anyMatch/allMatch untuk predicate global.
18 7. Hindari side effect pada lambda.

```

13.6 Checklist Pemahaman

13.6.1 Submateri yang Telah Dibahas

- ☐ Saya dapat menjelaskan stream sebagai pipeline, bukan struktur data.
- ☐ Saya dapat membedakan intermediate dan terminal operation.
- ☐ Saya dapat menjelaskan lazy evaluation.
- ☐ Saya dapat memakai `filter`, `map`, `flatMap`, `reduce`, dan `collect`.
- ☐ Saya dapat memakai `Collectors.groupingBy`.
- ☐ Saya dapat menjelaskan risiko side effect dan parallel stream.

13.6.2 Prioritas Belajar

Prioritas	Materi yang Perlu dikuasai
Wajib dikuasai	Pipeline, lazy evaluation, intermediate-terminal operation, <code>filter</code> , <code>map</code> , <code>flatMap</code> , <code>reduce</code> , <code>collect</code> .
Cukup Pahami Konsep	Parallel stream, side effect, short-circuit operation, collector lanjutan.
Sudah dikuasai	Termasuk materi yang pernah diuji, tetapi perlu pemahaman kuat karena sering keluar dalam soal implementasi ringkas.

14 Java Exception

Java Exception membahas mekanisme Java untuk merepresentasikan dan menangani kondisi error atau kejadian abnormal saat program berjalan. Exception membuat error dapat dipisahkan dari alur normal program, tetapi tetap harus dipakai dengan desain yang disiplin. Materi ini baru menurut isian awal, sehingga prioritasnya adalah memahami hierarki exception, checked vs unchecked exception, `try-catch-finally`, `throw`, `throws`, `try-with-resources`, dan cara merancang exception yang tidak merusak keterbacaan program.

Komentar 14.1: Keterbatasan Sumber

Section ini disusun menggunakan pengetahuan umum Java karena sumber NotebookLM yang terakses tidak memberikan pembahasan detail exception. Penjelasan dibuat mendalam karena materi ini disebut sebagai materi baru.

14.1 Outline Fundamental Konsep Materi

1. Konsep exception
 - (a) Exception merepresentasikan kondisi abnormal.
 - (b) Exception mengubah control flow dari lokasi error ke handler.
 - (c) Exception bukan pengganti validasi alur normal.
2. Hierarki exception
 - (a) `Throwable`.
 - (b) `Error`.
 - (c) `Exception`.
 - (d) `RuntimeException`.
3. Checked vs unchecked exception
 - (a) Checked exception wajib ditangani atau dideklarasikan.
 - (b) Unchecked exception tidak wajib ditangani.
 - (c) Error umumnya tidak ditangani aplikasi biasa.
4. Sintaks penanganan
 - (a) `try`.
 - (b) `catch`.
 - (c) `finally`.
 - (d) Multi-catch.
5. Melempar exception
 - (a) `throw` untuk melempar object exception.
 - (b) `throws` untuk mendeklarasikan potensi exception pada signature method.
6. Resource management
 - (a) `try-with-resources`.
 - (b) `AutoCloseable`.
7. Custom exception dan desain
 - (a) Kapan membuat exception sendiri.
 - (b) Menjaga pesan error, cause, dan level abstraksi.

14.2 Pentingnya Materi dan Relevansinya

Inti Relevansi.

Exception penting karena program nyata selalu berhadapan dengan input invalid, file hilang, koneksi gagal, parsing gagal, dan state yang tidak sesuai asumsi. Exception yang dirancang baik membuat error eksplisit dan dapat ditangani di level yang tepat.

Dalam OOP, exception adalah bagian dari kontrak method. Jika sebuah method dapat gagal karena kondisi eksternal, pemanggil harus tahu apakah kegagalan itu perlu ditangani. Namun, exception yang terlalu banyak atau terlalu umum juga merusak desain. Menangkap **Exception** secara luas, mengabaikan catch block, atau mengubah semua error menjadi unchecked exception tanpa alasan membuat program sulit didebug.

14.2.1 Dependensi dan Hubungan dengan Materi Lain

Exception menggunakan inheritance karena semua exception adalah subtype dari **Throwable**. Exception berhubungan dengan String untuk pesan error dan parsing, Collection/Stream untuk operasi yang dapat gagal, Assertion untuk validasi asumsi internal, Multithreading untuk exception pada thread, dan Design Pattern seperti Template Method atau Chain of Responsibility pada error handling.

14.3 Penjelasan Konsep Fundamental

14.3.1 Hierarki Exception

Inti Konsep.

Semua exception dan error di Java berakar dari **Throwable**. Cabang utama adalah **Error** untuk masalah serius pada runtime dan **Exception** untuk kondisi yang mungkin ditangani aplikasi.

Listing 82: Hierarki konseptual Throwable

```
1 Throwable
2 |-- Error
3 |   |-- OutOfMemoryError
4 |   |-- StackOverflowError
5 |
6 |-- Exception
7 |   |-- IOException           (checked)
8 |   |-- SQLException         (checked)
9 |   |-- RuntimeException      (unchecked)
10 |   |-- NullPointerException
11 |   |-- IllegalArgumentException
12 |   |-- IndexOutOfBoundsException
```

Error biasanya tidak ditangkap karena menandakan masalah serius seperti memory habis. **Exception** adalah kejadian yang lebih mungkin ditangani. **RuntimeException** adalah exception yang tidak wajib dideklarasikan atau ditangkap.

14.3.2 Checked dan Unchecked Exception

Inti Konsep.

Checked exception diperiksa compiler dan harus ditangani dengan **catch** atau dideklarasikan dengan **throws**. Unchecked exception adalah subtype **RuntimeException** dan tidak wajib ditangani.

Checked exception cocok untuk kegagalan eksternal yang realistis dan dapat dipulihkan, misalnya file tidak ditemukan atau koneksi gagal. Unchecked exception cocok untuk bug pemrograman atau pelanggaran kontrak, misalnya argumen invalid, indeks di luar batas, atau object null yang tidak seharusnya.

Listing 83: Checked exception wajib ditangani atau dideklarasikan

```
1 public String readFirstLine(Path path) throws IOException {
2     try (BufferedReader reader = Files.newBufferedReader(path)) {
3         return reader.readLine();
4     }
5 }
```

Listing 84: Unchecked exception untuk argumen invalid

```
1 public void withdraw(long amount) {
2     if (amount <= 0) {
3         throw new IllegalArgumentException("Amount harus positif");
4     }
5 }
```

14.3.3 try, catch, dan finally

Inti Konsep.

try membungkus kode yang mungkin gagal, catch menangani exception tertentu, dan finally menjalankan kode pembersihan baik terjadi exception maupun tidak.

Listing 85: try-catch-finally

```
1 FileInputStream input = null;
2 try {
3     input = new FileInputStream("data.txt");
4     int firstByte = input.read();
5     System.out.println(firstByte);
6 } catch (IOException e) {
7     System.err.println("Gagal membaca file: " + e.getMessage());
8 } finally {
9     if (input != null) {
10        try {
11            input.close();
12        } catch (IOException ignored) {
13            // Hindari mengabaikan exception penting pada kode nyata.
14        }
15    }
16 }
```

Kode di atas valid, tetapi verbose. Untuk resource, **try-with-resources** lebih idiomatis.

14.3.4 try-with-resources

Inti Konsep.

try-with-resources otomatis menutup resource yang mengimplementasikan **AutoCloseable**. Ini menggantikan banyak penggunaan **finally** untuk resource.

Listing 86: try-with-resources

```
1 public List<String> readAllLines(Path path) throws IOException {
2     try (BufferedReader reader = Files.newBufferedReader(path)) {
3         List<String> lines = new ArrayList<>();
4         String line;
```

```

5         while ((line = reader.readLine()) != null) {
6             lines.add(line);
7         }
8         return lines;
9     }
10 }

```

Resource ditutup otomatis setelah blok `try` selesai, baik normal maupun karena exception. Jika exception terjadi saat menutup resource, Java dapat menyimpannya sebagai suppressed exception.

14.3.5 throw dan throws

Inti Konsep.

`throw` adalah aksi melempar exception; `throws` adalah deklarasi pada method bahwa method tersebut dapat melempar exception tertentu.

Listing 87: throw vs throws

```

1 public User findUser(String id) throws UserNotFoundException {
2     if (id == null || id.isBlank()) {
3         throw new IllegalArgumentException("id kosong");
4     }
5
6     return repository.findById(id)
7         .orElseThrow(() -> new UserNotFoundException(id));
8 }

```

Exception argumen invalid tidak perlu masuk `throws` karena unchecked. Exception user tidak ditemukan perlu dideklarasikan jika ia checked exception.

14.3.6 Custom Exception

Inti Konsep.

Custom exception dibuat ketika domain aplikasi membutuhkan jenis error yang lebih spesifik daripada exception standar. Exception harus membawa pesan jelas dan, bila membungkus error lain, menyimpan cause.

Listing 88: Custom checked exception

```

1 public class UserNotFoundException extends Exception {
2     public UserNotFoundException(String id) {
3         super("User tidak ditemukan: " + id);
4     }
5 }

```

Listing 89: Custom unchecked exception untuk domain

```

1 public class InvalidOrderException extends RuntimeException {
2     public InvalidOrderException(String message) {
3         super(message);
4     }
5
6     public InvalidOrderException(String message, Throwable cause) {
7         super(message, cause);
8     }
9 }

```

14.4 Komponen Kunci Penting

Komponen Kunci	Makna atau Peran
Throwable	Akar hierarki error dan exception.
Exception	Kelas dasar kondisi abnormal yang mungkin ditangani aplikasi.
RuntimeException	Exception unchecked.
Error	Masalah serius runtime yang umumnya tidak ditangani aplikasi.
try/catch	Struktur penanganan exception.
finally	Blok pembersihan yang selalu dijalankan.
throw	Melempar exception.
throws	Mendeklarasikan exception pada method.
AutoCloseable	Kontrak resource yang dapat ditutup otomatis.

Tabel 19: Komponen kunci dalam materi Java Exception

14.5 Algoritma dan Rumus Penting

14.5.1 Prosedur Memilih Jenis Exception

Tujuan Algoritma.

Prosedur ini membantu menentukan apakah error harus menjadi checked exception, unchecked exception, atau cukup ditangani secara lokal.

Listing 90: Pseudocode pemilihan exception

```
1 Input: Kondisi error yang mungkin terjadi
2 Output: Strategi exception
3
4 1. Jika kondisi adalah bug pemrograman atau argumen melanggar kontrak:
5     gunakan unchecked exception.
6 2. Jika kondisi berasal dari faktor eksternal dan pemanggil realistis dapat
7     memulihkan:
8     gunakan checked exception atau API yang memaksa handling.
9 3. Jika error dapat diselesaikan di method yang sama:
10    tangani secara lokal dan jangan bocorkan detail.
11 4. Jika error perlu diteruskan ke layer lebih tinggi:
12    lempar exception dengan pesan jelas.
13 5. Jika membungkus exception lain:
14    sertakan cause.
15 6. Jangan menangkap Exception terlalu umum kecuali di boundary aplikasi.
```

14.6 Checklist Pemahaman

14.6.1 Submateri yang Telah Dibahas

- ☐ Saya dapat menjelaskan hierarki Throwable.
- ☐ Saya dapat membedakan checked dan unchecked exception.
- ☐ Saya dapat memakai try-catch-finally.
- ☐ Saya dapat memakai try-with-resources.
- ☐ Saya dapat membedakan throw dan throws.
- ☐ Saya dapat membuat custom exception yang masuk akal.
- ☐ Saya dapat menentukan level yang tepat untuk menangani exception.

14.6.2 Prioritas Belajar

Prioritas	Materi yang Perlu dikuasai
Wajib dikuasai	Hierarki exception, checked vs unchecked, <code>try-catch-finally</code> , <code>throw</code> , <code>throws</code> , <code>try-with-resources</code> .
Cukup Pahami Konsep	Custom exception, suppressed exception, cause chaining, best practice boundary handling.
Sudah dikuasai	Belum ditentukan; materi ini disebut sebagai materi baru sehingga harus diprioritaskan tinggi.

15 Java Assertion

Java Assertion membahas penggunaan kata kunci **assert** untuk memeriksa asumsi internal program saat development dan debugging. Assertion berbeda dari exception untuk validasi input publik. Assertion dipakai untuk menyatakan kondisi yang seharusnya selalu benar jika program ditulis dengan benar, misalnya invariant internal, postcondition sederhana, atau kondisi yang tidak mungkin terjadi.

Komentar 15.1: Keterbatasan Sumber

Section ini menggunakan pengetahuan umum Java. Assertion sering terlihat kecil, tetapi penting dibedakan dari exception agar tidak salah dipakai untuk validasi runtime yang wajib berjalan.

15.1 Outline Fundamental Konsep Materi

1. Konsep assertion
 - (a) Memeriksa asumsi internal.
 - (b) Dapat diaktifkan atau dinonaktifkan saat runtime.
 - (c) Jika gagal, melempar **AssertionError**.
2. Sintaks assertion
 - (a) `assert expression;`
 - (b) `assert expression : message;`
3. Aktivasi assertion
 - (a) `java -ea.`
 - (b) `java -enableassertions.`
 - (c) Default-nya assertion tidak aktif.
4. Kapan memakai assertion
 - (a) Invariant internal.
 - (b) Cabang yang tidak seharusnya dicapai.
 - (c) Pemeriksaan hasil antara dalam algoritma.
5. Kapan tidak memakai assertion
 - (a) Validasi parameter public method.
 - (b) Validasi input pengguna.
 - (c) Kode dengan side effect penting.

15.2 Pentingnya Materi dan Relevansinya

Inti Relevansi.

Assertion membantu menemukan bug internal lebih awal, tetapi tidak boleh menjadi satu-satunya mekanisme validasi yang dibutuhkan program saat production.

Karena assertion dapat dimatikan, program tidak boleh bergantung pada **assert** untuk menjaga keamanan, validasi input eksternal, atau perubahan state penting. Assertion adalah alat dokumentasi executable untuk asumsi programmer: jika asumsi salah, ada bug yang perlu diperbaiki.

15.2.1 Dependensi dan Hubungan dengan Materi Lain

Assertion berkaitan dengan Exception karena sama-sama dapat menghentikan alur program, tetapi tujuannya berbeda. Exception menangani kondisi gagal yang mungkin terjadi, sedangkan assertion memeriksa asumsi internal. Assertion juga berkaitan dengan inheritance dan design by contract karena dapat memeriksa invariant dan postcondition.

15.3 Penjelasan Konsep Fundamental

15.3.1 Sintaks Dasar Assertion

Inti Konsep.

Assertion ditulis dengan `assert kondisi;` atau `assert kondisi : pesan;`. Jika kondisi false dan assertion aktif, JVM melempar `AssertionError`.

Listing 91: Sintaks dasar assert

```
1 public int dividePositive(int a, int b) {
2     assert b != 0 : "b tidak boleh nol pada tahap ini";
3     return a / b;
4 }
```

Pesan setelah titik dua dievaluasi hanya jika assertion gagal. Pesan dapat berupa string atau object lain yang akan dikonversi menjadi teks.

15.3.2 Mengaktifkan Assertion

Inti Konsep.

Assertion default-nya tidak aktif. Jalankan program dengan `-ea` atau `-enableassertions` agar assertion dievaluasi.

Listing 92: Menjalankan program dengan assertion aktif

```
1 javac Main.java
2 java -ea Main
```

Karena assertion dapat tidak aktif, jangan menaruh side effect penting di dalam ekspresi `assert`.

Listing 93: Assertion dengan side effect yang buruk

```
1 // Buruk: remove hanya berjalan jika assertion aktif.
2 assert list.remove(value);
```

15.3.3 Assertion vs Exception

Inti Konsep.

Gunakan exception untuk validasi yang harus selalu berjalan; gunakan assertion untuk asumsi internal yang seharusnya tidak salah jika kode benar.

Listing 94: Exception untuk validasi public method

```
1 public void withdraw(long amount) {
2     if (amount <= 0) {
3         throw new IllegalArgumentException("Amount harus positif");
4     }
5     balance -= amount;
```

```

6     assert balance >= 0 : "Invariant saldo tidak boleh negatif";
7 }

```

Parameter `amount` berasal dari pemanggil, sehingga harus divalidasi dengan exception. Setelah operasi, assertion dapat dipakai untuk memeriksa invariant internal.

15.4 Komponen Kunci Penting

Komponen Kunci	Makna atau Peran
<code>assert</code>	Kata kunci untuk memeriksa asumsi internal.
<code>AssertionError</code>	Error yang dilempar ketika assertion gagal.
<code>-ea</code>	Opsi JVM untuk mengaktifkan assertion.
Invariant	Kondisi internal object yang harus selalu benar.
Precondition	Syarat yang harus dipenuhi sebelum operasi.
Postcondition	Syarat yang harus benar setelah operasi.

Tabel 20: Komponen kunci dalam materi Java Assertion

15.5 Algoritma dan Rumus Penting

15.5.1 Prosedur Menentukan Assert atau Exception

Tujuan Algoritma.

Prosedur ini mencegah penggunaan assertion untuk validasi yang seharusnya selalu aktif.

Listing 95: Pseudocode memilih assert atau exception

```

1 Input: Kondisi yang ingin diperiksa
2 Output: assert atau exception
3
4 1. Jika kondisi berasal dari input pengguna atau public API:
5     gunakan exception.
6 2. Jika kondisi harus tetap dicek di production:
7     gunakan exception.
8 3. Jika kondisi adalah asumsi internal yang seharusnya selalu benar:
9     gunakan assert.
10 4. Jika ekspresi pemeriksaan memiliki side effect penting:
11     jangan gunakan assert.
12 5. Jika kegagalan perlu ditangani pemanggil:
13     gunakan exception.

```

15.6 Checklist Pemahaman

15.6.1 Submateri yang Telah Dibahas

- ☐ Saya dapat menulis sintaks `assert`.
- ☐ Saya dapat mengaktifkan assertion dengan `-ea`.
- ☐ Saya dapat membedakan assertion dan exception.
- ☐ Saya dapat menjelaskan mengapa `assert` tidak boleh dipakai untuk validasi input publik.
- ☐ Saya dapat menghindari side effect dalam assertion.

15.6.2 Prioritas Belajar

Prioritas	Materi yang Perlu dikuasai
Wajib dikuasai	Sintaks assert, aktivasi <code>-ea</code> , <code>AssertionError</code> , assert vs exception.
Cukup Pahami Konsep	Invariant, precondition, postcondition, risiko side effect.
Sudah dikuasai	Belum ditentukan; materi ini tidak disebut sebagai materi yang sudah diuji dalam kuis.

16 Java Multithreading

Java Multithreading membahas eksekusi beberapa thread dalam satu program. Thread memungkinkan program melakukan pekerjaan secara konkuren, misalnya menangani banyak request, menjalankan pekerjaan background, atau memproses data paralel. Namun, multithreading juga membawa risiko race condition, deadlock, visibility problem, dan bug nondeterministik yang sulit direproduksi.

Komentar 16.1: Keterbatasan Sumber

Section ini disusun menggunakan pengetahuan umum Java dan dibuat lebih mendalam karena materi multithreading disebut sebagai materi baru. Fokusnya adalah konsep fundamental, sintaks Java, dan risiko desain.

16.1 Outline Fundamental Konsep Materi

1. Konsep thread dan concurrency
 - (a) Thread adalah alur eksekusi ringan dalam process.
 - (b) Concurrency bukan selalu parallelism.
 - (c) Scheduler menentukan kapan thread berjalan.
2. Membuat task
 - (a) `Thread`.
 - (b) `Runnable`.
 - (c) `Callable`.
 - (d) `ExecutorService`.
3. Shared state dan race condition
 - (a) Data bersama dapat diakses bersamaan.
 - (b) Operasi seperti increment tidak atomik.
4. Sinkronisasi
 - (a) `synchronized`.
 - (b) Monitor lock.
 - (c) `Lock`.
 - (d) Atomic classes.
5. Visibility dan Java Memory Model
 - (a) Perubahan satu thread belum tentu terlihat thread lain tanpa mekanisme sinkronisasi.
 - (b) `volatile`.
6. Koordinasi thread
 - (a) `join`.
 - (b) `wait/notify`.
 - (c) `Future`.
7. Risiko
 - (a) Deadlock.
 - (b) Starvation.
 - (c) Livelock.
 - (d) Data race.

16.2 Pentingnya Materi dan Relevansinya

Inti Relevansi.

Multithreading penting karena aplikasi modern sering menangani banyak pekerjaan sekaligus. Namun, kebenaran program konkuren tidak cukup diuji dengan “program tampak berjalan”, karena bug dapat muncul hanya pada interleaving tertentu.

Dalam Java, multithreading muncul di backend server, GUI, worker background, scheduled task, parallel processing, dan async IO. Untuk ujian, kemampuan utama adalah membaca kode yang memakai shared state dan menentukan apakah terjadi race condition atau perlu sinkronisasi.

16.2.1 Dependensi dan Hubungan dengan Materi Lain

Multithreading bergantung pada object, method, exception, collection, dan stream. Collection biasa seperti `ArrayList` tidak otomatis thread-safe. Exception pada thread harus ditangani di thread tersebut atau melalui `Future`. Stream memiliki parallel stream, tetapi side effect harus dihindari.

16.3 Penjelasan Konsep Fundamental

16.3.1 Thread, Runnable, dan Executor

Inti Konsep.

Thread menjalankan task. Di Java modern, task biasanya dinyatakan sebagai `Runnable` atau `Callable`, lalu dijalankan oleh `ExecutorService`.

Listing 96: Membuat thread dengan Runnable

```
1 Runnable task = () -> {
2     System.out.println("Berjalan di " + Thread.currentThread().getName());
3 };
4
5 Thread thread = new Thread(task);
6 thread.start();
```

Memanggil `start()` membuat thread baru. Memanggil `run()` langsung hanya menjalankan method biasa di thread saat ini.

Listing 97: ExecutorService untuk menjalankan task

```
1 ExecutorService executor = Executors.newFixedThreadPool(4);
2
3 Future<Integer> future = executor.submit(() -> {
4     return 10 + 20;
5 });
6
7 Integer result = future.get();
8 executor.shutdown();
```

16.3.2 Race Condition

Inti Konsep.

Race condition terjadi ketika hasil program bergantung pada urutan eksekusi thread yang tidak terkendali. Operasi terhadap shared mutable state harus dibuat aman.

Listing 98: Race condition pada increment

```
1 class Counter {
2     private int value = 0;
3
4     public void increment() {
5         value++; // Tidak atomik.
6     }
7
8     public int value() {
9         return value;
10    }
11 }
```

`value++` terdiri dari baca nilai, tambah satu, lalu tulis kembali. Dua thread dapat membaca nilai sama dan menulis hasil sama, sehingga satu increment hilang.

16.3.3 synchronized dan Atomic

Inti Konsep.

`synchronized` memastikan hanya satu thread masuk critical section pada monitor yang sama dan juga memberi jaminan visibility. Atomic class menyediakan operasi atomik untuk kasus tertentu.

Listing 99: Counter thread-safe dengan synchronized

```
1 class SafeCounter {
2     private int value = 0;
3
4     public synchronized void increment() {
5         value++;
6     }
7
8     public synchronized int value() {
9         return value;
10    }
11 }
```

Listing 100: Counter thread-safe dengan AtomicInteger

```
1 class AtomicCounter {
2     private final AtomicInteger value = new AtomicInteger();
3
4     public void increment() {
5         value.incrementAndGet();
6     }
7
8     public int value() {
9         return value.get();
10    }
11 }
```

16.3.4 volatile dan Visibility

Inti Konsep.

`volatile` memastikan pembacaan variabel melihat nilai terbaru yang ditulis thread lain. Namun, `volatile` tidak membuat operasi gabungan seperti increment menjadi atomik.

Listing 101: volatile untuk flag berhenti

```

1 class Worker implements Runnable {
2     private volatile boolean running = true;
3
4     public void stop() {
5         running = false;
6     }
7
8     @Override
9     public void run() {
10         while (running) {
11             doWork();
12         }
13     }
14
15     private void doWork() { }
16 }

```

16.3.5 Deadlock

Inti Konsep.

Deadlock terjadi ketika dua atau lebih thread saling menunggu lock yang tidak akan pernah dilepas karena masing-masing memegang lock yang dibutuhkan thread lain.

Listing 102: Pola deadlock konseptual

```

1 Thread 1:
2 1. lock A
3 2. menunggu lock B
4
5 Thread 2:
6 1. lock B
7 2. menunggu lock A
8
9 Hasil:
10 Keduanya menunggu selamanya.

```

Cara mengurangi risiko deadlock adalah memakai urutan pengambilan lock yang konsisten, mengurangi shared mutable state, memakai timeout lock, atau memakai struktur concurrent yang sudah tersedia.

16.4 Komponen Kunci Penting

Komponen Kunci	Makna atau Peran
Thread	Alur eksekusi konkuren.
Runnable	Task tanpa return value.
Callable	Task dengan return value dan dapat melempar exception.
ExecutorService	Pengelola eksekusi task berbasis thread pool.
synchronized	Mekanisme mutual exclusion dan visibility.
volatile	Visibility untuk variabel sederhana.
AtomicInteger	Operasi atomik untuk integer.
Future	Representasi hasil task asynchronous.
Race condition	Bug akibat urutan eksekusi thread yang tidak terkendali.
Deadlock	Kondisi thread saling menunggu lock selamanya.

Tabel 21: Komponen kunci dalam materi Java Multithreading

16.5 Algoritma dan Rumus Penting

16.5.1 Prosedur Mendeteksi Risiko Race Condition

Tujuan Algoritma.

Prosedur ini membantu membaca kode konkuren dan menentukan apakah shared state perlu dilindungi.

Listing 103: Pseudocode deteksi race condition

```
1 Input: Kode yang dijalankan oleh beberapa thread
2 Output: Risiko race condition dan mitigasi
3
4 1. Identifikasi semua data yang dapat diakses lebih dari satu thread.
5 2. Untuk setiap data:
6     periksa apakah data mutable.
7 3. Jika data immutable:
8     risiko race rendah.
9 4. Jika data mutable:
10    periksa apakah semua akses dilindungi mekanisme yang sama.
11 5. Jika ada operasi read-modify-write tanpa sinkronisasi:
12    tandai race condition.
13 6. Pilih mitigasi:
14    synchronized, Lock, atomic class, concurrent collection,
15    immutability, atau thread confinement.
```

16.6 Checklist Pemahaman

16.6.1 Submateri yang Telah Dibahas

- ☐ Saya dapat membedakan `start()` dan `run()`.
- ☐ Saya dapat memakai `Runnable`, `Callable`, dan `ExecutorService`.
- ☐ Saya dapat menjelaskan race condition.
- ☐ Saya dapat memakai `synchronized` dan `AtomicInteger`.
- ☐ Saya dapat menjelaskan `volatile`.
- ☐ Saya dapat menjelaskan deadlock dan cara mengurangnya.

16.6.2 Prioritas Belajar

Prioritas	Materi yang Perlu dikuasai
Wajib dikuasai	Thread, Runnable, ExecutorService, race condition, synchronized, volatile, AtomicInteger, deadlock.
Cukup Pahami Konsep	Future, Callable, visibility, thread pool, concurrent collection, parallel stream.
Sudah dikuasai	Belum ditentukan; materi ini disebut sebagai materi baru sehingga harus diprioritaskan tinggi.

17 Java Reflection

Java Reflection adalah kemampuan program Java untuk memeriksa dan memanipulasi metadata class, field, method, constructor, annotation, dan tipe saat runtime. Dengan reflection, program dapat mengetahui class apa yang sedang dipakai, method apa saja yang tersedia, membuat object secara dinamis, atau memanggil method berdasarkan nama. Reflection banyak dipakai oleh framework seperti dependency injection, serialization, ORM, testing framework, dan annotation processor runtime.

Komentar 17.1: Keterbatasan Sumber

Section ini disusun menggunakan pengetahuan umum Java. Reflection dibahas dari sisi konsep runtime, sintaks dasar, manfaat, dan risiko terhadap enkapsulasi.

17.1 Outline Fundamental Konsep Materi

1. Konsep reflection
 - (a) Inspeksi metadata runtime.
 - (b) Class object sebagai representasi metadata tipe.
 - (c) Package utama: `java.lang.reflect`.
2. Mendapatkan `Class<?>`
 - (a) `Type.class`.
 - (b) `object.getClass()`.
 - (c) `Class.forName`.
3. Inspeksi member
 - (a) Field.
 - (b) Method.
 - (c) Constructor.
 - (d) Modifier.
4. Instansiasi dan invocation dinamis
 - (a) Membuat object melalui constructor.
 - (b) Memanggil method melalui `Method.invoke`.
 - (c) Mengakses field melalui `Field`.
5. Annotation runtime
 - (a) Annotation dengan retention runtime.
 - (b) Membaca annotation melalui reflection.
6. Risiko reflection
 - (a) Melanggar enkapsulasi.
 - (b) Lebih lambat dari pemanggilan langsung.
 - (c) Rentan error runtime.
 - (d) Dipengaruhi security dan module system.

17.2 Pentingnya Materi dan Relevansinya

Inti Relevansi.

Reflection penting karena banyak framework Java bekerja dengan membaca metadata class saat runtime. Namun, reflection harus dipahami sebagai alat khusus, bukan pengganti pemanggilan method normal.

Tanpa reflection, framework seperti JUnit sulit menemukan method test berdasarkan annotation, serializer sulit membaca field object secara generik, dan dependency injection container sulit membuat object berdasarkan constructor. Tetapi reflection juga membuat kode lebih rapuh karena nama method/field menjadi string, error muncul saat runtime, dan akses private dapat merusak enkapsulasi.

17.2.1 Dependensi dan Hubungan dengan Materi Lain

Reflection bergantung pada class, method, access modifier, exception, annotation, dan generics. Reflection berhubungan dengan Design Pattern seperti Factory dan Dependency Injection. Type erasure pada generics membatasi informasi tipe yang tersedia di runtime.

17.3 Penjelasan Konsep Fundamental

17.3.1 Class Object

Inti Konsep.

Setiap tipe Java memiliki object metadata bertipe `Class<?>`. Object ini menjadi pintu masuk reflection untuk membaca nama class, superclass, interface, field, method, dan constructor.

Listing 104: Mendapatkan Class object

```
1 Class<String> stringClass = String.class;
2
3 String text = "Java";
4 Class<?> runtimeClass = text.getClass();
5
6 Class<?> loaded = Class.forName("java.lang.String");
```

`Class.forName` memakai nama fully qualified dan dapat melempar `ClassNotFoundException`. Ini umum pada plugin system atau framework yang memuat class berdasarkan konfigurasi.

17.3.2 Inspeksi Method dan Field

Inti Konsep.

Reflection dapat membaca daftar method, field, constructor, dan modifier. Method publik diwarisi dapat diambil dengan `getMethods`; method yang dideklarasikan langsung dapat diambil dengan `getDeclaredMethods`.

Listing 105: Inspeksi method dengan reflection

```
1 Class<?> clazz = BankAccount.class;
2
3 for (Method method : clazz.getDeclaredMethods()) {
4     System.out.println(method.getName());
5     System.out.println(Modifier.toString(method.getModifiers()));
6 }
```

Perbedaan penting:

- `getField`: field publik, termasuk yang diwarisi.

- `getDeclaredField`: field yang dideklarasikan di class tersebut, termasuk private.
- `getMethod`: method publik.
- `getDeclaredMethod`: method yang dideklarasikan di class tersebut.

17.3.3 Membuat Object dan Memanggil Method

Inti Konsep.

Reflection dapat membuat object dan memanggil method secara dinamis, tetapi hasilnya lebih rawan error runtime dibanding pemanggilan biasa.

Listing 106: Instansiasi dan pemanggilan method dengan reflection

```
1 Constructor<User> constructor =
2     User.class.getConstructor(String.class, String.class);
3
4 User user = constructor.newInstance("13524044", "Narendra");
5
6 Method method = User.class.getMethod("name");
7 String name = (String) method.invoke(user);
```

Operasi reflection dapat melempar banyak exception, misalnya method tidak ditemukan, instansiasi gagal, akses ilegal, atau invocation gagal. Invocation exception membungkus exception yang dilempar method yang dipanggil.

17.3.4 Mengakses Private Member

Inti Konsep.

Reflection dapat mencoba mengakses private member melalui `setAccessible(true)`, tetapi ini melanggar enkapsulasi dan dapat dibatasi oleh module/security policy.

Listing 107: Akses private field dengan reflection

```
1 Field field = User.class.getDeclaredField("name");
2 field.setAccessible(true);
3
4 String name = (String) field.get(user);
```

Dalam kode produksi biasa, akses private seperti ini sebaiknya dihindari. Jika framework melakukannya, biasanya ada alasan infrastruktur yang kuat. Untuk desain OOP, kebutuhan mengakses private field sering menjadi tanda API class belum dirancang dengan tepat.

17.3.5 Annotation Runtime

Inti Konsep.

Reflection dapat membaca annotation yang memiliki retention runtime. Ini memungkinkan framework mengenali metadata deklaratif.

Listing 108: Annotation runtime dan pembacaannya

```
1 @Retention(RetentionPolicy.RUNTIME)
2 @Target(ElementType.METHOD)
3 public @interface Route {
4     String path();
5 }
6
```

```

7 public class UserController {
8     @Route(path = "/users")
9     public void listUsers() { }
10 }
11
12 for (Method method : UserController.class.getDeclaredMethods()) {
13     Route route = method.getAnnotation(Route.class);
14     if (route != null) {
15         System.out.println(route.path());
16     }
17 }

```

17.4 Komponen Kunci Penting

Komponen Kunci	Makna atau Peran
Class<?>	Object metadata untuk tipe Java.
Field	Representasi field melalui reflection.
Method	Representasi method melalui reflection.
Constructor	Representasi constructor melalui reflection.
Modifier	Utility untuk membaca modifier seperti public/private/static.
setAccessible	Mekanisme untuk mencoba membuka akses member non-public.
Invocation exception	Exception pembungkus ketika method reflektif melempar exception.
Runtime annotation	Annotation yang dapat dibaca saat program berjalan.

Tabel 22: Komponen kunci dalam materi Java Reflection

17.5 Algoritma dan Rumus Penting

17.5.1 Prosedur Menggunakan Reflection

Tujuan Algoritma.

Prosedur ini membantu memakai reflection secara aman dan terbatas.

Listing 109: Pseudocode penggunaan reflection

```

1 Input: Nama class/member atau object runtime
2 Output: Metadata atau operasi dinamis
3
4 1. Dapatkan Class object.
5 2. Tentukan apakah perlu field, method, atau constructor.
6 3. Gunakan getX untuk public member atau getDeclaredX untuk member deklarasi
   class.
7 4. Jika member non-public benar-benar perlu:
8     pertimbangkan risiko enkapsulasi sebelum setAccessible.
9 5. Jalankan operasi reflection.
10 6. Tangani exception reflection.
11 7. Jika reflection hanya dipakai untuk menghindari desain interface:
12     pertimbangkan refactor ke polymorphism biasa.

```

17.6 Checklist Pemahaman

17.6.1 Submateri yang Telah Dibahas

- ☐ Saya dapat menjelaskan apa itu reflection.
- ☐ Saya dapat memperoleh Class<?> melalui .class, getClass, dan Class.forName.

- ☐ Saya dapat membaca method, field, dan constructor secara reflektif.
- ☐ Saya dapat menjelaskan risiko `setAccessible(true)`.
- ☐ Saya dapat menjelaskan penggunaan annotation runtime.
- ☐ Saya dapat menyebutkan risiko performa, enkapsulasi, dan runtime error.

17.6.2 Prioritas Belajar

Prioritas	Materi yang Perlu dikuasai
Wajib dikuasai	<code>Class<?></code> , <code>Field</code> , <code>Method</code> , <code>Constructor</code> , <code>Class.forName</code> , invocation, annotation runtime, risiko reflection.
Cukup Pahami Konsep	Module/security limitation, type erasure, framework use case, <code>InvocationTargetException</code> .
Sudah dikuasai	Belum ditentukan; materi ini perlu dipahami sebagai materi lanjut setelah Java runtime dan OOP dasar.

18 Design Pattern

Design Pattern adalah pola solusi yang sering muncul untuk masalah desain perangkat lunak berorientasi objek. Pattern bukan potongan kode yang harus dihafal mentah, melainkan nama dan struktur untuk solusi yang terbukti berguna dalam konteks tertentu. Materi ini harus dipahami bersama SOLID: SOLID membantu mengevaluasi kualitas desain, sedangkan design pattern memberi bentuk implementasi yang sering memenuhi prinsip tersebut.

Komentar 18.1: Keterbatasan Sumber

Section ini disusun menggunakan pengetahuan umum design pattern. Karena daftar pattern yang mungkin diajarkan tidak disebutkan rinci, catatan ini memilih pattern yang paling umum dan paling relevan untuk Java OOP: Singleton, Factory Method, Strategy, Observer, Adapter, Template Method, Decorator, dan Builder.

18.1 Outline Fundamental Konsep Materi

1. Konsep design pattern
 - (a) Solusi berulang untuk masalah desain berulang.
 - (b) Bukan kewajiban; dipakai ketika masalahnya cocok.
 - (c) Pattern memiliki intent, struktur, konsekuensi, dan tradeoff.
2. Kategori pattern
 - (a) Creational: pembuatan object.
 - (b) Structural: komposisi object/class.
 - (c) Behavioral: komunikasi dan variasi perilaku.
3. Pattern creational
 - (a) Singleton.
 - (b) Factory Method.
 - (c) Builder.
4. Pattern structural
 - (a) Adapter.
 - (b) Decorator.
5. Pattern behavioral
 - (a) Strategy.
 - (b) Observer.
 - (c) Template Method.
6. Hubungan dengan SOLID
 - (a) Strategy mendukung OCP dan DIP.
 - (b) Factory mengurangi coupling ke class konkret.
 - (c) Adapter mendukung DIP terhadap API eksternal.
 - (d) Observer memisahkan publisher dan subscriber.

18.2 Pentingnya Materi dan Relevansinya

Inti Relevansi.

Design pattern penting karena memberi kosakata desain. Dengan menyebut “Strategy” atau “Factory”, programmer dapat menjelaskan struktur solusi tanpa menggambarkan semua detail dari awal.

Namun, pattern harus dipakai secara pragmatis. Jika masalahnya sederhana, menambahkan banyak class pattern justru membuat kode berlebihan. Dalam ujian, yang penting adalah mengenali masalah desain, memilih pattern yang cocok, menjelaskan mengapa pattern itu menyelesaikan masalah, dan menulis skeleton Java yang benar.

18.2.1 Dependensi dan Hubungan dengan Materi Lain

Design pattern memanfaatkan inheritance, interface, generics, exception, collection, dan SOLID. Strategy dan Observer sangat bergantung pada interface. Factory dan Builder berkaitan dengan constructor dan object creation. Decorator dan Adapter memakai composition. Template Method memakai inheritance dan abstract method.

18.3 Penjelasan Konsep Fundamental

18.3.1 Strategy Pattern

Inti Konsep.

Strategy memisahkan algoritma yang dapat berubah ke dalam interface sehingga klien dapat mengganti perilaku tanpa mengubah class utama.

Strategy cocok ketika ada banyak variasi algoritma, misalnya metode diskon, metode pembayaran, sorting, validasi, atau pengiriman notifikasi.

- **Pattern name:** Strategy.
- **Problem:** Gunakan ketika sebuah class memiliki perilaku atau algoritma yang dapat bervariasi, dan pemilihan algoritma tersebut sebaiknya dapat diganti tanpa mengubah class pemakai. Gejala umumnya adalah banyak `if/switch` untuk memilih cara menghitung, memvalidasi, mengurutkan, atau mengirim sesuatu.
- **Solution:** Definisikan interface strategy yang menyatakan operasi umum. Setiap variasi algoritma dibuat sebagai concrete strategy. Context menyimpan reference ke interface strategy dan mendelegasikan pekerjaan ke object strategy tersebut.
- **Consequences:** Context menjadi lebih terbuka untuk ekstensi dan lebih mudah diuji karena algoritma bisa diganti. Tradeoff-nya adalah jumlah class bertambah dan klien atau factory perlu menentukan strategy mana yang dipakai.

Listing 110: Strategy Pattern untuk diskon

```
1 public interface DiscountStrategy {
2     BigDecimal apply(BigDecimal price);
3 }
4
5 public class NoDiscount implements DiscountStrategy {
6     @Override
7     public BigDecimal apply(BigDecimal price) {
8         return price;
9     }
10 }
11
12 public class PercentageDiscount implements DiscountStrategy {
```

```

13     private final BigDecimal percent;
14
15     public PercentageDiscount(BigDecimal percent) {
16         this.percent = percent;
17     }
18
19     @Override
20     public BigDecimal apply(BigDecimal price) {
21         return price.subtract(price.multiply(percent));
22     }
23 }
24
25 public class CheckoutService {
26     private final DiscountStrategy discount;
27
28     public CheckoutService(DiscountStrategy discount) {
29         this.discount = discount;
30     }
31
32     public BigDecimal finalPrice(BigDecimal price) {
33         return discount.apply(price);
34     }
35 }

```

18.3.2 Factory Method

Inti Konsep.

Factory memindahkan logika pembuatan object ke method/class khusus sehingga klien tidak bergantung langsung pada banyak constructor konkret.

- **Pattern name:** Factory Method.
- **Problem:** Gunakan ketika klien perlu membuat object, tetapi klien tidak seharusnya mengetahui class konkret yang dibuat. Pattern ini juga cocok ketika jenis object ditentukan oleh konfigurasi, input, environment, atau subclass.
- **Solution:** Sediakan method pembuatan object yang mengembalikan tipe abstrak, misalnya interface atau superclass. Concrete factory atau subclass menentukan class konkret mana yang dibuat.
- **Consequences:** Coupling klien terhadap constructor konkret berkurang dan object creation terkonsentrasi. Tradeoff-nya adalah desain dapat tetap perlu diubah saat tipe baru ditambahkan jika factory memakai `switch` statis, sehingga perlu dikombinasikan dengan registry atau dependency injection untuk ekstensi lebih fleksibel.

Listing 111: Factory sederhana untuk parser

```

1 public interface Parser {
2     Document parse(String text);
3 }
4
5 public class JsonParser implements Parser {
6     public Document parse(String text) { return new Document(); }
7 }
8
9 public class XmlParser implements Parser {
10    public Document parse(String text) { return new Document(); }
11 }
12
13 public class ParserFactory {

```



```

14     public static Parser create(String format) {
15         return switch (format) {
16             case "json" -> new JsonParser();
17             case "xml" -> new XmlParser();
18             default -> throw new IllegalArgumentException("Format tidak dikenal");
19         };
20     }
21 }

```

Factory sederhana masih dapat melanggar OCP jika setiap tipe baru harus mengubah switch. Namun, factory tetap berguna untuk memusatkan keputusan object creation di satu lokasi.

18.3.3 Builder Pattern

Inti Konsep.

Builder membantu membuat object kompleks dengan banyak parameter opsional tanpa constructor panjang dan membingungkan.

- **Pattern name:** Builder.
- **Problem:** Gunakan ketika pembuatan object membutuhkan banyak parameter, sebagian opsional, atau memiliki kombinasi konfigurasi yang membuat constructor terlalu panjang dan rawan salah urutan.
- **Solution:** Pisahkan proses konstruksi ke object builder. Builder menyediakan method bertahap untuk mengisi konfigurasi, lalu method `build()` membuat object final yang valid.
- **Consequences:** Kode pembuatan object menjadi lebih ekspresif dan object dapat dibuat immutable. Tradeoff-nya adalah ada class tambahan dan validasi harus dirancang jelas, terutama apakah validasi dilakukan saat setter builder dipanggil atau saat `build()`.

Listing 112: Builder Pattern

```

1 public class ReportConfig {
2     private final String title;
3     private final boolean includeSummary;
4     private final int maxRows;
5
6     private ReportConfig(Builder builder) {
7         this.title = builder.title;
8         this.includeSummary = builder.includeSummary;
9         this.maxRows = builder.maxRows;
10    }
11
12    public static class Builder {
13        private String title = "Untitled";
14        private boolean includeSummary = true;
15        private int maxRows = 100;
16
17        public Builder title(String title) {
18            this.title = title;
19            return this;
20        }
21
22        public Builder includeSummary(boolean includeSummary) {
23            this.includeSummary = includeSummary;
24            return this;
25        }
26
27        public Builder maxRows(int maxRows) {

```

```

28         this.maxRows = maxRows;
29         return this;
30     }
31
32     public ReportConfig build() {
33         return new ReportConfig(this);
34     }
35 }
36 }

```

18.3.4 Adapter Pattern

Inti Konsep.

Adapter mengubah interface suatu class agar cocok dengan interface yang dibutuhkan klien. Pattern ini berguna ketika memakai library eksternal yang API-nya tidak sesuai domain kita.

- **Pattern name:** Adapter.
- **Problem:** Gunakan ketika class atau library yang ingin dipakai memiliki interface yang tidak cocok dengan interface yang dibutuhkan sistem. Masalah ini umum terjadi saat mengintegrasikan API eksternal, legacy code, atau library pihak ketiga.
- **Solution:** Buat adapter yang mengimplementasikan interface target milik sistem. Adapter menyimpan adaptee, yaitu object/library asli, lalu menerjemahkan pemanggilan dari interface target ke method adaptee.
- **Consequences:** Klien tetap bergantung pada interface domain yang stabil dan detail API eksternal terisolasi. Tradeoff-nya adalah ada lapisan tambahan, dan adapter dapat menjadi tempat kompleks jika perbedaan model data antara sistem dan library terlalu besar.

Listing 113: Adapter Pattern

```

1 public interface Notifier {
2     void notify(String recipient, String message);
3 }
4
5 public class ExternalSmsClient {
6     public void sendSms(String phoneNumber, String text) { }
7 }
8
9 public class SmsNotifierAdapter implements Notifier {
10     private final ExternalSmsClient client;
11
12     public SmsNotifierAdapter(ExternalSmsClient client) {
13         this.client = client;
14     }
15
16     @Override
17     public void notify(String recipient, String message) {
18         client.sendSms(recipient, message);
19     }
20 }

```

18.3.5 Decorator Pattern

Inti Konsep.

Decorator menambahkan perilaku pada object secara dinamis dengan membungkus object yang memiliki interface sama.

- **Pattern name:** Decorator.
- **Problem:** Gunakan ketika perilaku tambahan perlu dikombinasikan secara fleksibel tanpa membuat banyak subclass untuk setiap kombinasi fitur. Contohnya logging, caching, authorization, compression, atau encryption.
- **Solution:** Definisikan component interface. Concrete component melakukan pekerjaan dasar. Decorator juga mengimplementasikan interface yang sama dan menyimpan component lain sebagai delegate, lalu menambahkan perilaku sebelum atau sesudah delegasi.
- **Consequences:** Perilaku dapat ditumpuk secara dinamis dan mendukung OCP. Tradeoff-nya adalah debugging dapat lebih sulit karena banyak wrapper, dan urutan decorator dapat memengaruhi hasil.

Listing 114: Decorator Pattern untuk logging

```
1 public interface Repository {
2     void save(Order order);
3 }
4
5 public class DatabaseRepository implements Repository {
6     public void save(Order order) { }
7 }
8
9 public class LoggingRepository implements Repository {
10     private final Repository delegate;
11
12     public LoggingRepository(Repository delegate) {
13         this.delegate = delegate;
14     }
15
16     public void save(Order order) {
17         System.out.println("Saving order");
18         delegate.save(order);
19     }
20 }
```

18.3.6 Observer Pattern

Inti Konsep.

Observer memisahkan publisher dari subscriber. Ketika state/event berubah, publisher memberi tahu semua observer tanpa mengetahui detail implementasinya.

- **Pattern name:** Observer.
- **Problem:** Gunakan ketika perubahan pada satu object perlu memberi tahu banyak object lain, tetapi publisher tidak boleh bergantung langsung pada class konkret subscriber. Contohnya event UI, domain event, notifikasi, atau sistem publish-subscribe sederhana.
- **Solution:** Publisher menyimpan daftar observer melalui interface listener. Observer mendaftar ke publisher. Saat event terjadi, publisher memanggil method callback pada setiap observer.

- **Consequences:** Publisher dan subscriber menjadi loosely coupled dan subscriber dapat ditambah/dihapus dinamis. Tradeoff-nya adalah alur program menjadi tidak langsung, urutan notifikasi perlu diperhatikan, dan subscriber yang lambat atau error dapat memengaruhi event dispatch jika tidak ditangani.

Listing 115: Observer Pattern sederhana

```

1 public interface OrderListener {
2     void onOrderCreated(Order order);
3 }
4
5 public class OrderService {
6     private final List<OrderListener> listeners = new ArrayList<>();
7
8     public void addListener(OrderListener listener) {
9         listeners.add(listener);
10    }
11
12    public void create(Order order) {
13        // Simpan order.
14        for (OrderListener listener : listeners) {
15            listener.onOrderCreated(order);
16        }
17    }
18 }

```

18.3.7 Template Method

Inti Konsep.

Template Method mendefinisikan kerangka algoritma di superclass dan menyerahkan beberapa langkah ke subclass.

- **Pattern name:** Template Method.
- **Problem:** Gunakan ketika beberapa algoritma memiliki urutan langkah yang sama, tetapi detail beberapa langkah berbeda antarvarian. Pattern ini cocok jika kerangka proses harus tetap konsisten.
- **Solution:** Superclass mendefinisikan method template, biasanya **final**, yang memanggil langkah-langkah algoritma dalam urutan tetap. Langkah tertentu dibuat sebagai abstract method atau hook method yang dioverride subclass.
- **Consequences:** Duplikasi kerangka algoritma berkurang dan urutan proses dikontrol superclass. Tradeoff-nya adalah subclass terikat kuat pada superclass, sehingga perubahan superclass dapat memengaruhi semua subclass dan risiko pelanggaran LSP harus dijaga.

Listing 116: Template Method Pattern

```

1 public abstract class DataImporter {
2     public final void importData(Path path) throws IOException {
3         String raw = Files.readString(path);
4         List<Record> records = parse(raw);
5         validate(records);
6         save(records);
7     }
8
9     protected abstract List<Record> parse(String raw);
10
11     protected void validate(List<Record> records) {

```

```

12         if (records.isEmpty()) {
13             throw new IllegalArgumentException("Data kosong");
14         }
15     }
16
17     protected abstract void save(List<Record> records);
18 }

```

18.3.8 Singleton Pattern

Inti Konsep.

Singleton memastikan hanya ada satu instance dari sebuah class dan menyediakan akses global ke instance tersebut. Pattern ini harus dipakai hati-hati karena akses global dapat menyulitkan testing.

- **Pattern name:** Singleton.
- **Problem:** Gunakan ketika secara konseptual hanya boleh ada satu instance yang mengoordinasikan resource atau konfigurasi tertentu, dan instance tersebut perlu diakses dari beberapa tempat. Contohnya registry sederhana atau konfigurasi immutable.
- **Solution:** Batasi pembuatan object melalui constructor privat atau enum, simpan satu instance global, dan sediakan akses terkontrol ke instance tersebut.
- **Consequences:** Menjamin satu instance dan akses mudah. Tradeoff-nya adalah global state dapat menyulitkan testing, menyembunyikan dependency, memperbesar coupling, dan berbahaya jika instance menyimpan mutable state yang diakses banyak thread.

Listing 117: Singleton sederhana dengan enum

```

1 public enum AppConfig {
2     INSTANCE;
3
4     public String appName() {
5         return "IF2010";
6     }
7 }

```

Di Java, enum singleton sering dianggap aman terhadap serialization dan reflection tertentu. Namun, singleton yang menyimpan mutable global state tetap berisiko tinggi.

18.4 Komponen Kunci Penting

Komponen Kunci	Makna atau Peran
Creational pattern	Pattern untuk pembuatan object.
Structural pattern	Pattern untuk menyusun relasi antarobject/class.
Behavioral pattern	Pattern untuk variasi perilaku dan komunikasi object.
Strategy	Memisahkan algoritma yang dapat diganti.
Factory	Memusatkan pembuatan object.
Builder	Membuat object kompleks secara bertahap.
Adapter	Menyesuaikan interface class/library lain.
Decorator	Menambah perilaku dengan wrapper.
Observer	Publisher memberi tahu subscriber.
Template Method	Superclass mendefinisikan kerangka algoritma.
Singleton	Membatasi instance menjadi satu.

Tabel 23: Komponen kunci dalam materi Design Pattern

18.5 Algoritma dan Rumus Penting

18.5.1 Prosedur Memilih Design Pattern

Tujuan Algoritma.

Prosedur ini membantu memilih pattern berdasarkan masalah desain, bukan berdasarkan hafalan nama pattern.

Listing 118: Pseudocode pemilihan design pattern

```
1 Input: Masalah desain OOP
2 Output: Pattern kandidat
3
4 1. Jika banyak variasi algoritma dan klien harus bisa mengganti perilaku:
5     pertimbangkan Strategy.
6 2. Jika object creation kompleks atau bergantung konfigurasi:
7     pertimbangkan Factory atau Builder.
8 3. Jika API eksternal tidak cocok dengan interface domain:
9     pertimbangkan Adapter.
10 4. Jika ingin menambah perilaku tanpa subclass berlapis:
11     pertimbangkan Decorator.
12 5. Jika satu event perlu memberi tahu banyak subscriber:
13     pertimbangkan Observer.
14 6. Jika kerangka algoritma tetap tetapi langkah tertentu bervariasi:
15     pertimbangkan Template Method.
16 7. Jika hanya boleh ada satu instance:
17     pertimbangkan Singleton, tetapi evaluasi risiko global state.
18 8. Jika pattern membuat desain lebih rumit daripada masalahnya:
19     jangan pakai pattern tersebut.
```

18.6 Checklist Pemahaman

18.6.1 Submateri yang Telah Dibahas

- ☐ Saya dapat menjelaskan fungsi design pattern dan kategorinya.
- ☐ Saya dapat mengenali Strategy, Factory, Builder, Adapter, Decorator, Observer, Template Method, dan Singleton.
- ☐ Saya dapat menulis skeleton Java untuk pattern utama.
- ☐ Saya dapat menghubungkan pattern dengan SOLID.
- ☐ Saya dapat menjelaskan tradeoff pattern dan risiko overengineering.

18.6.2 Prioritas Belajar

Prioritas	Materi yang Perlu dikuasai
Wajib dikuasai	Strategy, Factory, Adapter, Observer, Template Method, hubungan pattern dengan SOLID, skeleton Java pattern.
Cukup Pahami Konsep	Builder, Decorator, Singleton, kategori creational/structural/behavioral, tradeoff pattern.
Sudah dikuasai	Belum ditentukan; materi ini berada di akhir daftar UAS sehingga perlu dipakai sebagai sintesis dari SOLID, interface, inheritance, dan exception.